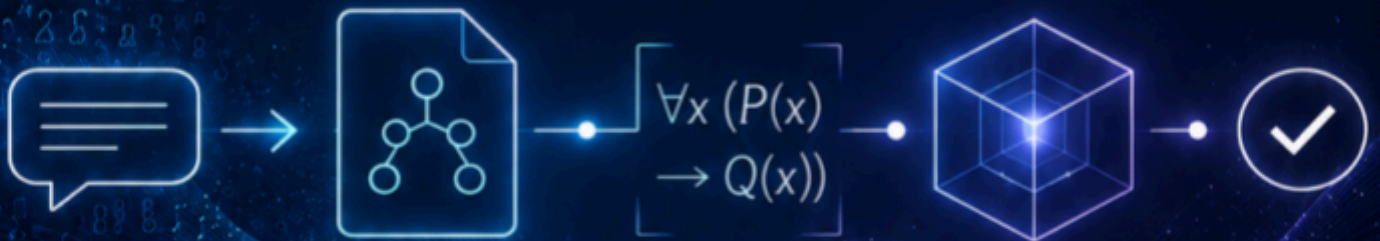


LINGUAGEM NATURAL EXECUTÁVEL

Um Levantamento Crítico de Semântica Formal,
Interpretação Abstrata, Execução Simbólica e
Arquiteturas de LLM Simbolicamente Fundamentadas



LINGUAGEM NATURAL EXECUTÁVEL

Um Levantamento Crítico de Semântica Formal, Interpretação
Abstrata, Execução Simbólica e Arquiteturas de LLM
Simbolicamente Fundamentadas

*Rumo a um Runtime Neuro-Simbólico para Documentos Longos, Regras e
Sistemas Agênticos*



Direitos autorais © [2026] Axiologic Research

Todos os direitos reservados.

Direitos de publicação KDP detidos pela Outfinity SRL (Iași, Romênia).

Disponível gratuitamente no site da Axiologic (www.axiologic.net).

Nota do Autor

Este trabalho foi criado sob a égide da Axiologic Research como parte de dois programas de pesquisa interconectados, um dedicado à Inteligência Artificial e outro ao Outfinitismo, ambos liderados por Sînică Alboaie. Para explorar os detalhes de como esses dois programas se intersectam e se desenvolvem mutuamente, convidamos os leitores a visitar a página do Outfinity Venture Studio em www.outfinity.ch. Embora modelos

de linguagem de Inteligência Artificial tenham sido usados para auxiliar a geração de texto, a filosofia fundamental, a estrutura temática e a abordagem analítica derivam exclusivamente de décadas de estudo dedicado do autor sobre fenômenos sociais, tecnologias sociais e tecnologias duras genuínas resultantes de vários projetos de pesquisa europeus e autofinanciados. Recentemente, no âmbito do projeto de pesquisa Achilles, temos estudado em profundidade a literatura gerada por IA e sua verificação automatizada usando abordagens neuro-simbólicas; todos os livros gratuitos disponibilizados ao público fazem parte do conjunto de obras que usamos para testar nossas tecnologias mais recentes.

Sumário

Resumo	5
Prefácio: Escopo, Método e Posição Crítica	8
1. Da Linguagem Fluente à Semântica Executável	11
1.1 O que a Execução Exige e Onde Pertence a Fronteira de Confiança	11
1.2 A Lacuna de Formalização: Por que um Solucionador Correto Ainda Pode Produzir a Resposta Errada	15
2. A Linhagem Intelectual da Linguagem Executável	18
2.1 Linguagens Controladas, Semântica Formal, Discurso e Lógica Natural	18
2.2 Parsing Semântico, Consultas Executáveis, Indução de Programas e Primeiros Sistemas Neuro-Simbólicos	21
3. Interpretação Abstrata e Execução Simbólica para Linguagem	24
3.1 Interpretação Abstrata como Semântica da Incerteza Delimitada	24
3.2 Execução Simbólica, Resolução de Restrições, Verificação de Modelos e Refinamento Semântico	28
4. O que a Era dos LLMs Realmente Alcançou	32
4.1 Autoformalização, Raciocínio Assistido por Programas, Lógica, Especificações Temporais e Planejamento	32
4.2 Análise de Programas Assistida por LLM, Exploração Simbólica, Transformadores Abstratos e NLP Certificado	36
5. Por que os Sistemas de Linguagem Executável Falham	40
5.1 Fundamentação, Identidade, Formalizações Espúrias, Ontologias e Conhecimento de Mundo	40
5.2 Pragmática, Não Monotonicidade, Escala, Segurança e Falhas de Avaliação	44
6. Famílias de Arquiteturas para Linguagem Natural Executável	48
6.1 Projetos Concorrentes: Representações Universais, Portfólios de Formalismos, Compilação Controlada e Sistemas Orientados pela Consulta	48
6.2 Um Pipeline de Referência do Texto aos Símbolos, à Execução e a Respostas em Linguagem Natural Restrita	53
7. LongTextJS, CircuitJS e nllAgent: Um Estudo de Caso Crítico	56
7.1 O que a Arquitetura Pública Implementa e Por que seus Limites Importam	57
7.2 Avaliação Independente: Pontos Fortes, Problemas Graves Atuais e Evolução Necessária	60
8. Construindo e Validando um Runtime Neuro-Simbólico	64
8.1 Estratégia de Implementação: Um Sistema de Linguagem Executável Mínimo, mas Cientificamente Honesto	64
8.2 Questões de Pesquisa, Benchmarks e um Programa de Publicação	70
Conclusão: Da Forma Probabilística à Consequência Simbólica	76
Referências	79

Resumo

Modelos de linguagem de grande porte tornaram a linguagem natural uma interface eficaz para a computação, mas eles não tornaram a linguagem natural executável no sentido forte em que um programa, uma consulta de banco de dados, um sistema de transição ou uma teoria lógica é executável. Um modelo pode gerar uma explicação convincente ao mudar o escopo de um quantificador, omitir uma exceção, inventar uma premissa ou aplicar a mesma regra de maneira diferente em dois lugares. As ferramentas externas só melhoram a confiabilidade quando o modelo traduziu a solicitação corretamente. Um provador de teorema pode provar a formalização errada perfeitamente; um intérprete de Python pode executar um programa que acidentalmente responde ao exemplo, deturpando a pergunta; um executor simbólico pode expor um caminho real através de software, mesmo que a regra da linguagem natural usada para construir a condição de caminho tenha sido mal traduzida. O problema difícil não é apenas adicionar um solver após um LLM. É construir uma cadeia auditável da linguagem de origem para símbolos fundamentados, de símbolos fundamentados a objetos formais executáveis, e de resultados simbólicos de volta à linguagem natural, sem permitir que o modelo de linguagem final altere o que foi estabelecido.

Este livro examina as tradições relevantes como um problema de pesquisa conectado. Os fundamentos anteriores incluem linguagens naturais controladas, semântica teórico-modelar e do discurso, lógica natural, análise semântica, programação lógica, sistemas de texto para máquina, síntese de programas a partir de exemplos e linguagem, motores de regras, tabelas de decisão, lógica temporal, planejamento automatizado e sistemas neurológico-simbólicos. As fundações de análise do programa incluem interpretação abstrata, execução simbólica e concolic, análise de fluxo de dados, verificação de modelos, resolução de SAT/SMT/CSP, Datalog e avaliação do ponto de fixação, autômatos e verificação com produção de provass. A literatura LLM recente acrescenta raciocínio auxiliado por programa, prompt de solucionador declarativo, autoformalização, tradução lógica de primeira ordem, tradução lógico temporal, planejamento formal, autorreparo guiado por solucionador, análise de programas assistidos por LLM, exploração simbólica guiada por LLM e a síntese de transformadores abstratos verificados. Em todos esses corpos de trabalho, os resultados mais fortes aparecem consistentemente quando o formalismo alvo é estreito o suficiente para ter uma semântica explícita e a saída aceita é verificada por um componente independente do LLM.

A pesquisa dá especial atenção à distinção entre dois usos da interpretação abstrata. O primeiro, já estabelecido em NLP, verifica um modelo neural sobre uma família formalmente definida de perturbações textuais. Domínios de intervalo e resumo relacional podem certificar que a previsão de um classificador não muda sob cada substituição permitida por um conjunto adversário especificado. Esta é uma garantia formal genuína, mas diz respeito à estabilidade do modelo sob uma relação de perturbação construída; não certifica que as paráfrases humanas arbitrárias preservem o significado, nem que a resposta original do modelo seja verdadeira. O segundo uso é mais especulativo e mais relevante para a linguagem natural executável: tratar o conjunto de interpretações admissíveis de um documento como uma redesenha de coleta e computação conservadora pode, não pode, e conclusões desconhecidas sobre esse conjunto. Isso requer domínios explícitos para identidade, tempo, modalidade, quantidade, status de origem, força normativa e cobertura. Também requer uma garantia

condicional honesta: a análise é sólida apenas em relação ao conjunto de interpretação, ontologia, a política de fundamentação e as regras de transferência fornecidas a ela.

A execução simbólica é complementar. A interpretação abstrata pode mostrar de forma barata que uma violação é impossível sob a abstração atual ou que uma violação permanece possível. A execução simbólica pode então tentar construir uma interpretação concreta e um caminho que o realize. Se tal caminho não existe, o testemunho fracassado pode revelar qual abstração ou escolha semântica era muito grosseira. Isso sugere um padrão de pesquisa chamado refinamento semântico guiado por contra-exemplos: o contra-exemplo de um solucionado, núcleo insatisfatório ou prova fracassada é mapeado de volta para as passagens de origem exata e compromissos semânticos que o geraram; o LLM ou um ser humano é solicitado a revisar apenas esses compromissos; a análise simbólica é repetida. O LLM é usado para pesquisar o espaço de formalizações, enquanto o núcleo simbólico determina o que se segue de cada formalização aceita.

Sistemas recentes mostram fragmentos importantes dessa visão. PAL delega computação procedural em Python. O SatLM gera restrições declarativas para um motor de satisfatibilidade. Lógica-LM e LINC traduzem tarefas de raciocínio em linguagem natural em lógica formal e dependem de solucionadores simbólicos ou provadores de teoremas para dedução. NL2TL e nl2spec traduzem os requisitos em especificações temporais, com nl2spec apoiando explicitamente a correção interativa no nível da subfórmula. O trabalho de planejamento formal mostrou grandes ganhos quando o LLMs constrói instâncias de planejamento que são então resolvidas e verificadas por ferramentas de som. O SymGPT traduz as regras de ERC em linguagem natural em uma gramática restrita e usa a execução simbólica para encontrar violações concretas do contrato inteligente, relatando milhares de violações de regras e caminhos explícitos de ataque. Na outra direção, o AutoBug decompõe a análise do programa em caminhos simbólicos manipulados pelo LLM, mas permanece aproximado; A SAIL usa LLMs para procurar transformadores abstratos, enquanto as verificações semânticas e sintáticas rígidas preservam a solidez global. Esses resultados são evidências de que o LLMs são mecanismos eficazes de proposta e pesquisa em torno de núcleos formais, não evidências de que a geração probabilística em si se tornou um raciocínio formal.

Uma recente revisão sistemática de NLP neuro-simbólica argumenta que as arquiteturas federativas, nas quais os módulos neurais e simbólicos permanecem autônomos e se comunicam através de interfaces estruturadas, muitas vezes mostram ganhos maiores do que arquiteturas que meramente injetam vieses simbólicos em redes neurais. Seus autores também alertam que a vantagem aparente ainda não é conclusiva, porque os sistemas federativos e injetivos são geralmente avaliados em diferentes tipos de tarefas. Este livro adota a mesma posição disciplinada. O caso arquitetônico para a separação é forte porque esclarece o limite da confiança, permite replay independente e mantém o componente simbólico inspecionável; a alegação empírica de que uma família supera universalmente outra permanece não comprovada [Chatzikyriakidis and Lappin 2026].

O livro também fornece um estudo de caso crítico da arquitetura pública NaturalLanguageLinterAgent, ou nllAgent, e seus componentes LongTextJS e CircuitJS. A implementação separa um programa imutável do lado do documento de um programa do lado da teoria reutilizável. Ele preserva âncoras de origem exata, tipos de observação versionados, identidades de produtores, cobertura, lacunas e status epistêmico. A compilação do circuito deriva as observações efetivamente exigidas por uma regra, a publicação verifica se os

produtores compatíveis existem, testes de compatibilidade em tempo de execução se o documento concreto produziu observações e cobertura suficientes e um verificador independente controla se uma conclusão de candidato é publicada. A aprendizagem ocorre em um espaço de trabalho descartável e não pode ser publicado automaticamente. Estas são decisões de governança extraordinariamente sérias para um sistema de análise de documentos assistido por LLM. O repositório público também registra limitações não resolvidas em vez de escondê-las. Seu registro de questões sérias atuais identifica a incrementalidade do agendador limitado, a propagação incompleta de alternativas de interpretação por meio de cada caminho de execução, análise de compatibilidade conservadora para comportamento processual opaco, planejamento de consultas mínima e indexação temporal, geração deliberadamente estreita controlada e suporte de mutação que permanece específico do cenário. Estas são limitações de pesquisa credíveis: dizem respeito ao substrato semântico e de execução, não apenas o polimento de interface do usuário ausente.

Nenhuma avaliação independente revisada por pares ou revisão técnica de terceiros do nllAgent foi localizada nas buscas realizadas para esta edição. Consequentemente, a avaliação do estudo de caso é explicitamente a própria revisão arquitetônica do autor, fundamentada no repositório público, documentação, escopo executável e questões sérias autodeclaradas. O julgamento é favorável sobre a separação de evidências, teoria, verificação e publicação, mas cauteloso sobre a completude semântica. As reivindicações de implementação mais fortes atuais dizem respeito a verificações estruturais e controladas em inglês. A resolução geral de identidade, a semântica de eventos ricos, o conhecimento do mundo atual, a transferência ampla de domínios e as reivindicações de qualidade estatisticamente apoiadas permanecem no trabalho de pesquisa, em vez da capacidade estabelecida.

A proposta central do livro é um runtime de linguagem natural executável multi-formalismo. Um LLM primeiro propõe um pequeno conjunto de estruturas semânticas candidatas, cada uma vinculada a trechos de origem exatos e tipada contra uma ontologia selecionada. Um núcleo de base simbólico admite apenas objetos cuja identidade, proveniência, escopo, tempo, modalidade e evidência satisfazem contratos explícitos. Um roteador seleciona um ou mais alvos formais de acordo com a questão: avaliação relacional ou Datalog para fechamento e consultas de documentoss; tabelas de decisão para políticas; SAT/SMT/CSP para restrições; autômatos para padrões ordenados; verificação de modelo para comportamento temporal; execução simbólica para testemunhas de caminho; e assistentes de prova ou autores para reivindicações dedutivas. A execução formal produz um resultado canônico contendo fatos aceitos, candidatos rejeitados, incerteza, testemunhas, objetos de prova, limites de cobertura e suposições. Um LLM final pode explicar esse resultado, mas cada frase na explicação deve ser licenciada pela estrutura canônica e rastreável a evidências ou uma derivação verificada.

consequente, a linguagem natural executável não deve ser anunciada como a execução de uma prosa irrestrita. Deve ser definido como um processo de compilação, fundamentação, execução e realização controlados sobre um ou mais candidatos semânticos explícitos. A garantia é condicional, mas útil: dada a revisão de fonte declarada, ontologia, fundamentação, conjunto de interpretações, semântica formal e limites de execução, o resultado relatado segue e pode ser reproduzido. O principal problema de pesquisa aberta não é o solver. É a construção, comparação, refinamento e validação das formalizações em que o solucionador opera.

Prefácio: Escopo, Método e Posição Crítica

A ambição da linguagem natural executável é fácil de expressar e fácil de exagerar. Um usuário escreve uma regra, política, hipótese científica, procedimento ou pergunta em linguagem comum; um sistema entende; o cálculo segue; o resultado é retornado como linguagem comum. Os modelos de linguagem contemporânea fazem com que o início e o fim deste pipeline pareçam resolvidos. Eles podem ler prosa irrestrita, gerar código e fórmulas, invocar ferramentas e explicar saídas fluentemente. A ilusão perigosa é que o meio desapareceu. De fato, o meio contém quase todos os problemas difíceis: decidir qual significado importa, identificar objetos através de passagens, distinguir evidências explícitas de pressupostos de fundo, preservar a modalidade e exceções, selecionar um formalismo apropriado, provar que o artefato gerado está bem formado e impedir que a explicação final diga mais do que a execução estabelecida.

Um levantamento útil não pode ser um catálogo de pequenos sistemas apresentados em subseções isoladas curtas. O campo é fragmentado precisamente porque as mesmas palavras são usadas para diferentes operações. “Execução simbólica” pode significar exploração do caminho no estilo de King sobre entradas simbólicas do programa, a execução de uma forma lógica gerada em uma base de conhecimento ou simplesmente o uso de um intérprete externo. “Verificação formal” pode se referir à correção de um resultado do solucionador, à correção de um programa compilado, à fidelidade da tradução em linguagem natural ou à robustez de uma rede neural sob uma família de perturbação predefinida. “Aterro” pode significar conectar palavras a colunas de banco de dados, entidades para abranger períodos de documentos, proposições ao conhecimento do mundo ou instruções de robôs a objetos perceptivos. Um tratamento sério deve mostrar como essas camadas se relacionam e onde as garantias param.

O método de pesquisa segue, portanto, mecanismos em vez de categorias de marketing. Para cada linha de trabalho, a análise faz cinco perguntas. Que artefato é produzido a partir da linguagem? Que semântica tem independentemente do LLM? Como o artefato é fundamentado na fonte ou no ambiente? Qual componente realmente determina o resultado? O que é e o que não é garantido quando o sistema é bem-sucedido? Os artigos são tratados como evidência para suas tarefas relatadas, não como prova de uma arquitetura geral. Fortes resultados numéricos são incluídos quando esclarecem o mecanismo, mas não são transferidos para domínios com diferentes estruturas semânticas. O horizonte da pesquisa é 2 de agosto de 2026 e inclui literatura revisada por pares, procedimentos oficiais, trabalhos de pesquisa primária e o repositório e documentação público nllAgent.

O termo “revisão” é usado com cuidado. Existem agora várias pesquisas amplas, mas cada uma abrange apenas parte do problema de ponta a ponta. O levantamento de Kuhn em linguagem natural organiza idiomas de acordo com precisão, expressividade, naturalidade e simplicidade [Kuhn 2014]. Hamilton e seus colegas avaliam se o NLP neuro-simbólico realmente entregou raciocínio, transferência, interpretação, eficiência de dados e generalização fora da distribuição, enquanto pesquisas mais recentes classificam os sistemas neuro-simbólicos orientados para tarefas e orientados por dados e conhecimento [Hamilton et al. 2024; Delvecchio et al. 2025; Wang et al. 2025b]. Pesquisas de execução simbólica e análise assistida por LLM descrevem os limites de engenharia da exploração de caminhos e o crescente papel dos componentes aprendidos [Baldoni et al. 2018; Wang et al. 2025a]. Uma revisão dedicada estuda os LLMs como formalizadores para planejamento automatizado

[Tantakoun et al. 2025] e a pesquisa de autoformalização se concentra em traduzir matemática informal em formulários com verificação de máquina [Weng et al. 2025]. Um roteiro bidirecional para a engenharia de requisitos examina tanto o uso do LLMs para tornar os métodos formais acessíveis quanto o uso de métodos formais para restringir os requisitos produzidos pelo LLM. Uma pesquisa focada com trinta e cinco artigos mapeia a formalização assistida por LLM dos requisitos e especificações de software, incluindo o trabalho direcionado a Dafny, C e Java; seu escopo confirma que o suporte à tradução está crescendo mais rápido do que a garantia semântica de ponta a ponta [Beg et al. 2025]. A revisão sistemática 2026 Frontiers desenvolve uma nova taxonomia para o NLP neuro-simbólico e defende uma maior atenção aos sistemas federativos [Chatzikyriakidis and Lappin 2026].

Nenhuma revisão pesquisada unifica linguagem controlada, fundamentação semântica, interpretação abstrata, execução simbólica, seleção de formalismo, resultados de reprovação e regeneração restrita como uma arquitetura de confiança. Essa conclusão é uma síntese da literatura, em vez de uma reivindicação de inexistência exaustiva: terminologia difere entre campos e artigos relevantes são distribuídos em linguística computacional, linguagens de programação, bancos de dados, métodos formais, engenharia de software, planejamento e robótica. Quando não existe uma revisão independente, especialmente para nllAgent, o texto rotula seus julgamentos como análise original, em vez de implicar endosso externo.

A posição do livro é opinativa, mas conservadora. Ele rejeita a ideia de que um grande modelo deve ser confiável como o raciocinador final simplesmente porque sua prosa se assemelha a uma prova. Ele também rejeita a ideia oposta de que apenas um analisador semântico completo escrito à mão pode suportar a execução formal útil. Em domínios abertos, um LLM é muitas vezes o melhor mecanismo disponível para propor eventos, relações, esquemas formais e mapeamentos de candidatos. A questão arquitetônica é como restringir esse poder. A resposta mais defensável é um sistema federado com artefatos intermediários explícitos, interfaces tipadas e versionadas, execução simbólica independente, estados desconhecidos conservadores e evidências repetitivas.

Esta posição tem uma importância importante. O objetivo do sistema nem sempre é produzir uma formalização definitiva. Por vezes, o artefacto correcto é um conjunto de alternativas. Um pronome pode se referir a dois possíveis atores; uma frase legal pode admitir escopos concorrentes; um requisito pode estar incompleto; duas fontes podem discordar; um documento pode não dizer se uma exceção se aplica. Um tempo de execução simbólico que arbitrariamente escolhe uma interpretação é menos rigoroso do que um LLM que afirma abertamente a ambiguidade. A executibilidade torna-se valiosa quando pode calcular o que se segue sob cada leitura admissível, o que se segue sob pelo menos uma, cujas conclusões dependem de uma escolha disputada e quais evidências adicionais resolveriam a escolha.

A arquitetura proposta trata estruturalmente a incerteza. As pontuações de confiança podem classificar os candidatos, mas não transformam uma interpretação proposta em um fato certificado. O status de um item registra como ele se tornou conhecido: extraído diretamente de um analisador determinístico, proposto por um modelo, assumido para um cenário, derivado de uma regra, verificado por um verificador ou confirmado por uma pessoa. A cobertura registra se um domínio foi examinado completamente, amostrado, recuperado ou não processado. Uma conclusão negativa só é permitida quando a pesquisa relevante é fechada sob uma política declarada. Esta é a diferença entre um sistema simbólico confiável e um classificador fluente com uma saída em forma de JSON.

A discussão permanece acessível em vez de prova pesada. A interpretação abstrata, a execução simbólica, a verificação do modelo e a semântica lógica são explicadas através de sua essência operacional e das obrigações que criam. A notação formal aparece apenas onde esclarece uma distinção de que a prosa pode se confundir. O objetivo não é apresentar a matemática decorativa. É dar aos projetistas de sistemas, revisores e pesquisadores precisão conceitual suficiente para decidir onde uma garantia proposta é válida, onde é condicional e onde é meramente aspiracional.

A tese resultante pode ser declarada claramente. LLMs deve descobrir formas formais plausíveis; componentes determinísticos e formalmente restritos devem controlar fundamentação, compilação, execução e aceitação; LLMs pode então perceber o resultado canônico como linguagem, mas apenas sob um contrato que impede adições não suportadas. Esta não é uma solução completa para o significado. É uma rota credível do processamento de linguagem probabilística para raciocínio limitado, executável e auditável.

1. Da Linguagem Fluente à Semântica Executável

1.1 O que a Execução Exige e Onde Pertence a Fronteira de Confiança

A palavra “executar” carrega um compromisso mais forte do que “interpretar”, “resposta” ou “razão”. Um programa é executável porque suas operações têm um efeito definido em um estado. Uma consulta de banco de dados é executável porque os operadores relacionais determinam quais tuplas são selecionadas, unidas, agrupadas ou excluídas. Um problema de planejamento é executável porque as ações têm pré-condições e efeitos, e um planejador procura uma sequência que atinja uma meta. Uma teoria lógica é executável em um sentido mais amplo porque um procedimento de inferência determina se uma conclusão segue, se um modelo existe ou se as restrições são satisfatíveis. Em cada caso, a operação decisiva é externa à retórica usada para descrever a tarefa. Um resultado pode ser reproduzido a partir de um artefato e semântica.

Uma resposta LLM é diferente. O modelo mapeia um contexto para uma distribuição sobre as continuções. Ele pode internalizar padrões que se assemelham a algoritmos, e pode resolver muitas tarefas corretamente, mas a regra que está sendo aplicada em uma determinada etapa não é inspecionável separadamente ou garantida para ser usada de forma consistente. O pedido de aprisionamento pode melhorar o desempenho, mas uma cadeia de frases não é um objeto de prova simplesmente porque é organizado como etapas. A autoconsistência mostra várias continuções, não várias execuções formalmente definidas. O uso da ferramenta pode fazer um valor exato ao deixar a interpretação que produziu a ferramenta chamada não verificada. A distinção crucial é entre um modelo que propõe um artefato executável e um modelo cuja própria geração é tratada como execução.

tanto, a linguagem natural executável começa com a materialização. O sistema deve criar um objeto com um sistema de gramática, tipo declarado, semântica e mapa de origem. Esse objeto pode ser uma consulta relacional, uma teoria lógica, uma máquina de estado, um conjunto de restrições temporais, uma tabela de decisão, um programa Datalog, um autômato finito ou um gráfico digitado com operadores registrados. A escolha específica depende da tarefa. O que importa é que o comportamento do objeto não é mais determinado pela distribuição do modelo de linguagem next-token. Ele pode ser rejeitado, executado, testado, comparado e reproduzido de forma independente.

Este requisito é mais forte do que a produção estruturada comum. Um modelo pode emitir JSON sintaticamente válido que contém entidades inventadas ou campos mutuamente inconsistentes. Um esquema valida a forma, não a verdade. Da mesma forma, um modelo pode gerar SQL legal que une as colunas erradas, lógica de primeira ordem válida que inverte um quantificador ou uma fórmula temporal que expressa a relação de precedência oposta. Portanto, um limite de confiança útil não é simplesmente “depois da geração JSON”. Encontra-se após fundamentação de fonte, verificação de tipos, compatibilidade semântica e compilação formal. Cada símbolo que afete a resposta deve ter uma origem admissível e um papel definido.

A arquitetura naturalmente separa quatro autoridades. A autoridade de origem determina quais texto, dados ou observações foram realmente fornecidas. A autoridade semântica determina quais tipos, relações, modalidades e políticas de identidade são permitidos. A autoridade de regras determina quais restrições ou transformações constituem a teoria que

está sendo executada. A autoridade de execução determina se um resultado candidato segue sob essa teoria. Um LLM pode ajudar todos os quatro, mas não deve silenciosamente mesclá-los. Quando o mesmo modelo inventa uma premissa, seleciona uma ontologia, aplica uma regra e escreve o veredicto, não há separação significativa entre evidência e conclusão.

A autoridade de origem começa com bytes e vãos estáveis. Um sistema de análise de documentos deve preservar a revisão, estrutura e localização exatas de cada fragmento citado. Deve distinguir uma citação de um resumo e um resumo de uma inferência. Quando um modelo propõe que uma sentença expressa uma obrigação, a observação deve apontar para a sentença e manter o modelo, pronto, esquema e alternativas que produziram a classificação. Se a citação não puder ser encontrada na fonte, a observação é rejeitada em vez de aceita como uma reconstrução plausível. Isso soa como engenharia de proveniência mundana, mas é uma pré-condição para cada garantia posterior.

A autoridade semântica é geralmente distribuída através de uma ontologia superior e pacotes de domínio. Uma ontologia universal é atraente porque promete interoperabilidade, mas tende a se tornar abstrata demais para decidir questões úteis ou muito grandes e contenciosas para governar. Um design melhor define uma pequena camada comum para vãos de origem, menções, entidades, eventos, tempo, mundos, status, proveniência, alternativas, cobertura e lacunas. Os pacotes de domínio definem conceitos como intervenção clínica, obrigação regulatória, evento de objeto narrativo, chamada API de software ou transação financeira. Cada pacote deve indicar não apenas nomes de campo, mas compromissos e testes semânticos. Um tipo denominado “aprovação” não é significativo, a menos que o sistema especifique se ele denota um documento, um ato, um Estado, uma decisão de autoridade ou alguma combinação.

A autoridade de regras deve ser traduzida separadamente da interpretação do documento. Uma política, teoria linter ou modelo científico muda a uma taxa diferente da fonte que ela analisa. Se as regras são incorporadas de forma invisível em prompts, torna-se difícil saber se duas corridas aplicaram a mesma teoria. Os artefatos de regras com versões permitem publicação referenciada, diffs semânticos e reversão. Eles também permitem várias teorias sobre a mesma representação de documentos. Um manuscrito pode ser verificado quanto à consistência terminológica, coerência causal e suporte de citação sem recompilá-lo em um prompt opaco diferente para cada regra.

A autoridade de execução deve ser tão pequena e mecânica quanto prática. Para consultas relacionais, pode ser um avaliador determinista. Para restrições, um solucionador SMT ou CSP. Para o comportamento temporal, um verificador de modelo. Para propriedades de programa específicas do caminho, um executor simbólico. Para obrigações de prova, um prover de teorema mais um pequeno verificador de prova. Para uma linguagem de circuito personalizado, pode ser um intérprete restrito com operadores registrados e verificadores independentes. O componente de aceitação não deve perguntar a um LLM se o resultado “parecer correto”. Deve verificar uma testemunha contra a fonte e a teoria.

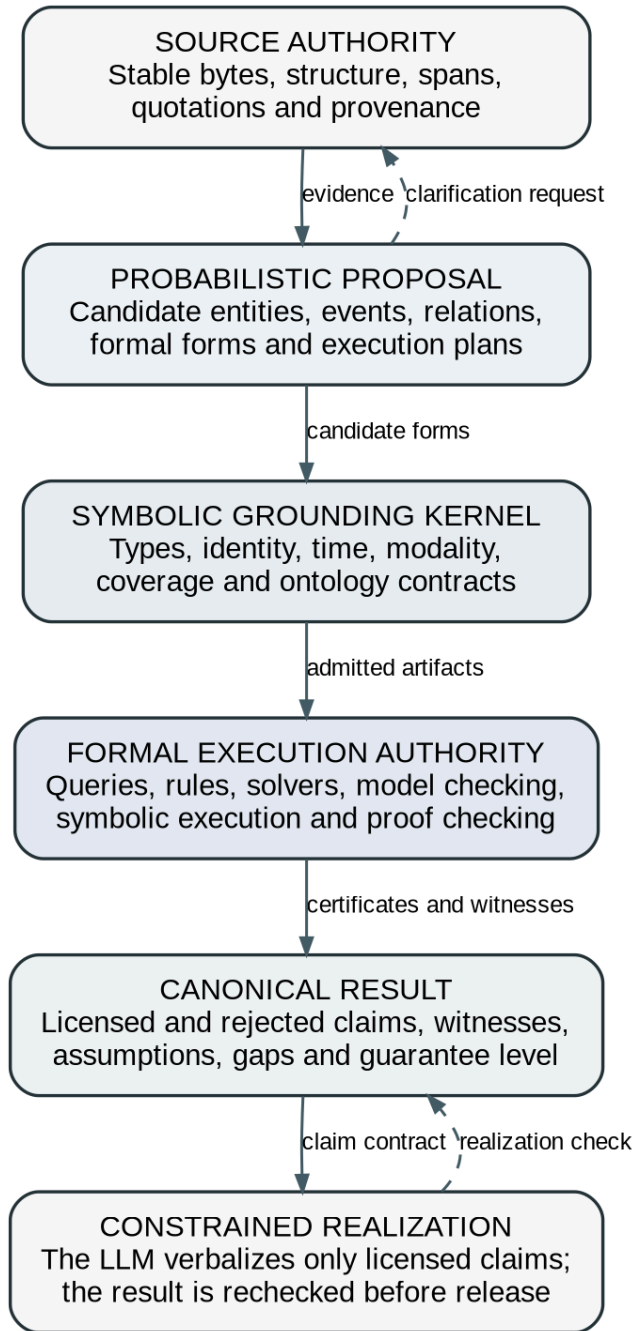


Figura 1. Limite de confiança para linguagem natural executável. O LLM propõe formulários semânticos, enquanto o fundamentação de fontes, a execução formal, a aceitação e o licenciamento final permanecem de forma independente.

Uma maneira útil de classificar sistemas é pela localização do oráculo final. Em um sistema puramente neural, o LLM é o oráculo. Em um sistema intensificado por ferramentas, uma ferramenta calcula um valor, mas o LLM pode permanecer o oráculo para saber se a chamada de ferramenta representa a questão. Em um sistema solucionador, o solucionador é o oráculo para as consequências de uma formalização gerada. Em um sistema fundamentado em evidências, o núcleo do solver e o fundamentação determinam conjuntamente o resultado aceito. Em um sistema de suporte de prova, mesmo a saída do solver pode ser verificada por um kernel menor e confiável. Cada passo estreita o espaço em que o erro probabilístico pode silenciosamente mudar o veredicto.

Isso não implica que toda aplicação precise de garantia de nível de teorema de melhoria. Um assistente de escrita pode usar circuitos determinísticos leves, enquanto um sistema regulatório ou de segurança pode exigir artefatos formais de prova. O princípio importante é que a garantia deve corresponder ao mecanismo. Um teste de regex determinístico pode certificar a presença de um padrão literal, não a ausência de um conceito. Um extrator de eventos assistido por modelo pode apoiar um aviso de continuidade proposto, não um relato mecanicamente certo da identidade narrativa. Um solucionador pode estabelecer insatisfabilidade de restrições, não a completude da tradução em linguagem natural. Rigor consiste em grande parte em recusar-se a transferir a confiança através dessas fronteiras.

A arquitetura mais forte é, portanto, federativa. O componente neural não é dissolvido em uma rede simbólica, e o componente simbólico não é reduzido a um regularizador diferenciável. Eles permanecem sistemas separados com uma interface tipada. O LLM é responsável pela amplitude, geração de hipóteses, normalização semântica e recuperação de linguagem incomum. O núcleo formal é responsável pela admissibilidade, execução, invariantes, testemunhas e reprodutibilidade. A revisão de 2026 por Chatzikyriakidis e Lappin chama sistemas relacionados federativo e observa que eles muitas vezes mostram fortes ganhos em tarefas onde a percepção pode ser separada da inferência. Os autores também enfatizam que as comparações permanecem confusas pelas diferenças de tarefa. O argumento arquitetônico para a federação é, no entanto, independente da superioridade de referência: módulos separados tornam as reivindicações auditáveis e permitem que um componente rejeite outro [Chatzikyriakidis and Lappin 2026].

A saída final da linguagem natural requer seu próprio limite. Uma vez que um solver retorna um resultado, é tentador fornecer o rastreamento para um LLM e solicitar uma resposta polida. Sem restrições, o modelo pode adicionar explicações em segundo plano, inferir motivos ou simplificar a incerteza em certeza. Um resultado canônico deve, portanto, enumerar as alegações que podem ser verbalizadas, seu status, evidência, suposições e escopo. O renderizador pode escolher a redação e a organização, mas deve ser verificado em relação à estrutura canônica. Um método prático é a realização em nível de reivindicação: o modelo gera frases associadas a identificadores de resultados; um verificador garante que cada frase seja implicada ou parafrasese uma reivindicação permitida; o material não suportado seja removido ou rotulado como comentário. Em configurações de alta garantia, a linguagem controlada com modelo pode ser preferível à prosa irrestrita.

O limite de confiança proposto neste livro pode ser resumido sem código. Os LLMs pode propor formas semânticas, mapeamentos de ontologias, escolhas de formalismo, reparos e explicações. Eles não estabelecem evidências, fecham uma pesquisa de mundo aberto, certificam a identidade, decidem a satisfatibilidade ou autorizam uma reivindicação final. Essas funções pertencem a contratos simbólicos explícitos. Essa separação preserva o valor único dos modelos de linguagem, evitando que sua fluência seja confundida com prova.

1.2 A Lacuna de Formalização: Por que um Solucionador Correto Ainda Pode Produzir a Resposta Errada

A fraqueza central da maioria dos sistemas de raciocínio neuro-simbólico não é o motor de inferência. Solucionadores SAT, solucionadores SMT, mecanismos de banco de dados, provadores de teoremas, verificadores de modelos e executores simbólicos são muitas vezes muito mais confiáveis do que o modelo de linguagem que os precede. A fraqueza reside na lacuna de formalização: a diferença entre o que a fonte significa no contexto e o objeto formal submetido à execução. Um sistema pode ser perfeitamente sólido, desde a entrada formal até a saída formal, sendo infundado da linguagem natural para responder.

Considere uma frase política simples: “Um revisor só pode aprovar uma liberação após o relatório de segurança estar completo, a menos que o comandante do incidente autorize uma exceção de emergência”. Um formalizador deve preservar pelo menos o ator, ação, objeto, condição temporal, estado de conclusão, exceção, autoridade e escopo de “apenas depois”. Deve distinguir a permissão da obrigação. Deve decidir se a exceção remove apenas a condição temporal ou autoriza a aprovação em geral. Ele deve conectar “o relatório de segurança” ao artefato correto e “o comandante do incidente” a uma identidade ou função. Se alguma dessas escolhas estiver errada, um solucionador pode aplicar corretamente a política errada.

O escopo do Quantifier é uma fonte clássica de erro. “Todos os pesquisadores revisaram um artigo” pode significar que cada pesquisador revisou pelo menos um artigo, não necessariamente o mesmo. “Um artigo foi revisado por cada pesquisador” tem uma estrutura diferente. A negação cria problemas semelhantes: “nem todas as amostras passaram” não significa que nenhuma amostra passou. Condicionais, comparativos, modalidade, relações temporais, anáfora e pressuposição criam distinções que a tradução em nível de superfície pode apagar. As tradições semânticas formais desenvolveram mecanismos sofisticados para esses fenômenos, mas a linguagem de domínio aberto adiciona conhecimento do mundo, convenções de gênero e objetivos implícitos que nenhuma representação única captura completamente.

A lacuna também aparece na análise semântica de denotações. Um sistema treinado apenas em uma pergunta e resposta final pode encontrar um programa que produza a resposta correta no ambiente de treinamento. Tais programas espúrios são um problema conhecido em análise semântica fracamente supervisionada [Pasupat and Liang 2016; Guu et al. 2017; Goldman et al. 2018]. A execução não distingue o programa pretendido de um acidental se ambos produzirem a mesma denotação. Os sistemas text-to-SQL enfrentam o mesmo risco: uma consulta pode retornar as linhas certas por causa dos valores de dados atuais enquanto contém a junção ou filtro errados. A decodificação guiada por execução remove programas que travam ou contradizem restrições intermediárias, mas não pode provar a fidelidade semântica [Wang et al. 2018].

A lacuna de formalização é melhor decomposta em obrigações separadas. O fundamentação lexical pergunta quais predicados ou campos correspondem a palavras e frases. Fundamentação referencial pede quais menções denotam a mesma entidade. O fundamentação estrutural pergunta quais argumentos, escopos e dependências pertencem juntos. A base temporal conecta frases a instantes, intervalos, calendários e pedidos. O fundamento modal distingue fatos, possibilidades, obrigações, permissões, intenções,

reivindicações relatadas e contrafactuais. O fundamento ontológico determina se um conceito pertence ao modelo de domínio selecionado. A base evidencial liga todas as premissas a um vão de origem, sensor, linha de banco de dados ou suposição declarada. Cada obrigação pode falhar de forma independente.

O sistema também deve decidir quanto significado para formalizar. Uma representação semântica total da prosa irrestrita não é nem prática nem necessária para a maioria das tarefas. A regra que está sendo executada cria demanda. Um circuito terminológico pode precisar apenas de períodos exatos e termos normalizados. Uma regra de conformidade temporal pode precisar de eventos, atores, datas e exceções. Uma análise causal pode precisar de reivindicações, intervenções, resultados e incerteza. A formalização orientada pela demanda reduz o custo e limita o número de compromissos sem suporte. Também cria um problema de cobertura explícito: se uma regra requer um conceito que os produtores disponíveis não podem se materializar, a corrida deve parar ou retornar desconhecida em vez de silenciosamente aproximar.

A ambiguidade deve ser representada como alternativas, não imediatamente desmornadas. Suponha que "Alex disse a Jordan que eles deveriam sair" aparece em um procedimento. O pronome pode se referir a Alex, Jordan ou um grupo, dependendo do contexto e das convenções de idiomas. Um modelo pode classificar os candidatos, mas uma alta confiança não é uma prova. Se o veredicto da regra downstream for o mesmo sob todas as ligações plausíveis, a ambiguidade é irrelevante e a execução pode prosseguir. Se o veredicto mudar, o resultado deve expor a dependência e solicitar esclarecimento ou contexto adicional. Este é um lugar onde a interpretação abstrata se torna conceitualmente útil: ela pode computar conclusões sobre um conjunto de interpretações sem enumerar cada combinação em detalhes completos.

Um mapeamento completo é muitas vezes impossível porque a linguagem natural é de mundo aberto. Um documento pode omitir fatos que são verdadeiros, assumir conhecimento comum ou confiar na prática institucional. O raciocínio do mundo fechado trata os fatos não registrados como falsos; o raciocínio de mundo aberto os trata como desconhecidos. Ambas as políticas são adequadas em contextos diferentes. Um inventário de banco de dados pode ser fechado para uma tabela e hora especificadas. Uma coleção de e-mails recuperados geralmente não é evidência completa de que nenhuma notificação foi enviada. A camada de fundamentação deve, portanto, angariar cobertura a observações e pesquisas. As conclusões negativas exigem um token de cobertura que declare o domínio, revisão de origem, escopo, método, exclusões e política de integridade.

O conhecimento externo introduz outras complicações. Um LLM geral contém informações extensas, mas não revertidas e potencialmente obsoletas. Um sistema simbólico não pode tratar a memória modelo como um fato atemporal. O conhecimento do mundo deve entrar através de pacotes de origem, datados, específicos de jurisdição e de conteúdo. Um pacote pode afirmar que uma regra legal foi eficaz em uma jurisdição particular durante um determinado intervalo ou que uma constante científica foi extraída de uma fonte nomeada. Os conflitos entre pacotes devem permanecer visíveis. Mundos fictícios ou hipotéticos devem ser selecionáveis sem contaminação por fatos do mundo atual. A documentação nllAgent trata corretamente esses pacotes como um limite, em vez de fingir que a memória de um modelo de fundação fornece autoridade formal.

A fidelidade da formalização pode ser testada, embora não universalmente provada. Um método é a verificação bidirecional: traduzir o objeto formal de volta para a linguagem controlada e compará-lo com a fonte. Outro é o teste contrastivo: gerar mudanças textuais mínimas que devem alterar um elemento formal e preservar o resto. Um terceiro é a comparação extensional: peça formalizações concorrentes para gerar exemplos distintivos ou resultados de consulta. Um quarto é a estabilidade da paráfrase: formulações semanticamente equivalentes devem compilar artefatos equivalentes ou comprovadamente relacionados. Um quinto é uma explicação contra-exemplo: quando o solucionador produz uma testemunha, mapear toda dependência para a fonte se estende e perguntar se o cenário é linguisticamente admissível. Um sexto é a revisão humana focada no pequeno conjunto de escolhas que afetam o resultado e não em todo o programa formal.

Sistemas interativos como o nl2spec ilustram o valor do reparo local. Em vez de pedir ao usuário para entender ou reescrever toda uma fórmula temporal, a interface associa subformulas com fragmentos de origem, permitindo que um componente equivocado seja corrigido [Cosler et al. 2023]. O feedback do Solver pode desempenhar um papel semelhante. O Logic-LM usa erros de sintaxe e execução para solicitar a reformalização [Pan et al. 2023]. Sistemas mais avançados podem usar núcleos insatisfatórios ou contra-exemplos em vez de mensagens de erro genéricas. O insight central é que o feedback formal deve restringir a questão linguística.

A lacuna de formalização também coloca limites nas alegações de “som” de ponta a ponta. Um sistema pode provar um teorema da forma: se toda observação aceita representa com precisão sua fonte e a ontologia e as regras estiverem corretas, então o veredicto segue. Isto é valioso. Não deve ser reescrito como “o veredicto é provado a partir do documento”, a menos que as obrigações de observação e fundamentação também tenham sido cumpridas. A garantia deve nomear suas suposições e incógnitas. Um envelope de solidez é mais honesto do que um rótulo de confiança binária.

Este livro, portanto, trata a formalização como um processo de compilação com vários artefatos intermediários, em vez de um único prompt. A fonte é segmentada e ancorada. Observações candidatas são produzidas sob esquemas. As alternativas de identidade e tempo estão representadas. A compatibilidade com ontologia é verificada. Um formalismo alvo é selecionado. O compilador produz um objeto executável e um mapa de origem. O executor produz uma testemunha, prova, modelo ou falha. O resultado é reproduzido e renderizado. Em cada fase, um erro tem uma categoria específica e um possível reparo. A arquitetura não elimina a incerteza semântica; impede que a incerteza seja escondida dentro da produção fluente.

Segue-se a conclusão mais importante da pesquisa. Os sistemas LLM apoiados por Solver devem ser avaliados separadamente quanto à fidelidade à formalização e à execução formal. Uma pontuação de resposta final elevada pode esconder programas espúrios. Uma pontuação baixa pode ser causada por uma boa formalização com um solucionador inadequado ou por uma má formalização com um solucionador perfeito. Os benchmarks precisam de ouro ou estruturas formais admissíveis, alinhamentos de evidências e pares mínimos semânticos, não apenas respostas. Até que essa separação se torne padrão, muitas alegações sobre “raciocínio formal com LLMs” permanecerão difíceis de interpretar.

2. A Linhagem Intelectual da Linguagem Executável

2.1 Linguagens Controladas, Semântica Formal, Discurso e Lógica Natural

A ideia de que a linguagem legível por humanos pode suportar inferência mecânica antecede grandes modelos de linguagem por décadas. Trabalhos anteriores abordaram o problema a partir da direção oposta: em vez de pedir a um modelo estatístico para interpretar a prosa irrestrita, os pesquisadores projetaram linguagens ou teorias semânticas que limitavam a ambiguidade suficiente para apoiar a tradução em lógica. Essas tradições são essenciais porque revelam o que um sistema de linguagem executável deve preservar mesmo quando um LLM executa a análise front-end.

Linguagens naturais controladas são o precedente de engenharia mais claro. Elas restringem o vocabulário, a gramática ou a interpretação, mantendo uma superfície que se assemelha a uma linguagem comum. Attempto Controlled English é o exemplo mais conhecido. As sentenças são traduzidas em estruturas de representação do discurso e depois em lógica formal, permitindo consultas e inferências enquanto permanecem legíveis para não-lógicos [Fuchs and Schwitter 1996; Fuchs et al. 2008]. Linguagens controladas foram usadas para representação do conhecimento, especificações técnicas, wikis semânticas, regras de negócios e requisitos. O levantamento de Kuhn mostra que a família é diversificada: algumas línguas são projetadas para clareza humana, em vez de precisão formal, enquanto outras trocam naturalidade pela semântica determinista [Kuhn 2014].

A lição duradoura não é que a língua irrestrita deva ser substituída por um dialeto controlado. É que um sistema executável precisa de uma camada de normalização cuja semântica é mais rigorosa do que a prosa comum. Um LLM pode servir como tradutor de linguagem aberta em uma forma semântica controlada, mas a forma controlada deve ter uma gramática e interpretação independentes. Sem esse limite, a “linguagem natural controlada” torna-se apenas um estilo de escrita preferido e não fornece nenhuma garantia computacional.

As linguagens controladas também expõem a importância do feedback do autor. Se uma frase está fora do fragmento controlado, um sistema tradicional o rejeita ou produz uma ambiguidade explícita. As interfaces LLM modernas tendem a fazer o oposto: elas selecionam uma leitura plausível e prosseguem. A rejeição é muitas vezes tratada como um defeito de usabilidade, mas em um sistema de alta garantia é uma forma de integridade epistêmica. Uma interface de linguagem executável de próxima geração deve combinar a flexibilidade da paráfrase LLM com a honestidade de um compilador de linguagem controlada. Pode propor várias leituras normalizadas, explicar as distinções e perguntar somente quando a distinção muda o resultado.

A semântica formal fornece uma linhagem teórica mais profunda. A semântica teórica-modelo representa o significado em termos de condições sob as quais as expressões são verdadeiras em um modelo. A composicionalidade conecta a interpretação de uma expressão complexa aos significados de suas partes e sua combinação sintática. A semântica estilo Montague, cálculos lambda dano, quantificadores generalizados e estruturas relacionadas oferecem contas precisas de escopo, estrutura de argumento e operadores lógicos. Eles demonstram que o significado de linguagem natural não é redutível a uma lista

plana de triplos. Quantificadores, negação, verbos intensionais, modalidade e contexto podem alterar o comportamento lógico de frases aparentemente simples.

Para a linguagem natural executável, a semântica formal contribui com dois requisitos. Em primeiro lugar, a representação intermédia deve preservar a estrutura do operador, em vez de apenas entidades e relações. “Todos”, “algum”, “não”, “somente”, “salvo”, “devem” e “antes” não são anotações em torno de uma proposição; eles determinam o espaço dos modelos. Em segundo lugar, a construção semântica deve ser do tipo consciente. Um intervalo temporal, uma pessoa, uma organização, uma proposição e uma obrigação não podem ser livremente substituídos simplesmente porque seus rótulos de superfície se assemelham uns aos outros. A verificação de tipos é uma das maneiras mais baratas de rejeitar formalizações incoerentes antes de invocar um solver.

A teoria da representação do discurso e as abordagens dinâmico-semânticas relacionadas abordam os fenômenos que a lógica sentença por frase lida mal. Um discurso introduz referências, transporta-as através de frases, atualiza o contexto e cria restrições de acessibilidade para pronomes e descrições. Os analisadores Boxer e DRS operacionalizaram essas ideias para análise semântica de ampla cobertura [Bos 2008; Liu et al. 2018]. O Banco de Significado Paralelo e as tarefas de análise compartilhadas do DRS tornaram possível avaliar representações de significado de escopo em todas as línguas [Abzianidze et al. 2019]. Esses esforços mostram progresso e dificuldade: vinculação variável, atribuição de sentido, escopo e relações de discurso são substancialmente mais difíceis do que a extração superficial.

Um sistema executável no nível do documento deve aprender com a semântica dinâmica sem exigir um DRS monolítico para cada fonte. Precisa de objetos de discurso persistentes, hipóteses de entidade de escopo, eventos, mundos e relações de atualização. Deve distinguir uma menção de uma entidade e um link de identidade de candidato de um estabelecido. Deve preservar contextos citados, hipotéticos, negados e fictícios. Um gráfico de conhecimento plano que mescla cordas iguais em um nó cometerá erros sistemáticos em documentos longos. Os nomes se repetem, os apelidos ocorrem, mudam de papéis e os documentos descrevem mundos mutuamente inconsistentes.

O custo prático da análise semântica formal completa explica por que muitos sistemas se moveram em direção a representações mais rasas. A rotulagem de papéis semânticos identifica a estrutura de predicação-argumento. A representação de significado abstrato representa conceitos e relações em um gráfico. A extração de informações identifica entidades, eventos e relações. Essas representações são úteis, mas sua semântica é muitas vezes incompleta para execução. Eles podem omitir escopo quantificador, modalidade, ancoragem temporal e a diferença entre conteúdo afirmado e relatado. Um tempo de execução da linguagem executável deve tratar tais estruturas como observações contendo evidências, e não como uma forma lógica completa. Um gráfico pode suportar muitas regras locais, deixando explícitos os aspectos não resolvidos.

A lógica natural oferece outro compromisso importante. Em vez de traduzir frases em uma linguagem formal irrestrita, os sistemas naturais-lógicos raciocinam através de transformações estruturadas perto da forma superficial. MacCartney e Manning desenvolveram relações de inferência baseadas na monotonicidade, implicação lexical e projetividade [MacCartney and Manning 2007, 2009]. NaturalLI mais tarde usou operadores de lógica natural para inferência eficiente sobre o texto [Angeli and Manning 2014]. LangPro combina uma gramática formal,

conhecimento lexical e prova de teorema para inferência em linguagem natural [Abzianidze 2017].

A lógica natural é atraente porque muitas questões documentais dizem respeito a relações de implicação que podem ser capturadas sem construir um modelo de mundo grande. Substituir o “cão” por “animal” é válido em um contexto de recuperação para cima, mas não necessariamente sob negação ou restrições universais. Esses cálculos são mais transparentes do que uma pontuação de similaridade de incorporação. Eles podem apoiar verificações de contradição, raciocínio taxonômico e inferência controlada. No entanto, a lógica natural não é um executor universal. Ele lida bem com certas monotonicidade e relações lexicais, mas não expressa naturalmente procedimentos arbitrários, aritmética, planejamento ou comportamento temporal complexo. Em uma arquitetura multi-formalismo, deve ser um motor entre vários.

As tradições semânticas também esclarecem por que o “remendo” tem vários níveis. A semântica formal pode especificar como uma expressão contribui para as condições da verdade, mas não determina automaticamente qual objeto do mundo real um nome denota ou se uma fonte é credível. O DRS pode representar um referente ao discurso, mas a resolução de identidade continua sendo uma tarefa empírica. A lógica natural pode propagar a implicação lexical, mas a relação lexical pode ser dependente do domínio. A executibilidade exige, portanto, uma parceria entre a estrutura linguística, a ontologia de domínio e a gestão de evidências.

Uma arquitetura útil pode distinguir pelo menos três produtos semânticos. O primeiro é um modelo de fonte que contém estrutura exata, spans e citações. A segunda é uma camada de interpretação contendo menções datilografadas, eventos, relações, modalidades e alternativas. A terceira é uma teoria executável que usa interpretações selecionadas para responder a uma pergunta específica. Técnicas semânticas formais incluem principalmente a segunda camada. Linguagens controladas e provers de teorema conectam o segundo e terceiro. Proveniência e cobertura conectam ambos de volta ao primeiro.

A literatura anterior também adverte contra a suposição de que uma representação pode servir a todas as tarefas. DRS é expressivo para o discurso, mas caro para produzir e raciocinar. A lógica de primeira ordem é bem compreendida, mas estranha para fenômenos temporais, probabilísticos ou desviáveis. As lógicas de descrição oferecem raciocínio descidível ontologia, mas restringem a expressividade. A lógica natural permanece próxima da linguagem, mas é especializada. O inglês controlado melhora a previsibilidade restringindo a entrada. A conclusão correta não é escolher um vencedor, mas preservar informações semânticas suficientes para reduzir uma tarefa no formalismo que se encaixa nela.

LLMs mudar a economia deste projeto. Eles podem gerar análises candidatas, reescrever linguagem descontrolada em formulários controlados, sugerir mapeamentos de ontologias e explicar alternativas formais aos usuários. Eles tornam prático tentar a interpretação semântica em documentos que as gramáticas escritas à mão rejeitariam. Eles não eliminam a necessidade das distinções semânticas desenvolvidas por trabalhos anteriores. Pelo contrário, essas distinções tornam-se os contratos pelos quais as propostas LLM são julgadas.

Um erro recorrente nos sistemas contemporâneos é tratar uma loja tripla gerada por LLM como uma representação semântica completa. Triples são convenientes para consultas de recuperação e grafômetro, mas não codificam escopo, proveniência, modalidade, tempo, negação, alternativas de identidade ou cobertura. Uma camada semântica executável deve ser mais rica, mas ainda finita e operacional. Pode armazenar uma afirmação como um objeto

datilografado com participantes, contexto temporal, polaridade, modalidade, mundo, evidência, produtor e status. Pode representar ligações alternativas sem escolher uma. Pode declarar quais dimensões estão ausentes. Isso é menos elegante do que uma única forma lógica, mas é mais fiel à forma como os sistemas reais adquirem significado.

A contribuição mais profunda da linhagem pré-LLM é, portanto, disciplina conceitual. A legibilidade humana não implica determinação semântica. A determinação formal é alcançada por restrição, digitação, contexto explícito e rejeição de construções não suportadas. O significado do documento é dinâmico e de escopo. Inferência perto da linguagem de superfície é possível, mas especializada. Múltiplas representações formais são legítimas porque respondem a perguntas diferentes. Essas lições definem a arquitetura que um compilador baseado em LLM deve recuperar em vez de ignorar.

2.2 Parsing Semântico, Consultas Executáveis, Indução de Programas e Primeiros Sistemas Neuro-Simbólicos

A análise semântica converte enunciados em linguagem natural em objetos formais que podem ser executados contra um banco de dados, base de conhecimento, ambiente ou intérprete. É o antecessor histórico mais próximo do pipeline de linguagem executável proposto aqui. O campo inclui a resposta a perguntas sobre tabelas e gráficos de conhecimento, texto-a-SQL, instrução seguinte, síntese de supressão regular, interpretação de comando robótico e geração de programas. Seus sucessos demonstram o poder da execução externa; suas falhas revelam por que a execução sozinha não estabelece a compreensão.

Os primeiros analisadores semânticos dependiam de gramáticas, léxicos e formas lógicas supervisionadas. Eles poderiam produzir representações transparentes, mas exigiam anotações caras e muitas vezes transferidos mal entre domínios. A supervisão fraca reduziu a carga de anotação pelo treinamento de denotações: uma pergunta é emparelhada com uma resposta, e o sistema procura por um programa que o produza. Essa abordagem permitiu conjuntos de dados mais amplos, mas criou o problema de programa espúria. Muitos programas podem acidentalmente retornar a mesma resposta, especialmente em pequenas tabelas ou bases de conhecimento. O sinal de treinamento, em seguida, recompensa o comportamento sem identificar a semântica pretendida [Pasupat and Liang 2016].

O problema é estrutural, não apenas estatístico. Suponha que a resposta para “Qual cidade sediou o evento de 2012?” é Londres. Um programa que seleciona a linha com o ano 2012 é pretendido. Um programa que seleciona a primeira linha também pode retornar Londres na mesa de treinamento. Mais dados podem reduzir as correlações acidentais, mas não podem garantir que o programa aprendido reflita o idioma. Exemplos abstratos, modelos de alinhamento, restrições de tipo e filtragem baseada em execução foram desenvolvidos para suprimir candidatos espúrios [Goldman et al. 2018; Lee et al. 2023]. Essas técnicas também são importantes para a formalização do LLM. Um resultado do solucionador deve ser tratado como um teste de uma formalização, não como seu certificado semântico.

As máquinas simbólicas neurais combinam a geração de programas neurais com um interpretador simbólico de Lisp [Liang et al. 2017]. O componente neural propõe programas, enquanto o executor rejeita candidatos inválidos e retorna denotações. Mou e seus colegas acoplam explicitamente a execução neural distribuída com execução simbólica para consultas em linguagem natural, relatando melhorias na precisão, eficiência e interpretabilidade [Mou et al. 2017]. Esses sistemas estabeleceram um padrão durável: os modelos neurais lidam com a variabilidade da linguagem; os intérpretes simbólicos impõem o significado operacional.

O Text-to-SQL tornou o padrão comercialmente importante. Uma pergunta do usuário é compilada em uma consulta cuja execução retorna dados. A decodificação guiada por execução pode executar candidatos SQL parciais ou completos e descartar aqueles que produzem erros ou resultados impossíveis [Wang et al. 2018]. Esquema ligando constrange nomes e relações. Os LLMs moderno melhora substancialmente a geração, mas a generalização entre domínios, questões ambíguas, convenções específicas de banco de dados e solicitações não respondíveis permanecem difíceis. A lição fundamental para a linguagem natural executável é que o próprio ambiente formal fornece restrições úteis. Tipos de coluna, chaves estrangeiras e erros de consulta fornecem feedback que a modelagem de linguagem pura não tem.

No entanto, o texto-para-SQL também mostra os limites de uma única métrica de precisão final. A correspondência de string exata pode rejeitar consultas equivalentes. A precisão da execução pode aceitar consultas semanticamente erradas que retornam o mesmo resultado. Uma avaliação rigorosa precisa de equivalência em instâncias de banco de dados variadas ou contra-exemplos cuidadosamente construídos. Isso é diretamente análogo à avaliação da formalização de regras: um programa formal gerado deve ser testado em cenários de distinção, não apenas em uma resposta observada.

A síntese do programa a partir da linguagem natural generaliza a ideia. Os sistemas geram expressões regulares, trechos de código, transformações de dados ou programas específicos de domínio pequenos. Exemplos podem restringir o espaço do candidato e os sistemas de tipo podem eliminar programas sem sentido. A geração neural de expressões regulares mostrou que o mapeamento de linguagem para programa pode funcionar mesmo com conhecimento de domínio limitado, ao mesmo tempo em que ilustra a fragilidade quando as descrições estão subespecificadas [Locascio et al. 2018]. Mais tarde, o LLMs, auxiliado por programa, moveu a computação para fora do modelo, mas o problema fundamental da síntese permaneceu: o modelo deve escolher um programa cuja semântica corresponda à solicitação.

Um compilador robusto deve, portanto, buscar várias formas de evidência. As descrições em linguagem natural restringem a intenção. Exemplos de saída de entrada restringem o comportamento extensional. Tipos constriam operações admissíveis. Os esquemas de domínio restringem o vocabulário. Testes de casos de ponta de prova. Um programa gerado que satisfaz todas essas restrições é mais credível do que um que apenas executa. Este princípio de várias evidências pode ser aplicado às regras do documento: autoridade textual, exemplos positivos e negativos, descobertas esperadas e mutações semânticas devem moldar um circuito antes da publicação.

A programação lógica fornece outro alvo executável. As regras do Datalog calculam as consequências para um ponto de referência e suportam avaliação incremental. O projeto e as línguas relacionadas representam as relações e a busca. A interpretação abstrata há muito tempo é utilizada para analisar programas lógicos, mostrando uma ponte conceitual entre a semântica declarativa e a análise de programas. Para aplicações em linguagem natural, a programação lógica é bem adequada para fechamentos monócritos: relações transitivas, taxonomias, propagação de dependência e regras de política sem padrões complexos. É menos adequado quando o raciocínio depende fortemente da incerteza, fontes inconsistentes ou exceções não-monotônicas, a menos que seja estendido com mecanismos explícitos.

Tabelas de decisão e sistemas de regras de produção são muitas vezes negligenciados em pesquisas acadêmicas, mas são altamente relevantes. Muitas políticas não são teorias lógicas profundas; são combinações finitas de condições e resultados. Uma tabela de decisão torna visível a cobertura, conflitos e combinações ausentes. Pode ser compilado em predicados executáveis e testado sistematicamente. A linguagem natural pode ser usada para escrever ou explicar a tabela, enquanto a mesa permanece a semântica autoritária. Para o linting de documentos corporativos, as tabelas de decisão podem ser mais seguras e fáceis de validar do que a lógica arbitrária de primeira ordem.

Instruções temporais e processuais exigem alvos mais ricos. Máquinas de estado finito, árvores de comportamento, linguagens de planejamento e lógicas temporais representam ações e condições ordenadas. A robótica usa há muito tempo os analisadores de linguagem que compõem as primitivas executáveis. Os recentes sistemas de manipulação

neuro-simbólica continuam esse padrão: um analisador de linguagem mapeia uma instrução para um programa cujos componentes podem incluir percepção neural e contagem simbólica, raciocínio espacial ou controle de ação. O sucesso de tais sistemas depende de uma biblioteca de habilidades restrita e ambiente fundamentado, não de cobertura semântica irrestrita [Tzifas and Kasaei 2026].

Programação lógica probabilística e sistemas neuro-simbólicos diferenciáveis tentam uma integração mais apertada. DeepProbLog, Logic Tensor Networks e sistemas relacionados combinam predicados aprendidos com estrutura lógica [Manhaeve et al. 2018; Badreddine et al. 2022]. Eles são valiosos quando a incerteza e o aprendizado são intrínsecos e podem injetar restrições no treinamento. No entanto, a diferença não é equivalente à garantia formal. Os valores de verdade difusa e as penalidades suaves podem orientar os modelos sem produzir obrigações de prova discretas. Para a linguagem natural executável, esses sistemas podem estimar ou classificar candidatos semânticos, mas um limite de aceitação de alta garantia ainda se beneficia de um verificador simbólico separado.

A revisão neuro-simbólica NLP de 2026 distingue os sistemas injetivos, que compilam informações simbólicas em modelos neurais, de sistemas federativos, que preservam componentes autônomos. Ele relata que os sistemas injetivos dominam a literatura, mas muitas vezes mostram ganhos modestos em tarefas amplas de NLU, enquanto os sistemas federativos mostram maiores ganhos em tarefas de raciocínio. Os autores alertam explicitamente que a comparação é confundida por diferentes benchmarks e que a generalização fora do domínio continua sendo um problema sério [Chatzikyriakidis and Lappin 2026]. A interpretação equilibrada é que a federação é arquitetonicamente atraente para auditabilidade e decomposição de tarefas, mas ainda não comprovada empiricamente como uma estratégia de escala universal.

A linhagem de parização semântica também esclarece o papel de uma língua intermediária. Uma representação deve ser expressiva o suficiente para capturar o significado relevante para tarefas, mas limitada o suficiente para compilar de forma confiável. Se estiver muito perto do texto de superfície, a execução permanece implícita. Se for muito geral, a complexidade da tradução e do solucionador se torna incontrolável. Beiser e Penz descrevem isso como um problema de linguagem intermediária em sistemas neuro-simbólicos contemporâneos: escolher um formalismo pode se mover em vez de resolver a dificuldade [Beiser and Penz 2025]. Uma arquitetura multi-formalismo reduz a pressão sobre qualquer idioma, mas introduz um novo requisito para roteamento da digitação e consistência de formacionalismo de tipo.

Um pivô semântico útil não deve, portanto, fingir ser a lógica final. Deve preservar as evidências e as distinções neutras em tarefas – entidades, menções, eventos, relações, tempo, modalidade, quantidades, mundos, proveniência, alternativas e cobertura – enquanto permitem a redução em executores especializados. Algumas consultas podem ser respondidas diretamente sobre o pivô usando operações relacionais. Outros exigem uma fórmula temporal compilada, sistema de restrição ou máquina de estado. O pivô é valioso porque separa a compreensão da linguagem de qualquer solucionador em particular, mantendo informações suficientes para a execução fundamentada na fonte.

O programa de pesquisa pré-LLM alcançou um sucesso substancial, mas limitado ao domínio. Ele mostrou que a linguagem natural pode ser compilada em programas executáveis, que os intérpretes simbólicos melhoram a sistemática, que os tipos e esquemas reduzem os erros e que a supervisão fraca cria atalhos semânticos. Também mostrou que ampla cobertura

e semântica exata estão em tensão. Os LLMs melhora substancialmente a cobertura e a geração de candidatos, mas eles herdam todos os problemas estruturais identificados pela análise semântica. A síntese correta não é descartar a maquinaria anterior. É colocar o LLMs na frente e ao redor dele, com interfaces formais que tornam as antigas lições aplicáveis em nova escala.

3. Interpretação Abstrata e Execução Simbólica para Linguagem

3.1 Interpretação Abstrata como Semântica da Incerteza Delimitada

A interpretação abstrata foi desenvolvida como uma teoria geral para a computação de aproximações seguras do comportamento do programa [Cousot and Cousot 1977]. Um programa pode ter infinitas entradas e estados, tornando impossível a execução exaustiva. Em vez de enumerar cada estado, uma análise representa conjuntos de estados através de propriedades abstratas, como intervalos, sinais, possíveis tipos de objetos, alcancem definições ou relações de alias. As operações abstratas são projetadas de modo que todo comportamento concreto relevante para a propriedade permaneça representado. A análise pode incluir comportamentos impossíveis e, portanto, relatar falsos alarmes, mas uma sobrecarga de som não omite silenciosamente um comportamento que a semântica concreta permite.

A ideia operacional é simples. Uma execução concreta rastreia valores exatos. Uma análise de intervalo pode rastrear que uma variável está entre zero e dez. Quando o programa acrescenta cinco, a operação de transferência abstrata produz um intervalo entre cinco e quinze. Em uma filial, os estados são refinados de acordo com a condição. Em uma fusão, a informação é unida. Para loops e processos recursivos, a análise procura um ponto de correção. O alargamento pode forçar a convergência sacrificando a precisão, enquanto o estreitamento pode recuperar alguns detalhes depois. A estrutura matemática fornece um vocabulário disciplinado para precisão, solidez, convergência e refinamento.

Aplicar a estrutura à linguagem requer uma semântica concreta. O objeto que está sendo aproximado não pode ser “significado” em um sentido indefinido. Para uma tarefa limitada, o domínio concreto pode ser o conjunto de estruturas semânticas admissíveis sob uma gramática, ontologia, a política de base, revisão de origem e a teoria de fundo especificada. Cada estrutura contém entidades possíveis, atribuições de eventos, relações temporais, modalidades e links de origem. A ambiguidade cria várias estruturas. A falta de informação deixa as dimensões sem restrições. Fontes conflitantes podem gerar mundos alternativos em vez de uma união inconsistente.

Um estado semântico abstrato resume esse conjunto. Pode registrar que um pronome pode se referir a qualquer uma das duas pessoas, que um evento ocorre dentro de um intervalo de tempo, que uma obrigação está presente sob uma interpretação e ausente sob outra, ou que uma quantidade está dentro de um intervalo. Um fato deve ser válido em cada interpretação representada. Um fato pode ser mantido em pelo menos um. Um fato não pode ser válido em nenhum. Desconhecido indica que o sistema não tem a representação, cobertura ou recursos computacionais necessários para decidir. Essa distinção de quatro vias é mais informativa do que uma probabilidade sozinha e mais honesta do que forçar cada pergunta a ser verdadeira ou falsa.

Considere um documento de política afirmando: “O operador envia o aviso após a validação. Se o modo de emergência estiver ativo, o supervisor pode renunciar ao período de espera. Suponha que não esteja claro se “o período de espera” se refere ao período anterior à validação ou a um período posterior. Um único parse faz uma escolha frágil. Um estado abstrato pode reter dois modelos temporais. Se ambos implicarem que o aviso deve seguir a

validação, essa conclusão deve ser. Se apenas um permitir aviso imediatamente, o envio imediato pode. Uma regra de conformidade pode identificar qual veredicto depende da frase ambígua e solicitar esclarecimentos somente quando necessário.

O benefício principal não é que a interpretação abstrata magicamente entenda a linguagem. É que dá uma semântica de execução de princípios, uma vez que um conjunto delimitado de leituras foi construído. Ele diz ao sistema como agregar alternativas sem confundir a possibilidade com necessidade. Ele também suporta domínios modulares. Identidade, tempo, quantidade, modalidade, proveniência e status normativo podem ter sua própria abstração e, em seguida, ser combinadas por meio de produtos reduzidos que trocam informações quando forem úteis.

Um domínio de identidade pode representar classes de equivalência de candidatos com relações explícitas, diferentes e não resolvidas. Não deve mesclar entidades simplesmente porque os nomes correspondem. Um domínio temporal pode usar ordens parciais, intervalos ou restrições de diferença. Um domínio de modalidade pode distinguir conteúdo afirmado, reportado, pretendido, permitido, exigido, proibido, hipotético e contrafactual. Um domínio de quantidade pode usar valores exatos, intervalos, unidades e compatibilidade dimensional. Um domínio de proveniência pode ordenar status de evidência, como extraídos, propostos, assumidos, derivados, verificados e confirmados por humanos, sem permitir que a confiança apague sua origem. Um domínio de cobertura pode rastrear se uma pesquisa está completa o suficiente para suportar reivindicações de ausência.

Os domínios devem ser escolhidos pela propriedade que está sendo verificada. Uma terminologia linter não precisa de uma rica treliza deôntica. Uma regra temporal não precisa de semântica lexical completa se eventos e datas já estiverem materializados. Essa seleção orientada por tarefas reduz a complexidade e reflete a análise estática orientada pela demanda. Em um tempo de execução de documentos longos, circuitos ou regras declaram os tipos de observação e as dimensões abstratas que eles exigem. O compilador materializa apenas essas dimensões e rejeita a execução quando um domínio crítico não pode ser fornecido.

As funções de transferência são a parte mais difícil. Na análise de programas comuns, a semântica de uma tarefa ou operação aritmética é fixada pela linguagem de programação. Na análise em linguagem natural, a operação que atualiza o estado semântico pode ser proposta por um LLM. Uma frase poderia introduzir um evento, modificar um estado anterior, estabelecer uma relação temporal, criar uma exceção ou relatar a crença de outra pessoa. Se um tradutor aprendido se torna diretamente a função de transferência, a solidez clássica é perdida.

Um design defensável separa a geração de candidatos da transferência admitida. O LLM propõe um conjunto finito de atualizações semânticas tipadas com vãos de origem e alternativas. Um materializador determinístico valida que a evidência citada existe, os campos satisfazem um esquema, unidades e tipos são compatíveis, referências apontam para objetos admissíveis e nenhum status é silenciosamente atualizado. O intérprete abstracta aplica as funções de transferência registadas nos registos admitidos. A análise resultante é sólida em relação ao conjunto de candidatos admitidos, não necessariamente em relação a cada interpretação que um ser humano poderia reconhecer. Esta condicionalidade deve ser declarada explicitamente.

Existem várias maneiras de fortalecer a suposição do candidato. Vários prompts ou modelos independentes podem propor alternativas. Geradores baseados em gramática ou ontologia podem garantir cobertura para fragmentos controlados. Variantes da parafrase podem testar a

estabilidade. Exemplos contrastantes podem revelar interpretações ausentes. A revisão humana pode se concentrar em ambiguidades relevantes. Um compilador pode deliberadamente gerar demais dentro de uma gramática delimitada e deixar que as restrições posteriores eliminem candidatos impossíveis. Cada método troca o custo de confiança de que o conjunto contém todas as leituras materialmente plausíveis.

A interpretação abstrata também é relevante para a propagação de regras em escala documental. Muitos problemas de plantio e conformidade se assemelham à análise de fluxo de dados. Os fatos são gerados em locais de origem, propagados através de escopos, unidos entre seções e invalidados por declarações posteriores. Uma definição pode estar disponível após a sua declaração, mas não antes. Um objeto pode mudar de estado em uma narrativa. Um requisito pode ser aplicado dentro de uma jurisdição ou versão. Uma reclamação pode adquirir provas mais tarde. Um mecanismo de ponto de referência incremental pode calcular as consequências e atualizar apenas as regiões afetadas quando o documento muda.

Essa perspectiva evita reprisar um LLM geral sobre um documento inteiro após cada edição. Artefatos estruturais e semânticos estáveis podem ser armazenados em cache por resumo de conteúdo. Um parágrafo alterado invalida as observações locais e os estados abstratos a jusante que dependem deles. O gráfico de análise identifica quais circuitos precisam ser replay. Isso é análogo a compilações incrementais e frameworks de análise estática. Também torna as explicações precisas: o sistema pode mostrar qual fonte alterou qual invariante abstrato e por quê.

O alargamento e a perda de precisão têm análogos semânticos. Uma longa narrativa pode conter centenas de possíveis identidades de objetos ou relações temporais. Manter todas as combinações é impraticável. O sistema pode resumir “um desses objetos”, “algum tempo dentro desse intervalo” ou “uma dessas jurisdições”. Esse alargamento pode introduzir avisos espúrios. O estreitamento ou refinamento local pode recuperar a precisão em torno de uma suspeita de violação. O objetivo do projeto não é evitar a abstração, mas tornar sua perda de precisão visível e reversível.

A literatura NLP estabelecida sobre robustez certificada usa ideias de interpretação abstrata de uma maneira diferente. Jia e seus colegas usaram a propagação de intervalos para treinar modelos cujas previsões são certificadas contra cada substituição de palavras em uma família predefinida, relatando 75% de precisão adversária no IMDB e SNLI sob seu modelo de ameaça [Jia et al. 2019]. Trabalho posterior utilizou smoothing randomizado, frameworks de reconhecimento de robustez e domínios especializados do Transformer para melhorar os limites certificados [Ye et al. 2020; Wang et al. 2023; Huang et al. 2026]. Estas são garantias genuínas, mas o domínio concreto é um conjunto de entradas de modelo perturbadas, não um conjunto de interpretações semânticas. A família de transformação é definida externamente e assumida para preservar o rótulo.

Essa distinção é importante. Certificar que a previsão de um classificador é invariante sob todas as substituições em uma lista de sinônimos não prova que toda substituição preserva o significado da frase no contexto. Isso não prova que o classificador esteja correto na frase original. Certifica a robustez em relação a um conjunto formal de perturbação. Esta literatura ainda é altamente relevante porque demonstra como os domínios abstratos podem escalar sobre as entradas de linguagem combinatórias e como a precisão da verificação depende da representação de correlações. Ele também fornece um aviso metodológico: a reivindicação formal mais forte é sempre relativa ao conjunto concreto realmente definido.

Uma direção de pesquisa promissora é conectar os dois usos. A análise neural certificada poderia vincular o comportamento do modelo de proposta semântica sob paráfrases controladas, enquanto a interpretação abstrata semântica poderia limitar as consequências de interpretações alternativas fundamentadas. O primeiro pergunta se a extração de um modelo muda sob transformações de superfície permitidas. Este último pergunta se uma conclusão em nível de documento muda em estruturas semânticas admissíveis. Juntos, eles poderiam identificar se a incerteza se origina na instabilidade do modelo ou na ambiguidade semântica genuína.

Os LLMs também pode ajudar a construir intérpretes abstratos, mas apenas sob aceitação mecânica. A SAIL demonstra esse princípio usando o LLMs para procurar transformadores abstratos enquanto impõe restrições sintáticas e semânticas e otimizando um custo que mede a insatisfação. Os transformadores aceitos são globalmente sólidos, e o sistema pode sintetizar transformadores precisos para operadores não lineares que não estavam disponíveis anteriormente [Gu et al. 2026]. A lição mais ampla é a arquitetura: os modelos generativos podem explorar um imenso espaço de design se um pequeno validador formal controlar a admissão.

Pedir diretamente a um LLM para realizar a interpretação abstrata é menos confiável. Experimentos em benchmarks de análise de programas mostram que os modelos podem seguir estilos de interpretação abstrata, mas fazem erros lógicos em estruturas complexas e estados de alucinação [Mitchell et al. 2025]. Esse resultado deve temperar as propostas em que um modelo narra um invariante e a narração é aceita como análise. O modelo é valioso para sugerir invariantes, partições de domínio ou predicados de refinamento. O cálculo do ponto de referência e os controles de solidez devem permanecer simbólicos.

Para a linguagem natural executável, a interpretação abstrata não é, portanto, nem uma metáfora nem uma solução completa. É uma teoria para a aproximação disciplinada depois que o sistema definiu o que conta como uma possível interpretação. Ele suporta semântica, propagação incremental, composição de domínio, raciocínio de ausência conservadora e refinamento direcionado. Sua garantia está condicionada à cobertura semântica. Esta limitação não é fatal; é o ponto exato em que a interpretação assistida por modelo deve ser avaliada e governada.

3.2 Execução Simbólica, Resolução de Restrições, Verificação de Modelos e Refinamento Semântico

A execução simbólica clássica executa um programa com entradas simbólicas em vez de valores específicos [King 1976]. Um estado simbólico contém um local de controle, expressões simbólicas para variáveis do programa e uma condição de caminho que descreve quais entradas atingem esse estado. Em uma filial, forks de execução. Um caminho adiciona a condição do ramo; o outro adiciona sua negação. Um solucionador de restrições determina se as condições são satisfatíveis e pode produzir insumos de concreto que realizam um caminho viável. O método é poderoso porque produz testemunhas: não apenas um aviso de que um erro pode existir, mas uma entrada e um caminho que o demonstram.

A dificuldade é a explosão do caminho. Os rativos se multiplicam, os loops e a recursão criam caminhos ilimitados, teorias complexas desafiam os solucionadores e bibliotecas externas são difíceis de modelar. Pesquisas de execução simbólica descrevem um grande espaço de projeto de estratégias de busca, fusão de estados, resumos composicionais, execução concóica, modelos de ambiente e otimizações de solucionadores [Baldoni et al. 2018]. Nenhum motor explora todos os comportamentos de software real. As reivindicações de execução simbólica, portanto, incluem limites, suposições de modelagem e limitações de cobertura.

A linguagem natural conecta-se à execução simbólica de várias maneiras distintas. O mais simples é o código de linguagem: um LLM gera um programa e um executor simbólico convencional analisa esse programa. Outra é a linguagem-especificação: o modelo gera uma propriedade ou restrição, que é verificada em relação ao código existente. Um terceiro é a execução do programa semântico: um documento é compilado em uma máquina de estado ou programa de regras cujas interpretações alternativas se tornam escolhas simbólicas. Um quarto é a exploração assistida por LLM: um modelo prioriza caminhos, propõe resumos ou razões sobre restrições que os solucionadores convencionais não podem expressar facilmente.

A terminologia deve ser precisa. Executar SQL, Python ou uma forma lógica gerada a partir de uma pergunta é execução formal, mas não é necessariamente execução simbólica no sentido sensível ao caminho. A diferença é importante porque a execução simbólica oferece condições de caminho e testemunhas concretas, enquanto a execução ordinária fornece um resultado para um estado concreto. Ambos são úteis. Uma pesquisa que os agrupa sem distinção faz com que as garantias pareçam mais fortes do que são.

O SymGPT fornece um dos exemplos mais claros de linguagem para especificação. O sistema estuda 132 regras em linguagem natural de três padrões ERC, traduz-as com um LLM em uma gramática restrita, sintetiza restrições que descrevem violações e usa a execução simbólica para pesquisar contratos Ethereum. Os autores relatam 5.783 violações em 4.000 contratos, incluindo 1.375 com caminhos de ataque explícitos para roubo, e relatam desempenho mais forte do que seis técnicas automatizadas e um serviço de auditoria especializado em segurança [Xia et al. 2026]. A força do design está na gramática estreita, síntese sistemática de restrições e caminhos concretos do programa. Seu risco semântico remanescente reside em saber se cada regra ERC foi traduzida fielmente.

Esse padrão generaliza. Uma política pode ser compilada em pré-condições, transições proibidas e exceções. Um procedimento pode ser compilado em estados e ações. Uma regra de continuidade narrativa pode modelar objetos e eventos e, em seguida, procurar um caminho

no qual um objeto é usado depois de ser deixado para trás sem recuperação. Um protocolo clínico pode modelar sequências de elegibilidade e intervenção. Um fluxo de trabalho científico pode modelar transformações de dados e requisitos de proveniência. Em cada caso, o executor simbólico deve operar em um programa digitado em vez de em prosa bruta.

A execução simbólica é especialmente útil para contra-exemplos. A análise abstrata pode indicar que uma violação é possível porque perdeu uma correlação. Uma busca de caminho pode tentar escolher identidades, horários, condições e exceções concretas que percebam a violação. Se o caminho for satisfatório, a testemunha pode ser processada como um cenário fundamentado. Se é insatisfatório, o sistema aprende que o aviso abstrato era espúrio sob a semântica atual.

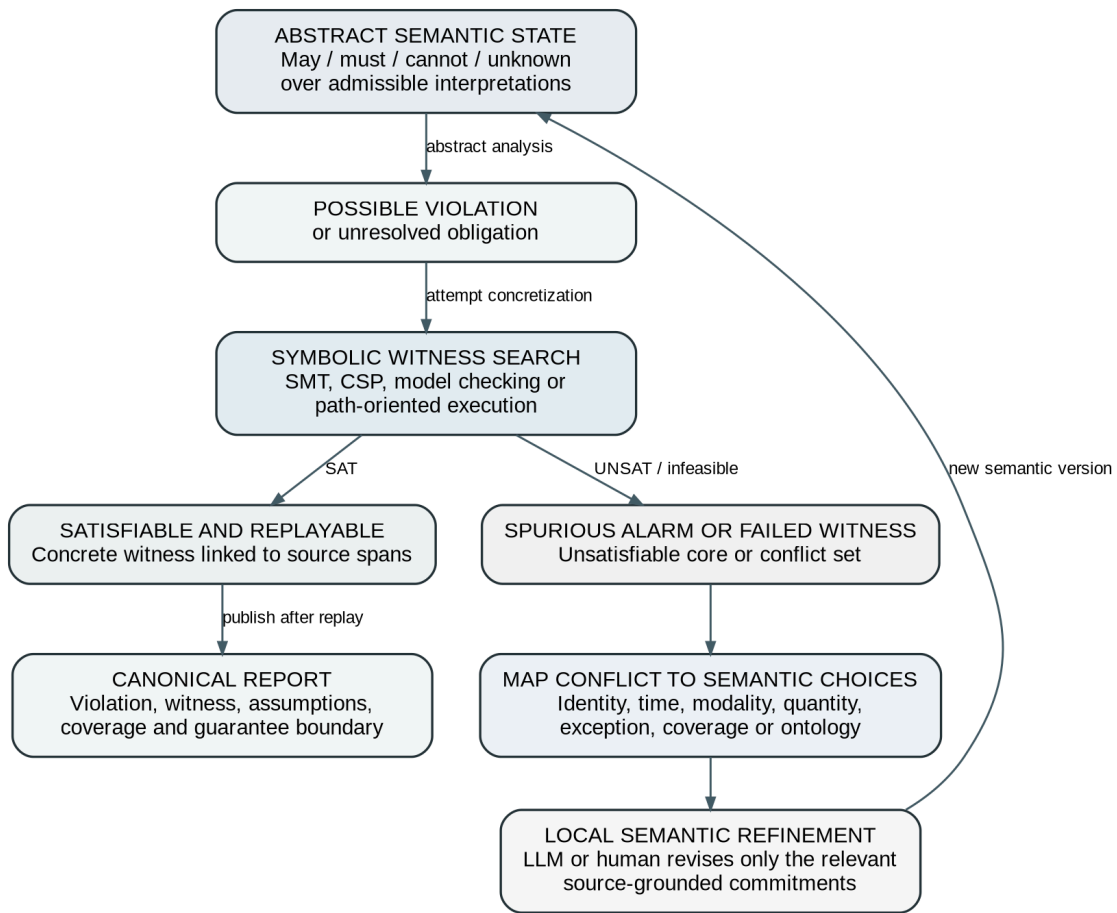


Figura 2. Refinamento semântico guiado por contra-exemplação. A análise abstrata identifica uma possível violação; a busca simbólica da testemunha confirma um caso rejogável ou expõe as distinções semânticas que devem ser refinadas.

Isso leva ao refinamento semântico guiado por contra-exemplos. O CEGAR tradicional começa com uma abstração grosseira do programa. A verificação do modelo encontra um contra-exemplo abstrato. Uma verificação de viabilidade determina se o contador exemplar corresponde a uma execução concreta. Se não, os predicados são adicionados para refinar a abstração e a análise se repete. Na configuração da linguagem, a abstração inclui escolhas semânticas. Um contra-exemplo espúrio pode surgir porque duas menções foram fundidas, uma exceção foi ignorada, um intervalo temporal foi ampliado ou uma declaração modal foi tratada como um fato.

Um ciclo de refinamento semântico deve preservar a explicação simbólica do fracasso. Suponha que um aviso de conformidade dependa da suposição de que “o conselho” em duas

seções denota o mesmo corpo. Replay simbólico mostra que o caminho se torna inviável se as entidades são distintas. O sistema mapeia o predicado de identidade às menções relevantes e faz uma pergunta focada: essas referências destinam-se a denotar o mesmo quadro? Um LLM pode reexaminar o contexto local e propor alternativas; um ser humano pode julgar, se necessário. O restante do modelo de documento permanece inalterado. Isso é mais confiável e eficiente do que pedir a um modelo para reler tudo e “tentar de novo”.

Núcleos insatisfatórios fornecem um mecanismo relacionado. Quando um conjunto de restrições é inconsistente, um solucionador muitas vezes pode retornar um subconjunto responsável pelo conflito. Mapear o núcleo de volta para a fonte identifica o conjunto mínimo de declarações e interpretações que não podem coexistir. Em uma revisão de políticas, isso pode revelar obrigações contraditórias. Na análise semântica, pode revelar que um quantificador, link de identidade e suposição temporal criam conjuntamente impossibilidade. O usuário recebe uma explicação fundamentada no conflito formal, em vez de uma crítica de modelo genérico.

Nem todo problema é melhor representado como execução de caminho. A verificação de modelos analisa sistemas de transição em relação às propriedades temporais. A lógica temporal linear pode expressar que algo eventualmente acontece, sempre se sustenta ou ocorre antes de outro evento. A árvore de cálculo e as lógicas probabilísticas expressam ramificação ou comportamento incerto. Sistemas de lógica natural-idiomática, como NL2TL e nl2spec, mostram que o LLMs pode construir especificações temporais quando o vocabulário e o domínio são controlados [Chen et al. 2023; Cosler et al. 2023]. O executor é então um verificador ou planejador de modelos, não um LLM.

A solução de restrições abrange um meio termo amplo. SAT representa escolhas booleanas. O SMT adiciona teorias como aritmética, matrizes, strings e funções não interpretadas. O CSP e o CP-SAT lidam com domínios, agendamento e otimização finitos. MaxSAT e restrições suaves expressam preferências. Muitas tarefas de linguagem natural – agendamento, alocação de recursos, verificação de consistência, relações familiares, quebra-cabeças e combinações de políticas – são melhor compiladas em restrições do que em código processual. O SatLM mostrou que as especificações declarativas mais um solucionador podem superar as abordagens auxiliadas pelo programa em tarefas aritméticas duras e lógicas [Ye et al. 2023]. O resultado suporta uma escolha de arquitetura central: o LLM deve descrever o problema, enquanto o solver realiza a pesquisa.

O planejamento formal fornece semântica de ação e busca direcionada a objetivos. O trabalho recente posiciona o LLMs como formalizadores de planejamento em vez de planejadores. Hao e seus colegas traduzem restrições de planejamento do mundo real em ferramentas formais de verificação, usam núcleos insatisfatórios para reparo e relatam sucesso de 93,9% no TravelPlanner em sua configuração, em comparação com linhas de base de modelo direto muito mais baixas [Hao et al. 2025]. Uma pesquisa de 2025 conclui que a principal oportunidade não é substituir os planejadores, mas usar o LLMs para construir e aperfeiçoar os modelos de planejamento [Tantakoun et al. 2025]. Esta é precisamente a divisão defendida aqui.

Os mecanismos de fluxo de dados e Datalog fornecem execução de ponto de correção em vez de busca de caminho. Eles são úteis para consequências monótonas e propagação em todo o documento. Os mecanismos de autômatos e padrões lidam com sequências de eventos ordenadas. As tabelas de decisão lidam com políticas finitas. A arquitetura deve encaminhar

cada problema para o formalismo mais simples que o expressa. A execução simbólica não deve se tornar um rótulo universal para cada componente simbólico.

LLMs pode ajudar a execução simbólica tradicional de maneiras mais seguras e arriscadas. Um papel mais seguro é a priorização do caminho. Um modelo pode identificar regiões de código que provavelmente contenham vulnerabilidades ou caminhos que provavelmente atingirão um alvo, enquanto o KLEE ou outro mecanismo mantém a decisão final de viabilidade. Trabalhos recentes, como o KLEECopilot, relatam melhorias substanciais de cobertura da pesquisa guiada por LLM, embora o status de pré-impressão e a sensibilidade do modelo exijam interpretação cautelosa [Chen et al. 2026]. O princípio é o som: as heurísticas aprendidas decidem onde procurar, não se um caminho é válido.

Um papel mais arriscado é substituir o solver. O AutoBug, apresentado por seus autores como execução simbólica de grande porte de modelo, decompõe o código em tarefas de raciocínio baseadas em caminhos e pede a um modelo para raciocinar diretamente sobre eles sem traduzir tudo em SMT. A abordagem é leve e agnóstica de linguagem, e os experimentos relatados melhoram a análise de programas baseada em LLM [Li et al. 2025]. Os autores caracterizam-no como aproximado. Pode analisar construções que são difíceis para os solucionadores convencionais, mas não fornece solidez clássica. Para uso de alta garantia, esse modelo é melhor tratado como um mecanismo em um portfólio cujos resultados exigem testes, replay ou confirmação independente.

Os LLMs também pode gerar testes, resumos ou modelos de hospedagem de caminho para APIs externas. Estudos relatam cobertura útil em programas que as ferramentas tradicionais lutam para lidar, mas o desempenho diminui com caminhos longos, APIs complexas, recursão e classificação de caminho inviável. Esses métodos são promissores porque os ambientes de software contêm semânticas que são caras para axiomatizar. Seu status de resultado deve permanecer “candidato” até que a execução concreta confirme o caminho ou um solucionador prova a condição.

A segurança é uma grande preocupação. As regras em linguagem natural e os documentos de origem são entradas não confiáveis. Um documento malicioso pode tentar a injeção imediata, fabricar referências ou explorar um tradutor gerador de código. Os artefatos formais devem ser dados, não código executável irrestrito. Quando os algoritmos personalizados são necessários, eles devem ser revisados extensões de host com resumos bloqueados, entradas e saídas digitadas, limites de recursos e sem autoridade ambiental. As invocações do Solver precisam de orçamentos de tempo e memória. Uma testemunha deve incluir a versão exata da teoria, compilador e motor que a produziu.

O padrão de análise combinada agora é claro. Uma análise de fluxo de dados e semântica grosseira identifica possíveis problemas em um longo documento. Os motores formais especializados testam apenas as fatias relevantes. A execução simbólica constrói testemunhas para reivindicações dependentes do caminho. A verificação de modelos lida com comportamentos temporais. Os solucionadores de retenção lidam com escolhas combinatórias. Testemunhas fracassadas e núcleos de solucionados impulsionam o refinamento semântico local. Toda conclusão aceita é retornada com um status e proveniência que reflete o elo mais fraco da cadeia.

Esta arquitetura não promete a verificação total da linguagem irrestrita. Promete algo mais operacional: cada reivindicação formal tem um significado executável; cada símbolo tem uma fonte ou suposição declarada; cada contra-exemplo pode ser reproduzido; cada reivindicação

de ausência está ligada à cobertura; e cada ambiguidade restante é representada em vez de oculta. A interpretação abstrata fornece escala e agregação conservadora. A execução simbólica fornece precisão e testemunhas. Sua combinação é uma das bases mais plausíveis para a linguagem natural executável.

4. O que a Era dos LLMs Realmente Alcançou

4.1 Autoformalização, Raciocínio Assistido por Programas, Lógica, Especificações Temporais e Planejamento

A chegada de grandes modelos de linguagem mudou o limite prático da compilação semântica. Os sistemas anteriores exigiam gramáticas específicas de tarefas, formas lógicas supervisionadas ou léxicos cuidadosamente projetados. Os LLMs pode propor artefatos formais a partir de descrições abertas com pouco ou nenhum treinamento específico de tarefas. Isso expande substancialmente a cobertura, mas também cria a tentação de inferir mais com ganhos de referência do que os mecanismos justificam. A questão relevante não é simplesmente se um método melhora a precisão da resposta. É que os erros foram transferidos para um componente simbólico, cujos erros permanecem na formalização, e que evidência independente existe de que o artefato formal representa a fonte.

Os modelos de linguagem auxiliados pelo programa, ou PAL, fornecem um ponto de partida limpo. O LLM lê um problema, gera um programa Python e delega computação para o interpretador [Gao et al. 2022]. No GSM8K, o sistema relatado melhorou substancialmente ao longo da cadeia de pensamento. O mecanismo é atraente porque os modelos de linguagem são muitas vezes melhores em decompor um problema do que na realização de aritmética longa exatamente. Uma vez que a decomposição esteja correta, o Python fornece operações determinísticas e estados.

A limitação do PAL é igualmente instrutiva. O intérprete certifica apenas que ele executou o programa gerado. Não certifica que o programa modele fielmente a questão da linguagem natural. Uma variável equivocada, condição omitida ou uma fórmula errada pode ser executada sem erros. O sucesso do sistema, portanto, depende de uma distribuição em que os programas são curtos, exemplos restringem fortemente a estrutura e os erros de execução revelam muitos erros. Para o raciocínio documental, análise de políticas e interpretação científica, o mapeamento é menos restrito e o intérprete fornece feedback mais fraco.

SatLM substitui o código processual por restrições declarativas [Ye et al. 2023]. O modelo indica variáveis e relacionamentos, e um mecanismo de satisfatibilidade procura uma solução. Isso pode ser mais fácil do que gerar um algoritmo, porque o modelo não precisa decidir a ordem das operações. As representações declarativas também permitem vários modelos, provas de inviabilidade e depuração no nível de restrição. Os ganhos relatados em tarefas de raciocínio duro suportam a ideia de que a escolha do solver deve seguir a estrutura do problema. Um cronograma é naturalmente um problema de restrição; um processo dinâmico pode ser uma máquina de estado; um problema de palavra aritmética pode ser um programa ou restrições.

A Lógica-LM formaliza problemas de lógica em linguagem natural e invoca os solucionadores simbólicos [Pan et al. 2023]. Ele também usa erros de solver para solicitar o auto-refinamento. Em vários conjuntos de dados, os autores relatam grandes melhorias médias em relação ao estímulo padrão e da cadeia de pensamento. A arquitetura demonstra dois princípios úteis. Primeiro, o motor simbólico remove a inconsistência dedutiva quando existe uma formalização válida. Em segundo lugar, as mensagens de erro formal fornecem um sinal de reparo melhor do que uma solicitação genérica para reconsiderar.

No entanto, o reparo de sintaxe não é um reparo semântico. Uma fórmula pode ser bem formada, satisfável e errada. O pipeline da Logic-LM pode corrigir predicados malformados ou falhas de solucionadores, mas pode não detectar que “alguns” se tornaram “tudo” ou que uma exceção foi omitida. Trabalhos subsequentes, como o Logic-LM ++, adicionam refinamento de várias etapas, e o LTRAG recupera exemplos de formalização guiados por pensamento, aumentando as melhorias em relação ao Logic-LM e LINC no FOLIO e AR-LSAT [Kirtania et al. 2024; Hu et al. 2025]. Esses ganhos sugerem que a própria formalização se beneficia de exemplos e críticas estruturadas. Eles não eliminam a necessidade de avaliar a fidelidade semântica diretamente.

LINC torna a modularidade explícita: um LLM serve como um analisador semântico, a lógica de primeira ordem representa o problema, e um provador teorema executa a dedução [Olausson et al. 2023]. No ProofWriter, a combinação relatada produziu grandes ganhos sobre os sistemas de cadeia de pensamento, incluindo modelos substancialmente maiores que o analisador. A análise de erros mostrou modos de falha complementares: o sistema simbólico evita muitos erros de raciocínio, enquanto as falhas de formalização permanecem. Esta é exatamente a evidência esperada de uma arquitetura federada. O sistema é mais forte quando a tarefa em linguagem natural já corresponde a uma teoria compacta de primeira ordem.

O FOLIO fornece uma referência útil porque contém problemas de raciocínio em linguagem natural de autoria humana emparelhados com a lógica de primeira ordem verificada [Han et al. 2024]. Avalia tanto o raciocínio quanto a tradução NL-FOL. O conjunto de dados continua desafiador, mesmo para modelos fortes, o que prejudica a ideia de que a formalização de primeira ordem é uma etapa de pré-processamento resolvida. O FOLIO também ilustra a importância da complexidade de autoria humana. Os conjuntos de dados de regras sintéticas podem tornar a linguagem incomumente regular e permitir que os modelos explorem modelos. Exemplos humanos introduzem variação lexical, conhecimento do mundo e escolhas de escopo mais realistas.

O FoVer adota outra abordagem verificando o raciocínio em linguagem natural através da lógica de primeira ordem [Pei et al. 2025]. Em vez de tratar o texto de raciocínio fluente como auto-autenticador, ele constrói obrigações formais e as verifica. Este é um padrão promissor para a geração com restrições de prova: um LLM pode propor um argumento, mas um verificador aceita apenas etapas inferenciais que correspondem a consequências formais válidas. A questão restante é o alinhamento entre cada frase e os predicados formais usados para verificá-la.

A pesquisa de autoformalização trata cada vez mais a tradução como uma disciplina distinta, em vez de um truque rápido. Pesquisas publicadas em 2025 catalogam a tradução de estratões de teoremas, linguagens lógicas e especificações formais, enfatizando a necessidade de ciclos de feedback, recuperação, representações intermediárias e benchmarks que separam a sintaxe da semântica [Weng et al. 2025]. O trabalho direto NL-to-FOL relata progresso significativo, mas também erros sistemáticos no conhecimento implícito, escopo de quantificador e design de ontologia [Yang et al. 2024; Lalwani et al. 2025]. Adicionar conhecimento de fundo implícito pode melhorar o desempenho da tarefa, aumentando o risco de que o sistema formal prove uma premissa não licenciada pela fonte.

O tratamento do conhecimento de fundo é uma escolha arquitetônica decisiva. Em um benchmark de quebra-cabeça, as premissas de senso comum podem ser implicitamente esperadas. Na verificação legal ou científica, a introdução de instalações não declaradas pode

invalidar o resultado. Um sistema rigoroso deve distinguir premissas de origem, premissas de papel de autoridade, axiomas de domínio e suposições sugeridas por modelo. O solucionador pode raciocinar sobre todos eles, mas a explicação final deve mostrar quais conclusões dependem de qual autoridade. Se uma suposição é essencial e não verificada, o resultado deve ser condicional.

A tradução temporal-lógica traz diferentes desafios. O NL2TL usa representações levantadas para aprender a estrutura temporal em todos os domínios e relata forte adaptação com relativamente poucos dados de domínio alvo [Chen et al. 2023]. A estratégia de elevação esconde proposições atômicas concretas enquanto aprende padrões lógicos reutilizáveis. Esta é uma visão valiosa do design: separar a estrutura de “sempre antes”, “eventualmente depois” ou “até” da base de eventos específicos de domínio.

nl2spec enfatiza a interação [Cosler et al. 2023]. O sistema mapeia partes da exigência de linguagem natural para partes da fórmula temporal gerada, permitindo que os usuários repedam erros de tradução locais. Esta interface reconhece que a ambiguidade nem sempre pode ser resolvida automaticamente. Também sugere como o refinamento semântico baseado em LLM deve funcionar: o usuário deve ver a decisão linguística, não apenas uma cadeia de operadores temporais.

O fundamentação continua a ser o ponto frágil. Uma fórmula temporal pode ter a forma correta enquanto suas proposições atômicas se referem aos eventos ou objetos errados. O trabalho em equivalência lógica fundamentada relata que os mecanismos de fundamentação explícitos melhoram a correção, reforçando a distinção entre sintaxe de fórmula e fixação semântica. Para um tempo de execução em linguagem executável, os átomos temporais devem ser referências a observações digitadas com spans e identidades, não rótulos de forma livre inventados pelo tradutor.

O planejamento automatizado fornece um back-end simbólico particularmente maduro. As linguagens de planejamento definem tipos, predicados, ações, pré-condições, efeitos, estados iniciais e metas. Os planejadores então procuram uma sequência de ação válida. LLMs são planejadores pobres de horizonte longo quando eles devem manter o estado e as restrições internamente, mas eles podem ser eficazes na formalização de um problema para um planejador. Uma pesquisa de 2025 identifica a construção e o refinamento do modelo como o papel central do LLMs nesta área [Tantakoun et al. 2025].

Hao e seus colegas demonstram uma versão rigorosa do padrão no planejamento do mundo real [Hao et al. 2025]. O sistema formaliza restrições, invoca ferramentas de verificação e usa núcleos insatisfeitos para reparar problemas. Os autores relatam um sucesso de 93,9% no TravelPlanner em seu ambiente experimental, muito acima das linhas de base diretas LLM com as quais se comparam. O resultado é importante porque mostra que as ferramentas formais podem transformar a confiabilidade do planejamento quando o problema é representável. Isso não significa que o LLM se tornou um planejador de som. Isso significa que o planejador e o verificador carregam o estado de horizonte longo, enquanto o LLM constrói e repara a instância formal.

O planejamento também expõe restrições ocultas. As solicitações em linguagem natural incluem expectativas de senso comum, preferências e condições de viabilidade que podem não ser declaradas como requisitos formais. Sistemas como o LRPlan tentam a colaboração entre a linguagem e os modelos de raciocínio para restrições explícitas e implícitas, enquanto benchmarks como o PlanningArena mostram que até mesmo modelos fortes lutam com a

seleção de ferramentas, memória de contexto e consistência lógica [Karthikeyan et al. 2025; Zheng et al. 2025]. Um sistema de alta garantia deve formalizar as restrições implícitas de uma fonte confiável ou rotulá-las como suposições. “Comunidade” não é uma categoria de proveniência suficiente.

RuleArena e benchmarks relacionados testam LLMs contra sistemas de regras realistas e acham que o aumento de ferramentas ajuda, mas não torna a formalização trivial. As tarefas do SemEval 2026 sobre conteúdo e raciocínio formal mostram ainda que os modelos são influenciados pelo conteúdo semântico, mesmo quando a forma lógica deve determinar a resposta. As entradas neuro-simbólicas modulares usam FOL e Z3 para reduzir o viés de conteúdo, mas a tradução formal e o ruído multilíngue permanecem difíceis [Grimaldi 2026; Kartáč et al. 2026; Wang et al. 2026]. Esses resultados reforçam a necessidade de desembaraçar a interpretação linguística da inferência formal.

Uma das observações mais importantes nesta literatura é que sistemas bem-sucedidos usam linguagens intermediárias específicas de destino. O PAL usa o Python. SatLM usa restrições declarativas. Lógica-LM e LINC usam lógica. NL2TL usa a lógica temporal. Os sistemas de planejamento usam modelos semelhantes a PDDL. O SymGPT usa uma gramática adaptada às regras do ERC. O campo não descobriu uma representação formal universal para a linguagem natural. Os LLMs pode rotear a linguagem em vários ecossistemas formais quando exemplos, esquemas e feedback estão disponíveis.

Isso suporta uma arquitetura de portfólio em vez de um “bycode de linguagem” universal. Um pivô semântico pode preservar evidências, entidades, eventos, escopo e modalidade, enquanto um seletor de formalismo escolhe o alvo de execução. O seletor em si pode ser assistido por um LLM, mas sua decisão deve ser validada contra contratos de capacidade. Um formalismo é compatível somente se puder expressar as distinções necessárias e as observações fundamentadas necessárias existirem. Um planejador não deve ser selecionado para uma consulta puramente taxonômica; a lógica de primeira ordem não deve se aproximar silenciosamente da dinâmica temporal incerta; um mecanismo relacional simples não deve reivindicar decidir exceções legais desvinculadas.

A evidência empírica, portanto, apoia uma afirmação mais restrita, mas substancial. LLMs pode melhorar drasticamente a construção de objetos formais executáveis. Os sistemas simbólicos externos podem melhorar a correção, consistência e interpretabilidade. O feedback do Solver pode orientar o reparo. Os resultados mais fortes aparecem em domínios limitados com linguagens explícitas, esquemas claros e respostas avaliáveis. O desafio não resolvido é a compilação semântica: garantir que o objeto formal preserve o significado relevante da fonte e que todas as premissas sejam fundamentadas e autorizadas.

4.2 Análise de Programas Assistida por LLM, Exploração Simbólica, Transformadores Abstratos e NLP Certificado

A interação entre LLMs e análise de programas é executada em ambas as direções. Os LLMs pode ajudar ferramentas formais a lidar com especificações, padrões de código e ambientes que são difíceis de modelar. As ferramentas formais podem verificar o código gerado pelo LLMs, verificar as redes neurais e restringir os componentes aprendidos. Esta literatura é especialmente valiosa para a linguagem natural executável porque testa a mesma questão arquitetônica sob semântica mais clara: onde pode um modelo probabilístico contribuir sem se tornar o oráculo final?

A análise estática tradicional depende de abstrações desenhadas à mão, funções de transferência, resumos e modelos de ambiente. A execução simbólica depende da busca de caminho e das codificações do solver. Essas ferramentas oferecem garantias baseadas em princípios, mas lutam com recursos de linguagem dinâmica, código de biblioteca, manipulação de strings complexas, serviços externos e escalabilidade. Os LLMs pode reconhecer a intenção do código, propor invariantes, resumir funções, identificar locais de bugs prováveis e gerar testes. Uma pesquisa de 2025 da análise assistida por LLM categoriza o trabalho em métodos estáticos, dinâmicos e híbridos e enfatiza tanto a oportunidade quanto os riscos das saídas probabilísticas [Wang et al. 2025].

Um papel arquitetônico é a orientação heurística. Um executor simbólico pode manter sua semântica e usar um LLM para classificar caminhos, identificar regiões vulneráveis ou selecionar limites de loop. Se a heurística estiver errada, a cobertura pode sofrer, mas os caminhos aceitos permanecem resolvidos verificados. Isso é análogo ao uso de um modelo neural para pedidos de busca em prova de teoremas. A distinção entre orientação e aceitação é crítica. Uma pontuação aprendida pode determinar onde os recursos são gastos; não deve determinar se uma condição de caminho é satisfatória.

KLEECopilot representa essa direção. O preprint de 2026 usa orientação LLM para direcionar KLEE para vulnerabilidades prováveis e introduz heurísticas para a fuga de loop, relatando melhorias substanciais na cobertura de blocos e linhas básicas e dezenas de violações exclusivas [Chen et al. 2026]. Como um pré-impressão recente, os resultados exigem replicação independente e sensibilidade aos assuntos familiares modelo. No entanto, o padrão de design é plausível: LLMs contribui com priorização semântica que a pesquisa clássica não tem, enquanto KLEE mantém a semântica de execução.

Outro papel é a geração de testes de satisfação de caminhos. Um modelo pode ler código e propor entradas esperadas para atravessar um caminho de destino. Execução concreta então valida se o caminho foi alcançado. Essa abordagem pode lidar com as convenções ou formatos de dados API que são onerosos para o SMT. Estudos relatam taxas de sucesso úteis, mas também fracassos em longos caminhos, recursão, ramos inviáveis e bibliotecas complexas. A interpretação segura é que o LLM é um gerador de teste. Um concreto, traço de instrumentação ou solucionador permanece o validador.

O AutoBug adota uma abordagem mais radical usando o LLMs para executar o raciocínio decomposto pelo caminho diretamente [Li et al. 2025]. Em vez de traduzir todas as restrições em uma teoria do solver, ela apresenta condições de caminho no nível do código para o modelo. Isso pode ser agnóstico de linguagem e pode analisar semântica que é difícil de codificar. Os autores relatam melhorias em vários benchmarks de análise de programas,

particularmente para modelos menores. Eles também caracterizam a análise como aproximada. O sistema não oferece solidez clássica, e suas conclusões podem variar com o modelo.

AutoBug é valioso porque mostra onde os métodos simbólicos falham operacionalmente. Muitos programas reais chamam bibliotecas opacas, manipulam objetos complexos ou dependem de contratos informais. Um modelo de linguagem pode inferir comportamento provável de nomes, comentários e padrões de código aprendidos. Mas essa flexibilidade é a mesma fonte de incerteza. Em um portfólio orientado a garantia, a análise LLM aproximada deve gerar candidatos, resumos ou verificações priorizadas. Os resultados devem ser confirmados por testes, contratos, monitoramento de tempo de execução ou um motor formal sempre que uma reivindicação de segurança é feita.

LLMs pode propor invariantes para interpretação abstrata. Um candidato invariante pode acelerar a prova se um verificador verificar se ele verifica se ele detém inicialmente, é preservado por transições e implica a propriedade. Esta é uma poderosa divisão do trabalho: a descoberta invariante é criativa e combinatória, enquanto a verificação invariante é mecânica. Padrões semelhantes aparecem na prova de teoremas e na síntese de programas. O modelo de linguagem procura; o verificador aceita.

A SAIL estende este princípio à síntese de transformadores abstratos [Gu et al. 2026]. Construir um transformador que seja ao som e preciso para um operador não linear é difícil. SAIL pede LLMs para gerar candidatos, mas impõe restrições sintáticas e semânticas e usa um custo matematicamente fundamentado para a falta de bondade. O sistema relatado sintetiza transformadores de som global em vários domínios, combina com projetos manuais e encontra transformadores precisos que não estavam na literatura. O significado vai além da verificação da rede neural. É uma evidência de que o LLMs pode automatizar a construção do método formal quando cada artefato aceito está sujeito a obrigações de prova independentes.

O contraste com pedir a um LLM para “agir como um intérprete abstrato” é nítido. Mitchell e seus colegas levaram modelos a raciocinar através da interpretação abstrata nos programas SV-COMP [Mitchell et al. 2025]. Os modelos podem seguir estilos composicionais ou de ponto fixo em alguns casos, mas cometeram erros lógicos e alucinaram em estruturas complexas. O resultado é consistente com um padrão geral: os modelos podem imitar processos formais e gerar candidatos úteis, mas a confiabilidade diminui quando o processo requer estado exato persistente. Um intérprete abstrato real deve executar as operações de rede; o modelo pode sugerir domínios, predicados ou explicações.

A verificação formal dos modelos neurais NLP é outro ramo importante. O trabalho de robustez certificado pergunta se a previsão de um modelo permanece inalterada em todas as entradas em um conjunto de perturbação definido. Jia e seus colegas usaram a propagação de intervalo para substituições de palavras e relataram 75% de precisão adversária no IMDB e SNLI sob as famílias de substituição construídas [Jia et al. 2019]. O SAFER utilizou suavização aleatória e pôde ser aplicado a modelos de caixa-preta, incluindo BERT [Ye et al. 2020]. As incorporações de reconhecimento robustas apertaram os limites de certificação, melhorando a precisão robusta certificada no IMDB de 76,73 para 84,78% na configuração relatada [Wang et al. 2023].

A verificação do transformador é difícil porque a auto-atenção contém produtos, operações softmax e correlações entre as posições. Os domínios especializados melhoram os limites. A Interpretação de Resumos Parametrizados para Verificação Transformadora introduziu

domínios afez parametrizados para produtos internos e relatou limites mais apertados e verificação de instâncias que os métodos anteriores não podiam provar [Huang et al. 2026]. Esta linha de trabalho demonstra que a análise formal pode ser dimensionada para arquiteturas NLP não triviais quando a propriedade e o conjunto de entrada são definidos com precisão.

O escopo da garantia não deve ser desfocado. Um classificador certificado pode ser robusto para cada substituição extraída de um conjunto especificado e ainda estar errado. Uma lista de substituição pode incluir “sinônimos” contextualmente inadequados ou omitir paráfrases significativas. A certificação diz que o modelo é invariante sobre a família de perturbação formal. Não estabelece equivalência semântica na linguagem humana. Esta distinção deve ser explícita em qualquer sistema de linguagem executável que utilize modelos de extração certificados.

Uma integração útil anexaria um certificado de robustez a uma observação semântica. Suponha que um extrator de evento identifique que uma sentença declara uma obrigação. Um modelo certificado pode mostrar que esta classificação é estável sob uma família controlada de substituições lexicais. A observação ainda é proposta, mas tem uma propriedade de estabilidade adicional. A análise simbólica downstream pode registrar essa evidência sem atualizar a observação para o fato. Esse status em camadas é mais informativo do que uma pontuação de confiança.

Os métodos formais também verificam o código gerado pelo LLMs. Geração de teste, análise estática, verificação de tipos e assistentes de prova podem detectar muitos erros. Os sistemas de síntese de programas usam cada vez mais o compilador e testam o feedback para o reparo iterativo. O mesmo padrão deve reger os circuitos semânticos gerados e o código de formalização. Um agente de codificação pode criar uma implementação de regras, mas a publicação deve exigir compilação, testes de mutação, casos de referência em linguagem natural, repetição e verificadores independentes. Um LLM não deve ser autorizado a criar um comportamento executável oculto dentro de um artefato de teoria declarativa.

A literatura sobre modelagem de protocolo a partir da linguagem natural oferece um resultado preocupante. O trabalho apresentado no ICSE 2025 investigou a geração de modelos Tamarin ou ProVerif a partir da documentação do protocolo. O uso ingênuo de LLM foi ineficaz; o pipeline bem-sucedido exigiu decomposição semântica, desambiguação humana limitada e transformações intermediárias sonoras. Gerou modelos de escala moderada corretos para 10 dos 18 protocolos reais [Mao et al. 2025]. O resultado é significativo porque os protocolos contêm papéis, mensagens, suposições criptográficas, estado e ordem temporal. Também mostra que mesmo uma documentação estruturada forte não compila automaticamente em modelos formais de segurança.

Os casos de falha são instrutivos. A documentação omite pressupostos que os especialistas inferem. O mesmo termo pode denotar um tipo de mensagem, um estado ou uma instância. As propriedades de segurança podem ser declaradas informalmente ou distribuídas por seções. Um modelo de protocolo formal deve tornar as capacidades de adversários explícitas. Estes são análogos diretos do raciocínio documental. A solução não foi um prompt maior, mas uma arquitetura que separa a análise semântica, desambiguação e transformação verificada.

Revisões externas do campo mais amplo convergem em uma conclusão semelhante. A revisão neuro-simbólica de 2026 NLP favorece uma investigação mais forte das arquiteturas federativas, mas adverte que as comparações atuais não são controladas e que nenhum sistema federativo de escala fundamental foi estabelecido [Chatzikyriakidis and Lappin 2026]. A

pesquisa de planejamento-formalizador trata LLMs como construtores e refinarias de modelos formais, não substituidores de planejadores [Tantakoun et al. 2025]. O levantamento de análise de programas enfatiza os sistemas híbridos e a necessidade de gerenciar alucinação e a reprodutibilidade [Wang et al. 2025]. Estas são linhas independentes de evidência para a mesma divisão do trabalho.

O campo, portanto, alcançou mais do que um padrão superficial “LLM plus solver”. Ele demonstrou reparos guiados por solucionadores, formalizações verificadas, planejamento formal, orientação de caminhos, proposta invariante, síntese de transformadores e robustez certificada. Também expôs um limite estável. Quando o LLM controla a aceitação final, as garantias tornam-se aproximadas. Quando o LLM propõe artefatos que um sistema formal separado verifica, garantias fortes podem ser retidas para a propriedade verificada. A questão sem resposta é como tornar o limite de linguagem natural para artegraficamente disciplinado.

A arquitetura proposta mais adiante neste livro aplica as lições de análise de programas diretamente. As transformações semânticas candidatas são tratadas como um código sintetizado. Eles digitaram contratos, mapas de origem, limites de recursos e testes. Os domínios abstratos são registrados em vez de improvisados durante uma execução. Os predicados de refinamento são sugeridos pelo LLMs, mas validados através de distinções concretas. As carteiras de Solver expõem objetos de prova ou testemunhas. O renderizador de linguagem consome resultados canônicos, não traços brutos. Neste projeto, a flexibilidade do LLM é um ativo precisamente porque é cercada por portões de aceitação simbólica.

5. Por que os Sistemas de Linguagem Executável Falham

5.1 Fundamentação, Identidade, Formalizações Espúrias, Ontologias e Conhecimento de Mundo

O erro dominante em sistemas de linguagem executável não é uma dedução incorreta de premissas aceitas. É a admissão de premissas, objetos ou relações erradas. A fundamentação é o processo pelo qual os símbolos adquirem referência operacional: um predicado está conectado a uma frase, uma entidade a uma ou mais menções, um evento a evidência, uma expressão de tempo a um intervalo, um campo de banco de dados a um conceito ou uma ação para uma capacidade de ambiente. Um programa formal sem fundamentação confiável é exato no sentido menos útil. Ele executa consistentemente ao descrever o mundo errado.

O fundamentação começa com a localização da fonte. Toda observação semântica deve ser vinculada a evidências de origem exata ou explicitamente marcada como uma suposição externa. Uma citação deve ser revalidada contra a revisão de fonte imutável. Um resumo do documento não pode substituir a fonte porque pode omitir qualificações. Os resultados da recuperação não podem suportar uma reivindicação sobre a ausência, a menos que o processo de recuperação seja concluído para o domínio relevante. Esta disciplina de origem impede uma falha comum nos sistemas LLM: um modelo produz texto de suporte plausível que não ocorre no documento, e a saída estruturada downstream dá à invenção uma aparência de autoridade.

O fundamentação lexical conecta a linguagem à ontologia. A frase “aprovação” pode denotar uma ação, um estado, um artefato assinado, uma permissão ou uma decisão institucional. Um esquema de domínio deve escolher e documentar a interpretação necessária por uma regra. Os LLMs é útil porque pode inferir contexto e propor mapeamentos, mas um mapeamento deve ser testado em exemplos e alternativas. Uma correspondência de string entre uma frase e um nome de tipo não é compatibilidade semântica. A documentação nllAgent adverte explicitamente contra o mapeamento de dois conceitos simplesmente porque seus rótulos correspondem; requer um adaptador ou sonda semântica quando os significados diferem.

A fundamentação da identidade é mais difícil e mais consequente. A análise de documentos geralmente precisa saber se duas menções denotam a mesma pessoa, organização, objeto, conjunto de dados, requisito ou evento. Nomes iguais não implicam identidade, e nomes diferentes não implicam diferença. Títulos mudam, os apelidos ocorrem, os pronomes são ambíguos, as organizações têm subdivisões, e menções fictícias ou citadas podem não pertencer ao mundo de origem. Uma camada de identidade segura separa menções de hipóteses de entidade e candidatos a identidade. Ele registra o produtor, a evidência, o escopo, o mundo e as alternativas para cada link.

A fusão prematura de identidade cria erros em cascata. Em uma narrativa, dois personagens com o mesmo sobrenome podem ser mesclados, produzindo violações de continuidade impossíveis. Em um documento regulatório, “a Autoridade” pode se referir a diferentes órgãos em diferentes jurisdições. Na documentação do software, “cliente” pode denotar um usuário, um componente de aplicativo ou um endpoint HTTP. Uma vez mescladas, as regras posteriores tratam todas as propriedades como pertencentes a um objeto. O erro

torna-se difícil de diagnosticar porque o solver vê um identificador consistente. Um sistema conservador deve permitir a identidade não resolvida e retornar desconhecido quando uma regra requer um link que não foi estabelecido.

O fundamentação temporal tem camadas semelhantes. As datas podem ser explícitas ou relativas. “Dentro de trinta dias” requer um evento âncora e uma política de calendário. “Antes da liberação” pode se referir a uma versão planejada ou real. “Após aprovação” pode significar imediatamente após ou em qualquer momento posterior. Jurisdições definem dias de negócios de forma diferente. Um solucionador temporal pode gerenciar intervalos e ordenar exatamente uma vez que essas escolhas são feitas, mas não pode inferir a âncora pretendida de um rótulo arbitrário. Observações temporais devem, portanto, incluir a expressão textual, valor normalizado ou restrição, âncora, calendário, fuso horário, confiança e alternativas.

O fundamentação normativo é frequentemente mal administrado. “Deve”, “deveria”, “pode”, “pode” e “espera-se que” tenham força específica do domínio. As convenções de redação jurídica diferem da linguagem comum. Uma exceção pode derrotar uma regra, restringir seu escopo ou criar uma permissão separada. Autoridades conflitantes podem ter prioridades. Uma simples representação booleana de compliance perde essas distinções. Lógica dentária, prioridades de regras, tabelas de decisão ou sistemas desviários podem modelá-las, mas o mapeamento source-to-rule continua sendo um julgamento linguístico e institucional.

As formalizações espúrias surgem quando o programa errado produz o resultado esperado. A análise semântica fracamente supervisionada tornou esse problema explícito, mas é difundido na avaliação LLM. Uma fórmula gerada pode resolver um benchmark porque os exemplos têm modelos regulares. Uma consulta text-to-SQL pode retornar as linhas atuais corretas, apesar de uma junção errada. Um modelo de planejamento pode encontrar um plano porque uma restrição omitida é irrelevante na instância testada. Um circuito de regras pode passar dez exemplos enquanto codifica o conceito errado. A precisão da resposta final por si só não consegue distinguir esses casos.

A defesa apropriada é a mutação semântica. O avaliador muda um aspecto da fonte ou ambiente, preservando outros, em seguida, verifica se a formalização muda de forma esperada. Se “tudo” se torna “alguns”, o quantificador deve mudar. Se uma exceção for removida, o caminho anteriormente isento deve ser proibido. Se duas entidades forem renomeadas para a mesma string, a identidade não deve se mesclar automaticamente. Se a instância do banco de dados mudar, uma consulta SQL incorreta acidentalmente deve falhar. O teste de mutação avalia o significado do programa, em vez de apenas seu comportamento em um dispositivo elétrico.

A geração de contra-exemplos pode comparar formalizações concorrentes. Dado dois programas lógicos ou executáveis, um solver pode procurar um modelo em que suas saídas diferem. O cenário resultante é traduzido para a linguagem natural e apresentado a um revisor. Isso transforma uma comparação de fórmulas opacas em uma questão concreta: “Sob interpretação A, um supervisor que não completou o treinamento pode aprovar; sob a interpretação B, eles não podem. Qual é a intenção?” O método é especialmente útil quando vários LLMs produzem formas plausíveis, mas não equivalentes.

O design de ontologia cria outro modo de falha. Um esquema universal é atraente para reutilização, mas pode ocultar desentendimentos de domínio e forçar conceitos em categorias inadequadas. Um esquema extremamente estreito, por outro lado, produz muitas línguas locais incompatíveis. Um design em camadas é preferível. A camada superior define fonte, evidência,

hipóteses de identidade, mundos, tempo, modalidade, status, cobertura e lacunas. Os pacotes de domínio definem tipos e relações substantivas. Os pacotes de mapeamento declaram correspondências, perdas e suposições entre domínios.

A compatibilidade com ontologia deve ser operacional, não retórica. Um circuito declara os tipos e garante que ele consome. Um produtor declara quais tipos pode se materializar, sob quais status e modos de cobertura. A publicação verifica se existe um produtor compatível. O tempo de execução verifica se o documento concreto produziu as observações necessárias. Se o circuito precisa de identidades confirmadas pelo homem, mas apenas candidatos a identidade proposto por modelo existem, ele não pode ser executado na garantia reivindicada. Se uma regra temporal precisa de um calendário jurisdicional ausente, o resultado é uma lacuna operacional, não uma data adivinhada.

O conhecimento do mundo é particularmente perigoso. Os LLMs contém amplo conhecimento, mas não há proveniência estável, data efetiva ou jurisdição. Tratar a memória do modelo como uma base de fatos formal cria dependências invisíveis e conclusões obsoletas. Um sistema simbólico deve carregar pacotes de conhecimento explícitos com identificadores de fonte, resumos, intervalos efetivos, jurisdições, políticas de conflito e limites máximos de garantia. O pacote pode ser curado, recuperado ou assistido por modelo, mas suas reivindicações aceitas são artefatos externos. Um pacote legal pode expirar. Um pacote científico pode ser substituído. Um pacote de mundo fictício pode deliberadamente contradizer a realidade atual.

Os conflitos entre fontes não devem ser apagados através da votação por maioria, a menos que a tarefa defina tal política. O sistema pode precisar de raciocínio paraconsistente, prioridade de fonte, versão temporal ou mundos separados. Uma reivindicação pode ser apoiada por uma fonte e contrariada por outra. O resultado canônico deve preservar ambos e declarar qual política, se houver, os resolve. Isso é essencial para pesquisas científicas e análises regulatórias, onde a discordância é informação e não ruído.

Suposições de fundo devem ser visíveis. Os benchmarks lógicos geralmente assumem que os nomes denotam indivíduos distintos ou que as categorias são disjuntos. Documentos reais raramente afirmam tais axiomas. Um LLM pode inseri-los porque eles tornam o quebra-cabeça solucionável. Em um sistema de pesquisa ou conformidade, essas suposições devem aparecer em uma camada de autoridade separada. A resposta final pode então afirmar que uma conclusão segue somente se a suposição se mantiver. Isso impede que o conhecimento implícito do mundo se mascare como evidência documental.

A cobertura é o fundamentação da ausência. Para concluir que um documento não contém uma aprovação necessária, o sistema deve saber o que foi pesquisado, quais seções e canais foram incluídos, como o conceito foi reconhecido e se o método de busca estava completo para a reivindicação. Uma varredura lexical pode estar completa para uma frase exata, mas não para as paráfrases. A recuperação pode ser um recall alto, mas não o mundo fechado. Um registro com curadoria humana pode estar completo para uma liberação. A cobertura deve ser um objeto de primeira classe consumido por regras negativas.

A arquitetura nllAgent contém uma versão importante deste princípio. O LongTextJS registra cobertura e lacunas ao lado de observações. A compatibilidade do circuito verifica se o programa real satisfaz o status e a cobertura necessários. O conhecimento ausente não é relatado como conformidade. Isso é mais forte do que o padrão comum de pedir a um LLM

para retornar verdadeiro, falso ou não encontrado, porque não é interpretado em relação a um domínio de pesquisa documentado [AssistOS-AI 2026].

A dificuldade restante é como validar observações produzidas por modelo. A validação do esquema é necessária, mas fraca. A ancoragem de evidências impede citações inventadas, mas não má interpretação. As pontuações de calibração ajudam a classificar os candidatos, mas não estabelecem significado. A abordagem prática mais forte combina sinais independentes: analisadores de linguagem controlada para fragmentos tratáveis; conjuntos de modelos ou prompts diversos; restrições de ontologia; consistência de sentença cruzada; mutações semânticas; exemplos negativos; revisão humana para distinções de alto impacto; e contra-exemplos que expõem consequências.

Fundamentação também deve ser conduzido por consulta. Um sistema não precisa resolver todas as entidades ou eventos em um documento de 300 páginas. Deve derivar a demanda semântica da regra que está sendo executada, recuperar os escopos relevantes e materializar apenas os tipos necessários. No entanto, o processamento de consulta em primeiro lugar não deve confundir relevância com integridade. Se uma regra perguntar se existe alguma exceção em qualquer parte do texto da autoridade, a recuperação deve fornecer um argumento de cobertura ou a resposta permanece desconhecida. A execução orientada pela demanda reduz o trabalho; não justifica o fechamento prematuro.

A conclusão fundamental é severa, mas construtiva. Não há remédio puramente simbólico para um símbolo ligado ao referente errado. O fundamentação não é um detalhe preliminar NLP; é parte da computação confiável. O sistema deve expor a cadeia da fonte ao símbolo, preservar alternativas, rastrear a cobertura e recusar a execução incompatível. Uma vez que essa cadeia é disciplinada, ferramentas formais maduras podem adicionar confiabilidade substancial. Sem ele, as ferramentas limitam a certeza das consequências de uma ficção plausível.

5.2 Pragmática, Não Monotonicidade, Escala, Segurança e Falhas de Avaliação

Mesmo uma representação bem fundamentada pode falhar porque a linguagem natural faz mais do que fatos monótonos de estado. Os palestrantes implicam, pressupõem, revisam, citam, brincam, negociam e confiam no contexto institucional. Os documentos contêm exceções, prioridades, definições que mudam mais tarde e declarações cuja força depende do gênero. A linguagem natural executável deve decidir qual desses fenômenos pretende apoiar e deve retornar desconhecida fora desse escopo.

A pragmática é o primeiro limite. Uma frase pode comunicar mais do que suas condições de verdade. “O relatório está quase completo” pode implicar que não está completo. “Alguns testes passados” podem sugerir pragmaticamente que nem tudo o fez, embora a lógica clássica não implica isso. A ironia inverte o sentimento aparente. Solicitações indiretas usam declarações como ações. Um formalizador que trata inferências pragmáticas como fatos pode exagerar; um que os ignora pode perder a intenção prática do autor. O tratamento adequado depende da aplicação. Um mecanismo de conformidade deve geralmente privilegiar a autoridade explícita e rotular a interpretação pragmática como proposto. Um agente de conversação pode precisar de modelos pragmáticos mais ricos, mas não deve representá-los como evidência concreta.

A presunção cria um desafio relacionado. “O comitê parou de rever a proposta” pressupõe que ela a revisou. As descrições definidas geralmente pressupõem existência ou singularidade. Os documentos podem explorar tais construções sem afirmar explicitamente o plano de fundo. Um sistema formal deve decidir se os pressupostos entram como suposições, inferências desviáveis ou requisitos para esclarecimento. Afirmá-los automaticamente pode não ser seguro; ignorá-los pode tornar o discurso incoerente. O sistema de status deve preservar sua origem.

As regras em linguagem natural são muitas vezes não-monotônicas. Novas informações podem retirar uma conclusão. “Os funcionários podem acessar o sistema” pode ser derrotado por “a menos que sua certificação tenha expirado”. Lógica padrão, programação de conjunto de respostas, lógicas desviáveis e regras priorizadas fornecem mecanismos formais para exceções. A lógica clássica de primeira ordem pode codificar exceções explicitamente, mas conjuntos de regras grandes tornam-se difíceis de gerenciar e informações inconsistentes podem banalizar a inferência se a explosão comum se aplicar.

Um tempo de execução prático deve suportar a semântica de múltiplas regras. O Datalog monotone é apropriado quando os fatos só se acumulam. As regras defeasabilidade são apropriadas para inadimplências e exceções. O raciocínio paraconsistente é apropriado quando as fontes entram em conflito e o sistema deve evitar derivar tudo. As tabelas de decisão são apropriadas quando as combinações de políticas são finitas e explícitas. A semântica escolhida deve ser versionada com o pacote de regras; não pode ser inferida ad hoc pelo LLM durante a execução.

Os sistemas normativos acrescentam autoridade e prioridade. Uma regra posterior pode substituir uma anterior. Um regulamento de nível superior pode antecipar um procedimento local. Jurisdição e matéria de data efetiva. As permissões e obrigações podem entrar em conflito. Os paradoxos dentários mostram que codificações lógicas ingênuas podem produzir resultados contra-intuitivos. Muitos casos práticos são melhor tratados por prioridades explícitas de regras, exceções e condições de violação do que por um cálculo universal

deôntico. A linguagem natural executável deve favorecer a clareza operacional sobre a grandeza teórica.

A escala cria várias explosões de interação. Documentos longos contêm muitos vãos e observações de candidatos. Menções ambíguas criam atribuições de identidade combinatórias. As relações temporais criam alternativas de ordem parcial. As regras criam consequências derivadas. Execução simbólica cria caminhos. Os solucionadores enfrentam grandes sistemas de restrição. As janelas de contexto e os custos do contexto LLM impõem limites separados. Uma arquitetura ingênua que formaliza cada frase em todas as lógicas possíveis é inviável.

A materialização orientada pela demanda é a primeira resposta. Os circuitos declaram o que consomem. Os índices de tempo de execução estruturam e informações lexicais estáveis, em seguida, produz semântica mais rica apenas para escopos relevantes. A computação incremental reutiliza artefatos inalterados. A interpretação abstrata resume alternativas. Caminhos de controle de exploração simbólica fundidos e delimitados. Os portfólios da Solver encaminham casos simples para motores baratos. Orçamentos rígidos produzem resultados incompletos explícitos em vez de truncamento oculto.

A segunda resposta é a modularidade semântica. Um documento pode ser analisado através de visualizações: seções, janelas de tempo, jurisdições, cenas narrativas, estágios experimentais ou camadas de autoridade. As consequências de identidade e regras podem ser locais para uma vista. As relações de visão cruzada são introduzidas apenas quando necessário. Isso reduz a fusão global acidental e apoia o paralelismo. O sistema ainda deve registrar quando uma conclusão depende da suposição de que uma visão está completa.

A terceira resposta é a precisão encenada. Uma passagem estrutural ou lexical barata elimina muitos casos. Um passe de extração assistido por modelo produz candidatos para os restantes. A análise abstrata identifica possíveis violações. O refinamento simbólico lida apenas com ambiguidades relevantes para resultados. A revisão humana é reservada para escolhas não resolvidas de alto impacto. Isso se assemelha a pipelines de análise estática que usam análises baratas antes da prova de teoremas caros.

Ameaças de segurança surgem em todas as fases. A injeção rápida em documentos de origem pode instruir o LLM a ignorar esquemas ou fabricar resultados. O tradutor deve receber o conteúdo de origem como dados sob um prompt específico de função, e sua saída deve ser validada de forma independente. A fonte nunca deve ganhar autoridade para modificar o tempo de execução, pacote de regras ou política de publicação. Qualquer texto que afirme ser uma instrução do sistema ainda é conteúdo de documentos.

O código gerado é um risco ainda maior. Um DSL que permite JavaScript, Python, importações ou acesso à rede transforma o aprendizado semântico em execução de código. Os circuitos declarativos devem normalizar para dados simples e chamar apenas operadores registrados. Algoritmos personalizados devem ser instalados como extensões revisadas com contratos digitados, efeitos, limites de recursos e resumos bloqueados. O sandbox é necessário, mas a restrição de capacidade é mais forte do que depender apenas de um limite de processo.

Os ataques do Solver incluem negação de serviço por meio de restrições patológicas, objetos de prova mal formados e exploração de bibliotecas nativas. Orçamentos, restrições teóricas, verificação de provas e fixação de versões são necessários. Um modelo pode deliberadamente ou acidentalmente construir uma fórmula que é tecnicamente válida, mas

computacionalmente explosiva. A seleção do formalismo deve estimar o custo e rejeitar tarefas fora da política.

O envenenamento por dados afeta produtores semânticos e suítes de referência. Se um agente de codificação gera tanto um circuito quanto os exemplos que o validam, os artefatos podem compartilhar o mesmo mal-entendido. A geração de testes independentes, mutações semânticas, casos de ancoragem com curadoria manual e papéis de revisão separados reduzem esse risco. A arquitetura de aprendizagem nllAgent impede corretamente um agente de codificação de publicar seu próprio candidato. A publicação manual é um controle de governança, embora sua eficácia dependa da qualidade dos benchmarks e verificações independentes.

A realização em linguagem natural pode reintroduzir reivindicações não suportadas. Um modelo pode converter “possível sob uma interpretação” em “o documento viola a regra”. Pode omitir suposições ou transformar uma busca incompleta em ausência. A realização deve, portanto, ser limitada por um esquema de resultados canônicos e verificada no nível de reivindicação. Saídas de alto impacto podem usar modelos controlados. Os comentários explicativos devem ser claramente separados dos achados verificados.

A avaliação é talvez a falha mais sistêmica. Muitos benchmarks medem apenas a precisão final da resposta. Isso confunde a extração, a formalização, o raciocínio e a renderização. Um modelo pode chegar à resposta correta pelo motivo errado. Um sistema formal pode ser penalizado por retornar desconhecido onde o benchmark assume uma convenção não declarada. Os conjuntos de dados sintéticos podem recompensar o reconhecimento de modelos. As métricas LLM-as-a-judge podem compartilhar vieses com os sistemas que estão sendo avaliados.

Um benchmark rigoroso deve conter várias camadas. A camada de origem inclui texto e estrutura exatos. A camada semântica inclui uma ou mais interpretações admissíveis com períodos de evidência. A camada formal inclui programas de destino ou testes de equivalência extensional. A camada de execução inclui modelos, provas, testemunhas ou núcleos insatisfeitos. A camada de resultados inclui status canônico e cobertura. A camada de realização inclui reivindicações permitidas e estalhistas proibidos. Os erros podem ser atribuídos em vez de entrarem em colapso em uma pontuação.

Os pares mínimos são essenciais. Mudar uma palavra ou relação deve produzir uma diferença formal previsível. Parafraseas famílias testam invariância. Recuperação do teste de seções de distratores. Ensaio de identificação teste de fundamentação. Evidências ausentes testam comportamento desconhecido. Fontes conflitantes testam a política de paraconsistência ou autoridade. Os casos de contorno temporal testam calendários e intervalos. O interceptação de texto testa o isolamento de segurança. Mutações semânticas testam se um circuito realmente implementa sua regra.

A avaliação também deve medir a abstenção. Um sistema confiável deve saber quando não tem uma ontologia, produtor, cobertura ou orçamento. Desconhecido não é um fracasso quando a questão não é apoiada. As métricas devem distinguir a abstenção justificada da incompletude evitável. Curvas de cobertura-qualidade podem mostrar como a precisão e a falsa mudança de garantia à medida que o sistema admite mais semântica proposta por modelo.

Questões de implementação independentes. Um compilador de circuitos e seu avaliador “sombra” não devem compartilhar todas as funções críticas se o teste diferencial for reivindicado. Os escores de mutação devem vir de mutações executáveis, não descrições de prosa. Os perfis de modelo ao vivo devem ser avaliados separadamente dos equipamentos determinísticos. O planejamento e a realização precisam de benchmarks específicos de tarefas, em vez de serem inferidos a partir do desempenho da auditoria. Estes são precisamente vários dos problemas sérios reconhecidos no repositório nllAgent [AssistOS-AI 2026].

As revisões externas devem ser interpretadas com cuidado semelhante. A pesquisa *Frontiers 2026* é revisada por pares e útil, mas sua alegação de que os sistemas federativos têm um melhor desempenho é explicitamente qualificada pela confusão de tarefas. O planejamento e a análise de programas mapeiam as tendências, mas não podem estabelecer que uma arquitetura proposta funcione em documentos irrestritos. Nenhuma revisão independente do nllAgent foi encontrada. Um livro responsável deve, portanto, usar revisões para identificar consenso e discordância, em seguida, declarar seus próprios julgamentos onde as evidências estão ausentes.

O princípio central de avaliação é a atribuição causal. Quando um sistema melhora, os pesquisadores devem determinar se o ganho veio de melhor análise semântica, correção de resolução, recuperação, amostragem no tempo de teste ou vazamento de referência. Quando ele falha, eles devem identificar a camada. Não se trata apenas de higiene científica. Dados de erro em camadas é o que permite que a arquitetura melhore sem substituir todo o sistema ou ocultar regressões em pontuações agregadas.

A linguagem natural executável continuará a ser uma capacidade limitada. A pragmática, o conhecimento do mundo aberto, o contexto social e as instituições em evolução garantem que nenhum núcleo simbólico finito capture todo o significado. O objetivo não é remover esse limite, mas torná-lo explícito e útil. Um sistema pode oferecer fortes garantias para fragmentos estruturais, relacionais, temporais, quantitativos e governados por regras, ao mesmo tempo em que retornam resultados condicionais em outros lugares. O sistema mais perigoso não é aquele que admite limites. É um que converte a linguagem não modelada em uma saída simbólica confiante e chama o resultado verificado.

6. Famílias de Arquiteturas para Linguagem Natural Executável

6.1 Projetos Concorrentes: Representações Universais, Portfólios de Formalismos, Compilação Controlada e Sistemas Orientados pela Consulta

Arquiteturas de linguagem executável podem ser organizadas de acordo com o qual eles colocam o compromisso semântico. Alguns tentam traduzir cada entrada em uma representação universal. Outros retêm vários formulários de candidatos e selecionam um formalismo por tarefa. Alguns restringem a linguagem de entrada até que a compilação seja determinística. Outros preservam a linguagem aberta, mas formalizam apenas as observações exigidas por uma consulta. Cada projeto resolve um problema diferente, e um sistema maduro provavelmente os combinará em vez de selecionar um exclusivamente.

A representação intermediária universal é a arquitetura mais intuitiva. O texto é compilado em um grafo semântico rico ou linguagem lógica, e cada operação downstream consulta ou transforma esse objeto comum. As vantagens são a unidade conceitual, a reutilização e a possibilidade de verificações de consistência global. Uma única representação pode suportar vários aplicativos e criar uma interface estável entre o NLP e o raciocínio.

Sua fraqueza é o excesso semântico. Para servir a todas as tarefas, a representação deve abranger entidades, eventos, discurso, tempo, modalidade, quantidade, causalidade, normas, incerteza, proveniência e conceitos específicos de domínio. Ou a linguagem se torna extremamente complexa ou simplifica as distinções que alguma tarefa precisa. Uma representação universal também cria um único ponto de falha: uma análise global errada contamina todas as análises. A pressão para escolher uma identidade, escopo ou ontologia cedo pode esconder ambiguidade.

Um pivô semântico é uma variante mais modesta. Não pretende ser uma forma lógica completa. Ele padroniza extensões de origem, menções, hipóteses de entidade, observações, mundos, valores de tempo, modalidade, proveniência, alternativas, cobertura e lacunas. Os compiladores especializados reduzem as porções selecionadas em alvos formais. Isso preserva a infraestrutura comum sem fingir que uma semântica de execução se encaixa em todo o significado. LongTextJS, como documentado em nllAgent, está perto desta ideia: é um programa de origem endereçável de origem mundial e observação, em vez de uma língua de melhoria universal [AssistOS-AI 2026].

A arquitetura de múltiplos candidatos trata a formalização como busca. Um LLM produz várias formas lógicas, programas ou grafos semânticos. As restrições de cheques e ontologia eliminam candidatos inválidos. Execução, exemplos e contra-exemplos classificam ou distinguem os sobreviventes. O sistema pode retornar uma conclusão obrigatória quando todos os candidatos concordam e pode ser concluído quando apenas alguns o fazem. Esta arquitetura é apropriada quando a ambiguidade é genuína ou quando a confiabilidade da formalização é limitada.

Seu custo é combinatório. Os formulários candidatos multiplicam-se entre frases e escolhas de identidade. Um produto ingênuo cross-product torna-se inviável. A solução é representar a

estrutura compartilhada e as alternativas locais de forma compacta, usar domínios abstratos para resumi-los e refinar apenas as escolhas relevantes para resultados. A confiança pode orientar a exploração, mas os candidatos de baixa probabilidade não devem ser removidos de uma análise sólida, a menos que a política aceite explicitamente a incompletude probabilística.

A compilação de linguagem controlada tem a abordagem oposta. A entrada é reescrita em uma linguagem natural restrita cuja gramática e semântica são determinísticas ou altamente restritas. Os usuários podem escrever diretamente no idioma controlado, ou um LLM pode propor uma paráfrase controlada para confirmação. Uma vez aceita, a compilação é confiável. Esta arquitetura é particularmente forte para requisitos, contratos, procedimentos e criação de regras.

Sua limitação é a cobertura e adoção. Os documentos comuns contêm construções fora do fragmento controlado. Reescrever pode mudar o significado. Os usuários podem aprovar uma paráfrase sem perceber uma diferença sutil. A arquitetura funciona melhor quando o formulário controlado é exibido ao lado da fonte, as diferenças são localizadas e os contra-exemplos formais tornam concretas as escolhas importantes. A linguagem controlada deve ser um ponto de verificação semântico explícito, não uma normalização invisível.

Arquiteturas de consulta e primeiro evitam a formalização total. Um circuito ou pergunta declara quais observações e relações ele precisa. O tempo de execução preserva a fonte estruturalmente, recupera regiões relevantes e materializa apenas tipos semânticos exigidos. Consultas relacionais, tabelas e regras de decisão operam sobre o mundo finito resultante. Este projeto é melhor para documentos longos e alinha computação com propósito.

Uma visão de sistemas de banco de dados 2026 intimamente relacionada distingue operadores semânticos interpretados dos compilados. No design interpretado, um LLM é invocado dentro do loop de processamento de dados para cada par de linhas, registros ou candidatos. No projeto compilado, o LLM é invocado durante uma fase de construção separada para traduzir a operação semântica em código local determinístico que pode ser executado repetidamente sem uma chamada de modelo. Experimentos preliminares relataram reduções substanciais na latência e no uso do modelo, mantendo grande parte da qualidade de saída [Dong and Wang 2026]. A proposta é importante porque atinge independentemente a mesma intuição arquitetônica desenvolvida neste livro: o modelo de linguagem é mais útil como um compilador ou construtor de plano, enquanto o tempo de execução repetido deve ser estável e inspecionável. Sua limitação é igualmente instrutiva. O código gerado pode aproximar-se eficientemente do operador definido pelo modelo sem fornecer uma prova semântica de que ele captura o conceito pretendido do usuário. A compilação remove a estocástica por item; ela não resolve por si só a base ou a fidelidade de especificação.

O risco é falso fechamento. A recuperação pode perder evidências relevantes e a análise da demanda pode omitir um conceito necessário. Os sistemas de consulta devem acompanhar a cobertura e os produtores. Se uma conclusão negativa exigir a busca de todo o documento para qualquer exceção, algumas passagens recuperadas são insuficientes. A arquitetura deve distinguir um resultado de consulta delimitado de uma garantia de mundo fechado. Os contratos de compatibilidade e cobertura do nllAgent são um mecanismo importante nesta família.

Uma arquitetura monoformalismo reduz todas as tarefas em uma lógica, muitas vezes lógica de primeira ordem, Datalog ou SMT. Simplifica as obrigações de implementação e prova. Para um domínio restrito, esta pode ser a melhor escolha. A lógica-LM e a LINC ganham muito do

seu valor a partir de um idioma de destino estável. Um sistema para agendamento pode razoavelmente usar uma linguagem de restrição. Um verificador de protocolo pode usar um cálculo de processo.

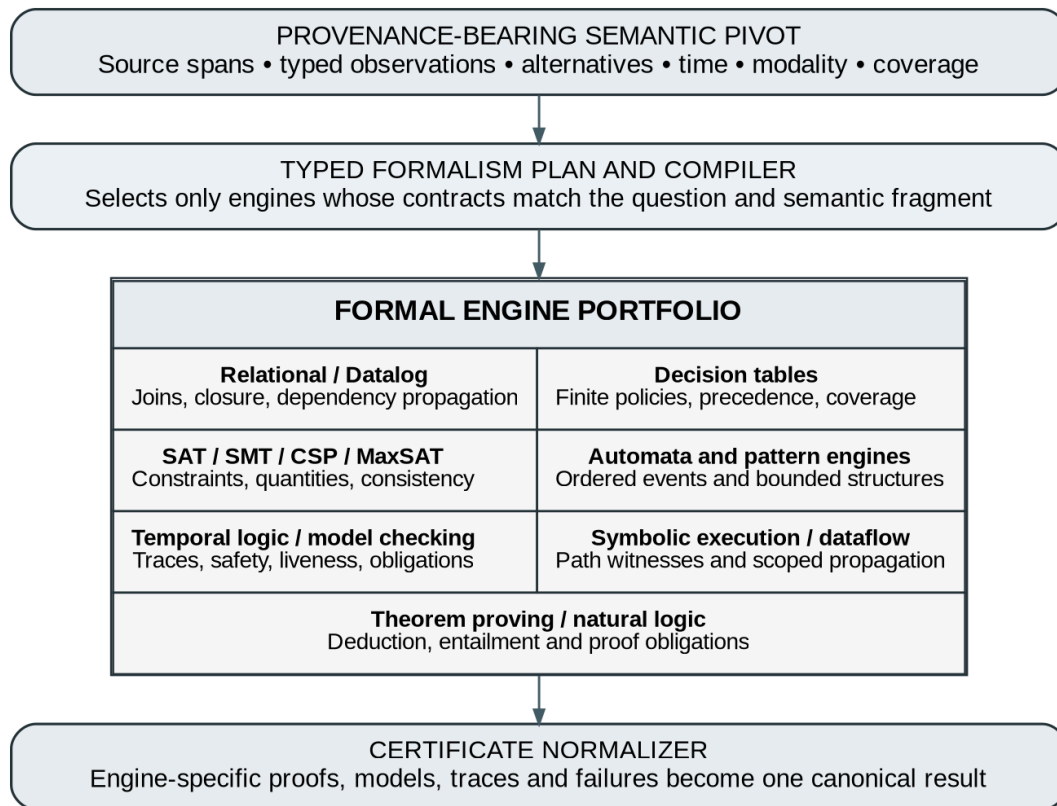


Figura 3. Um pivô semântico composto de proveniência pode ser compilado em um portfólio de motores formais especializados e normalizado em um resultado canônico.

Para tarefas amplas em linguagem natural, um portfólio de solver é mais plausível. A avaliação relacional lida com consultas de documentos. Datalog calcula o fechamento monótono. As tabelas de decisão expressam políticas finitas. SAT e SMT lidam com restrições e aritmética. A verificação do modelo lida com o comportamento temporal. A execução simbólica constrói testemunhas de caminho. A lógica natural lida com certas relações de comenda perto da linguagem superficial. A Automata reconhece padrões ordenados. O pivô semântico fornece átomos aterrados, e um plano de formalismo seleciona um ou mais motores.

O portfólio cria novas obrigações. O roteador deve conhecer as capacidades expressivas e operacionais de cada motor. Um compilador deve preservar o significado entre as reduções. Os resultados de diferentes motores precisam de um status comum e um modelo de proveniência. Se dois motores raciocinarem sobre os predicados sobrepostos, a consistência deve ser verificada ou explicitamente deixada em aberto. As políticas de recursos devem impedir que um modelo selecione um solucionador desnecessariamente caro.

Um contrato de formalismo útil afirma os tipos de entrada, operadores apoiados, semântica, cobertura necessária, classe de garantia, certificado esperado, modelo de custo e limitações conhecidas. Um motor lógico temporal pode exigir eventos aterrados e um sistema de transição finito. Um motor SMT pode suportar funções reais aritméticas inteiras, mas não irrestritas, não lineares. Uma tabela de decisão pode exigir condições enumeradas finitas. Um motor de lógica

natural pode suportar monotonicidade, mas não a coreferência arbitrária entre a frase. O roteador pode então rejeitar um plano proposto cujos requisitos excedem o motor.

A geração de transporte de provas é outra família de arquitetura. O artefato primário não é a resposta em linguagem natural, mas um resultado canônico mais uma prova, testemunha, traço de execução ou derivação verificável. O modelo de linguagem gera uma explicação a partir desse artefato. Cada alegação explicativa está ligada a um elemento de resultado. Um verificador independente garante que a explicação não exceda o conteúdo formal.

Este projeto é valioso porque a geração e a verificação têm forças diferentes. Um LLM pode escrever uma explicação clara de um contra-exemplo ou restrição insatisfatória. Um resultado simbólico pode garantir que o cenário seja real sob o modelo aceito. A explicação ainda pode ser enganosa se esconde suposições ou incertezas, de modo que o resultado canônico deve incluir essas dimensões e o contrato do renderizador deve exigir-lhes quando material.

Arquiteturas abstratas-semânticas colocam uma camada de interpretação abstrata antes da execução especializada. Observações candidatas preenchem pode e deve ser domínio. A propagação barata determina quais propriedades são definitivamente satisfeitas, definitivamente impossíveis ou potencialmente violadas. Apenas violações potenciais são reduzidas em problemas simbólicos caros. Isso reduz a carga do solucionador e fornece estados desconhecidos de princípios. Também cria uma arquitetura incremental natural para edições de documentos.

As arquiteturas guiadas contra-exemplos fecham o ciclo. Uma prova fracassada ou testemunha espúria não é apenas um erro; identifica qual distinção semântica importa. O sistema mapeia fatos do solver para obter extensões, pede ao LLM ou ao usuário para refinar a interpretação relevante e reproduz a análise. Essa arquitetura é especialmente promissora porque transforma feedback formal em compreensão de linguagem direcionada, em vez de auto-reflexão genérica.

Arquiteturas de aprendizagem regem como novas regras, ontologias e circuitos entram em produção. Um extremo permite que um agente LLM gere um código executável durante cada execução. Isso maximiza a flexibilidade e minimiza a garantia. O extremo oposto permite apenas regras manuscritas e formalmente verificadas. Um meio prático usa agentes de codificação na realização de espaços de trabalho, gera benchmarks e mutações, restringe artefatos a formas declarativas e requer publicação explícita. O tempo de execução da produção lê lançamentos imutáveis. Este é o modelo de governança implementado no nllAgent e deve ser generalizado.

A seguinte comparação resume o trade-off central sem afirmar que um design domina cada tarefa.

Tabela 1. Principais famílias de arquitetura e seus trade-offs essenciais.

Arquitetura	Força essencial e risco principal
Representação semântica universal	Reutilizar e consistência global; riscos de sobrecomplexidade e compromisso semântico prematuro.
Pivotático e rebaixamentos especializados	Preservar evidências comuns e suporta vários mecanismos; requer um compilador cuidadoso e contratos entre motores.
Legislações de candidatos múltiplos	Representa a ambiguidade e permite que possam/devem conclusões; corre o risco de crescimento combinatório.

Arquitetura	Força essencial e risco principal
Compilação de linguagem controlada	Semântica forte e formas invisíveis de usuário; sacrifica a cobertura e pode introduzir mudanças de significado durante a reescrita.
Materialização de consulta em primeiro lugar	Escalas para documentos longos e necessidades específicas de tarefas; pode confundir recuperação com integridade, a menos que a cobertura seja explícita.
Objectivo formal único	Base de confiança simples e avaliação em um domínio delimitado; mau ajuste para fenômenos de linguagem heterogêneos.
Portfólio de Solver	Corresponde ao método computacional à estrutura do problema; adiciona roteamento, interoperabilidade e obrigações de normalização de certificados.
Geração de transporte de provas	Constrita respostas em linguagem natural ao conteúdo verificado; requer uma verificação robusta de realização no nível de reivindicação.
Análise abstrata mais refinamento simbólico	Balança e precisão; solidez permanece condicionada à cobertura de interpretação.
Circuitos eruditos geridos	Permite a evolução sem automodificação em tempo de execução; depende de fortes benchmarks, testes de mutação e revisão de publicação.

A arquitetura geral mais credível é um híbrido das opções médias. Ele usa um pivô semântico portador de procedência, preserva alternativas onde eles importam, realiza materialização orientada por consultas, rotas para um portfólio de formalismo e aceita apenas resultados de prova ou de testemunhas. A linguagem controlada pode servir como um ponto de verificação interativo para formalizações de alto impacto. A interpretação de controles abstratos controla a escala, enquanto o refinamento guiado por contraexemplo melhora a precisão. A aprendizagem continua fora da produção.

Esta conclusão não é apenas teórica. Isso se reflete na literatura empírica. PAL, SatLM, Logic-LM, LINC, tradutores de lógica temporal, formalizadores de planejamento, SymGPT e sistemas de verificação certificada todos têm sucesso, estreitando o alvo formal. Nenhum resultado demonstra que uma lógica universal ou um modelo de ponta a ponta pode substituir o portfólio. A arquitetura deve, portanto, abraçar a heterogeneidade, minimizando as interfaces confiáveis entre os componentes.

6.2 Um Pipeline de Referência do Texto aos Símbolos, à Execução e a Respostas em Linguagem Natural Restrita

Um pipeline de linguagem executável de referência deve ser entendido como um compilador e uma cadeia de ferramentas de verificação em vez de um prompt de agente. Cada estágio materializa um artefato imutável ou endereçado de conteúdo. Uma corrida pode ser repetida sem pedir ao modelo para reconstruir seu raciocínio anterior. Os erros são atribuídos às etapas. Aprender pode melhorar um produtor ou teoria sem alterar silenciosamente o significado dos resultados antigos.

O primeiro artefato é o pacote de origem. Ele contém o texto ou dados canônicos, resumo de revisão, linguagem, estrutura, canais e vãos estáveis. Parágrafos, cabeçalhos, tabelas, citações e blocos de código mantêm ordem e identidade. Os registos externos têm chaves igualmente estáveis. O pacote fonte é a autoridade lexical: ele estabelece o que estava presente, não o que significa.

O segundo artefato é o contrato de tarefa. Ele afirma a questão, liberação de regras, escopo de fonte, garantia desejada, suposições permitidas, mundo e jurisdição, orçamentos e política de produção. A tarefa determina a demanda semântica. Uma solicitação para encontrar acrônimos indefinidos requer diferentes observações de uma solicitação para verificar a conformidade temporal. O contrato também define se a ausência pode ser concluída e quais formas de incerteza exigem revisão humana.

O terceiro estágio constrói um gráfico de evidências. Os produtores deterministas acrescentam observações estruturais. Analisadores de linguagem controlada adicionam observações para os fragmentos que reconhecem. Os produtores apoiados pelo LLM adicionam eventos, relações, identidades, modalidades ou quantidades de candidatos ligados ao esquema. Cada registro de observação ancora as âncoras, o produtor, o modelo ou o algoritmo, o status, as alternativas e a cobertura. O modelo não escreve diretamente no gráfico canônico; um materializador verifica novamente vãos, tipos e integridade referencial.

A identidade e a gestão do mundo ocorrem na mesma camada, mas não são resolvidas silenciosamente. Menções são evidências locais. As hipóteses de entidade coletam menções sob um mundo e escopo. Os candidatos a identidade estabelecem relações iguais, diferentes ou não resolvidas com apoio. As interpretações mutuamente exclusivas pertencem a mundos separados ou a alternativas protegidas. O gráfico pode, portanto, representar discordância sem corromper a consistência global.

O quarto estágio calcula um estado semântico abstrato. Abstrações específicas do domínio resumem possíveis identidades, ordens temporais, status normativo, quantidades e níveis de evidência. Deve manter fatos em todos os mundos admitidos; os fatos podem ter pelo menos um. Desconhecido registra uma lacuna em vez de um palpite probabilístico. O estado abstrato suporta a propagação de regras baratas e identifica quais questões exigem maior precisão.

O quinto estágio produz um plano de formalismo. O planejador não é uma decisão LLM de forma livre. Ele compara a tarefa com contratos de formalismo registrado. Uma simples consulta de documento pode permanecer relacional. Uma política pode compilar em tabelas de decisão e restrições. Um procedimento pode ser compilado em um sistema de transição. Uma violação dependente do caminho pode invocar a execução simbólica. Uma reivindicação dedutiva pode invocar um prover teorema. Os nomes dos planos exigiam entradas, rebaixamentos, versões do motor, limites de recursos e certificados esperados.

O LLM pode propor um plano porque reconhece a estrutura do problema e a linguagem de domínio. Um verificador determinístico valida que o plano usa formalismos apoiados, que toda observação necessária tem um produtor compatível e que nenhuma distinção semântica essencial para a tarefa é perdida. Por exemplo, uma regra envolvendo exceções e prioridade não pode ser compilada em um motor monótono sem uma codificação explícita. Uma regra envolvendo identidade não pode prosseguir se o domínio de identidade não for resolvido e o veredicto depender disso.

O sexto estágio executa a compilação digitada. Este compilador deve ser pequeno, determinístico e fortemente testado. Ele converte objetos semânticos em variáveis de solver, relações, transições ou predicados. Mapas de origem conectam cada elemento formal de volta a observações e vãos. Os compiladores devem gerar obrigações de prova quando uma redução é deficitária ou depende de suposições. Diferentes compiladores podem ser comparados através de mutações semânticas e modelos distintivos.

A sétima etapa é a execução formal. Fechamentos de computação de mecanismos relacionais e Datalog. As tabelas de decisão identificam linhas combinadas e conflitantes. Os solucionadores SAT/SMT/CSP encontram modelos ou provam inviabilidade sob suas teorias apoiadas. Os verificadores de modelos avaliam propriedades temporais. Os executores simbólicos exploram as condições do caminho e retornam testemunhas concretas. Assistentes de prova ou teóricos produzem derivações. A saída do motor ainda não é um veredicto voltado para o usuário; é um registro de execução de certificado.

A oitava etapa é a verificação de repetição. Um verificador separado verifica a testemunha ou a prova contra o problema formal ligado à fonte e a teoria publicada. Para uma descoberta de documento, ele confirma que as observações citadas existem, combinam com a revisão de origem, os status satisfazem a regra e o rastreamento de execução suporta a reivindicação. Para um resultado negativo, confirma que as premissas de cobertura e completude são suficientes. O replay é valioso mesmo quando o solucionador subjacente é confiável porque valida a conexão entre a saída do candidato e a política liberada.

Se a análise abstrata previu uma violação, mas a execução simbólica não pode perceber, o refinamento semântico começa. O sistema extrai o núcleo insatisfatório, os predicados conflitantes ou a falha da prova. Mapas de origem identificam os vãos relevantes e as escolhas semânticas. O LLM recebe uma tarefa limitada: reconsiderar este link de identidade, escopo temporal, mapeamento de exceções ou alinhamento de predicados. Ele propõe candidatos revisados; o materializador e o compilador revalidam deles; a execução se repete. A revisão humana é solicitada quando a distinção é de alto impacto ou permanece sem solução.

Este laço deve ser monótono em relação à evidência, não apenas confiança. Um refinamento pode dividir uma entidade, reduzir um intervalo de tempo, adicionar uma exceção negligenciada ou marcar uma premissa sem suporte. Ele não deve atualizar uma observação proposta para o fato, porque o novo resultado é conveniente. Cada mudança é registrada como uma nova revisão semântica. Os caminhos anteriores permanecem reproduzíveis.

O nono artefato é um resultado canônico. Ele contém a regra ou identidade de consulta, status aceito, suposições, deveres e podem conclusões, candidatos rejeitados, incógnitas, cobertura, evidência de origem, referências de provas ou testemunhas e limites de execução. Também pode incluir um gráfico de explicação conciso verificável por máquina. Este artefato é autoritário sobre qualquer renderização de prosa.

O décimo estágio realiza a linguagem natural. O renderizador recebe apenas o resultado canônico e os trechos de origem selecionados. Ele não retoma o raciocínio original. Cada frase gerada está associada a uma ou mais reivindicações de resultados. Um verificador compara a frase com as reivindicações permitidas, garante que a modalidade e a incerteza sejam preservadas e rejeita adições sem suporte. A saída pode distinguir achados, pressupostos, interpretações e recomendações.

exemplo, a resposta correta pode ser: “Sob a interpretação de que ambas as referências ao Conselho denotam o mesmo órgão, a regra é violada porque a aprovação precede a revisão de segurança registrada. O documento não estabelece se as referências denotam o mesmo corpo; portanto, o resultado geral é necessário. Um renderizador fluente, mas inseguro, pode omitir a condição e anunciar uma violação. O resultado canônico impede essa simplificação.

O pipeline também suporta geração, não apenas linting. Um usuário fornece uma ideia ou objetivo. Os circuitos de planejamento constroem uma especificação controlada com testemunhas regra-a-plano. Um LLM realiza um rascunho. Os circuitos de auditoria analisam o rascunho usando a mesma evidência e caminho de verificação. A revisão continua até que a auditoria canônica satisfaça a política de lançamento ou o orçamento seja esgotado. O modelo de geração nunca se torna o oráculo final para saber se seu próprio texto satisfaz as regras.

A governança envolve a cadeia de ferramentas. Ontologias, produtores, circuitos, compiladores, contratos de formalismo e renderizadores têm versões e resumos. Os agentes de aprendizagem operam em áreas de preparação descartáveis. Publicação requer benchmarks, mutações semânticas, verificações de compatibilidade e aprovação explícita. A produção lê os lançamentos imutáveis. Os perfis de modelo ao vivo são avaliados separadamente porque os dispositivos determinísticos não estabelecem a qualidade do provedor. Os pacotes de conhecimento do mundo atual têm datas e jurisdições efetivas.

A arquitetura deve expor uma hierarquia de garantias. As garantias estruturais dizem respeito a vão exatos, ordenações e validade de esquema. O assobio garante preocupação com o alinhamento de evidências e o status de identidade. A execução garante preocupações com consequências de um problema formal. As garantias de ponta a ponta combinam-nas condicionalmente. O usuário deve ver a camada mais fraca. Um resultado derivado de eventos propostos por modelo e uma transição mecanicamente verificada não deve ser rotulado simplesmente como “verificado”; deve afirmar que a execução é verificada sobre as observações propostas.

Um pequeno kernel confiável é um objetivo de design importante. O kernel inclui canonização de origem, esquema e verificação de tipo, compiladores formais, executores registrados ou verificadores de prova, verificadores de replay e validação canônica-resultado. As instruções LLM, geradores candidatos, heurísticas de recuperação e modelos de explicação permanecem do lado de fora. Os componentes confiáveis ainda podem conter bugs, mas seu comportamento é estável, testável e revisável. Esta é uma melhoria substancial em relação à confiança em um modelo de fundação em evolução como um raciocinador monolítico.

O gasoduto proposto não é a única implementação possível, mas capta as obrigações reveladas pela literatura. Isso explica por que um solucionador é necessário, mas insuficiente. Dá interpretação abstrata e execução simbólica papéis distintos. Ele suporta várias linguagens formais sem abandonar uma camada de evidência comum. Permite a criatividade LLM na descoberta, reparação e explicação, preservando a autoridade simbólica sobre fundamentação e os resultados aceitos. Mais importante ainda, transforma o fracasso em um artefato nomeado

– um tipo sem suporte, uma identidade não resolvida, cobertura incompleta, uma perda de compilação, uma formalização insatisfatória ou uma frase não licenciada – em vez de uma vaga alucinação.

7. LongTextJS, CircuitJS e nllAgent: Um Estudo de Caso Crítico

O repositório público NaturalLanguageLinterAgent é uma das poucas tentativas de tornar explícito o limite entre interpretação de linguagem e execução de regras em uma implementação executável. O projeto não é um crítico geral da linguagem natural, e sua documentação não o apresenta como um só. É uma biblioteca ESM do Node.js 22+ e tempo de execução da linha de comando com dois modos de produto de linguagem controlada sobre a mesma arquitetura LongTextJS-CircuitJS. O modo de auditoria aplica uma teoria versionada a um documento de Markdown existente e emite um produto de auditoria canônico. O modo de especificação aplica teorias de planejamento a uma ideia de alto nível e emite um plano de geração verificado; um modelo de linguagem opcional pode perceber esse plano como prosa, mas os circuitos de auditoria inalterados permanecem na aceitação do oráculo [AssistOS-AI 2026].

O repositório é útil como um estudo de caso porque incorpora a tese central deste livro em forma de software: modelos podem propor observações semânticas ou prosa, mas a autoridade de produção é atribuída a versões imutáveis, verificações de compatibilidade explícita, execução de circuito restrito, verificadores e artefatos canônicos. O projeto também mantém um registro público de questões sérias. Essa prática é importante porque uma grande superfície de documentação pode fazer com que as intenções arquitetônicas pareçam garantias implementadas. A discussão abaixo distingue o que está operacionalmente estabelecido do que permanece uma hipótese de pesquisa.

Nenhuma avaliação independente revisada por pares ou uma revisão técnica de terceiros de nllAgent, LongTextJS ou CircuitJS foi localizada através de pesquisas dos nomes dos projetos, nome do repositório, URL de documentação e combinações com “review”, “avaliação”, “benchmark” e “replicação” em 2 de agosto de 2026. Os resultados da pesquisa levaram ao projeto em si, em vez de uma avaliação externa. O julgamento neste capítulo é, portanto, a revisão arquitetônica independente do autor, com base no repositório público, documentação, exemplos executáveis, processo de liberação declarado e arquivo de questões sérias atuais. A ausência de uma revisão externa não é nem elogio nem crítica; isso significa que as reivindicações do projeto devem permanecer estreitas até serem reproduzidas de forma independente.

7.1 O que a Arquitetura Pública Implementa e Por que seus Limites Importam

O sistema define três atividades de produção que são deliberadamente separadas. A auditoria de produção lê um comunicado publicado imutável, compila um documento no LongTextJS, executa os circuitos de validação do lançamento, verifica as descobertas candidatas e persiste um resultado canônico CNL/Audit-1 com um relatório renderizado. A especificação de produção compila uma ideia em observações LongTextJS, aplica circuitos de planejamento e produz um artefato canônico CNL / Plan-1; a realização opcional pode gerar Markdown e enviá-lo para o mesmo mecanismo de auditoria. A aprendizagem ocorre em um espaço de trabalho de preparação descartável. Um agente de codificação pode gerar artefatos e benchmarks de criação de candidatos, mas os controles do lado do host admitem apenas arquivos na lista de permissões, e um mantenedor deve publicar e ativar explicitamente uma versão. Isso torna o aprendizado análogo ao desenvolvimento de software governado, em vez de automodificação em tempo de execução.

A unidade de trabalho mais importante não é o relatório final, mas o diretório de execução. Uma auditoria preserva a entrada copiada, o programa de origem, a base selecionada, os resultados de compatibilidade e a cobertura, o rastreamento do circuito, os resultados do verificador, as descobertas, o relatório canônico legível por máquina e a visão legível por humanos. As corridas de planejamento preservam os artefatos do plano correspondente. Uma execução que pára porque o conhecimento está faltando ainda registra a falha e pode criar um problema reutilizável. A regra de design de que falta conhecimento nunca é relatado como conformidade é fundamental. Isso evita uma falha comum dos agentes do documento: traduzir “Eu não encontrei evidências” em “o documento satisfaz a regra”.

LongTextJS deve ser entendido como um programa do lado da fonte apenas em um sentido restrito. Ele não atribui comportamento imperativo a sentenças arbitrárias. Ele compila uma revisão de origem fixa em um mundo finito, queryable de estruturas e observações. As propriedades valiosas são identidade de origem, vãos estáveis, tipos de observação versionada, identidade do produtor, cobertura, lacunas explícitas e replayability. Um artefato do lado do documento pode ser executado porque os consumidores registrados podem consultar e transformar essas relações sob semântica declarada; está mais próximo de uma representação intermediária semântica ou programa de carregamento de banco de dados do que do código de aplicação irrestrito.

Essa distinção é mais do que terminologia. Quando um modelo traduz uma frase, sua saída não deve se tornar autoritária simplesmente porque é serializada em uma linguagem de programação. O texto fonte permanece evidência; a observação traduzida registra uma interpretação dessa evidência, gerada por um produtor nomeado sob um esquema e perfil modelo. O circuito continua a ser uma teoria separada. Uma descoberta só se torna publicável depois que um verificador verifica as condições esperadas pelo lançamento. Essas camadas impedem que um analisador se torne silenciosamente um juiz e impedem que uma explicação se torne o único registro de uma inferência.

O pacote de fundação atual ilustra o escopo semântico pretendido. Os comandos de auditoria, plano e benchmark ordinários selecionam um base-core limitado 1.1.0 a menos que explicitamente desativado. O pacote reconhece formas controladas e inglesas documentadas para estado, tipo, precedência temporal exata expressa por “antes”, afirmações exatas de

aritmética, quantidade e emoção literal. Cinco circuitos verificados por replay verificam apenas os invariantes lógicos, temporais, aritméticos, físicos elementares e emocionais/tipo. O pacote e o resumo selecionados são persistidos. As mudanças de fatos políticos, geográficos, sociais, econômicos e legais são intencionalmente excluídas da verdade de fundação atemporal. Este é um limite de modelo mundial defensável. Um sistema que codifica fatos atuais sem datas, jurisdições, fontes e políticas de conflito efetivas não pode oferecer execução semântica reprodutível.

O sistema suporta materialização determinística e assistida por modelo. A seleção de tradução pode preferir um agente de modelo de idioma AchillesAgentLib configurado, usar o atual adaptador de agente de codificação restrito ou executado sem um modelo para agentes determinísticos. Cada chamada de modelo recebe uma habilidade específica do papel, em vez de todo o espaço de trabalho do agente. Se uma observação crítica não puder ser materializada dentro do esquema e orçamento de recursos necessários, o resultado é incompatível, incompleto ou exausto. O modelo não controla a construção de fontes, a política de publicação ou o significado de um verificador. Esta é precisamente a direção correta para um sistema neuro-simbólico federativo: o modelo é poderoso na borda, enquanto artefatos simbólicos e aceitação de controle de políticas de host.

A cobertura é tratada como uma obrigação executável, em vez de uma pontuação de confiança informal. A compilação do circuito funciona de trás para frente a partir de nós de saída para determinar quais observações externas podem influenciar um resultado. Publicação registra contratos que ligam essas demandas aos produtores disponíveis. A compatibilidade com tempo de execução então pergunta se o documento concreto realmente produziu observações com o tipo, status, cardinalidade, estrutura, canal e cobertura necessários. A capacidade declarada de um produtor não é confundida com evidências de que o documento atual foi processado com sucesso. Quando uma demanda crítica não é atendida, a corrida pára em um limite nomeado.

Este design orientado pela demanda é adequado para documentos longos. Evita pedir a um modelo que produza uma interpretação para todos os fins de cada frase antes que qualquer pergunta seja conhecida. Um circuito pode solicitar as dimensões semânticas de que necessita: termos exatos, afirmações do estado, quantidades, relações temporais ou observações de domínio. O compilador pode materializar apenas observações relevantes para a saída e persistir o que foi e não foi coberto. A vantagem científica não é apenas um custo mais baixo. Limita compromissos semânticos desnecessários e torna visível a formalização incompleta.

CircuitJS é a representação do lado da teoria. Os módulos de circuito editáveis por agentes são restritos a uma expressão declarativa, normalizados para dados simples e passados pelo mesmo compilador usado para descrições de circuitos não-código. Eles podem invocar apenas operadores e verificadores registrados; eles não podem esconder o comportamento executável arbitrário dentro de uma regra aprendida. Um aplicativo host pode instalar uma extensão de tempo de execução revisada contendo algoritmos reais, mas as entradas e o contexto são cópias de dados simples congelados, os resultados devem ser dados simples, a identidade de cache inclui o resumo da extensão e as versões que dependem do bloqueio de extensão que digiram. A interface de linha de comando padrão não aceita caminhos de extensão executáveis arbitrários.

Este limite aborda um dos riscos práticos mais sérios nos sistemas de regras “aprendidos com agentes”. Se um agente de codificação pode gerar JavaScript arbitrário em cada circuito,

aprender um linter torna-se geração remota de código e o comportamento semântico de uma liberação torna-se difícil de inspecionar. Restringir a criação de circuitos comuns para operações registradas cria um vocabulário operacional finito. Algoritmos especializados permanecem possíveis, mas eles entram através de um caminho separado de engenharia, revisão e controle de versão. A arquitetura, portanto, distingue a evolução das teorias da evolução do tempo de execução confiável.

O limite do verificador é igualmente importante. A execução do circuito pode criar conclusões de candidatos, mas um verificador controla se esses candidatos entram na auditoria ou no plano canônico. Nos exemplos limitados, o verificador pode reler âncoras de origem, observações, metadados de lançamento e a política que está sendo aplicada. Não é suficiente que um nó de execução emite um objeto cujos campos pareçam um aviso. O verificador estabelece a propriedade específica esperada pelo produto. Isso não prova automaticamente a interpretação semântica correta, mas impede que a saída de circuito acidental ou maliciosa se mascarie como uma descoberta aceita.

O modo de especificação demonstra como a mesma arquitetura pode suportar geração controlada sem conceder autoridade ao modelo de escritor. Os circuitos de planejamento constroem um plano canônico e exigem testemunhas de regras aplicadas para regras aplicadas. A realização opcional transforma o plano em prosa. A prosa candidata passa pelo modo de auditoria e o modelo de realização não julga sua própria conformidade. Isso é mais forte do que a geração restrita somente rápida porque o objeto autoritário é o rastreamento de plano e auditoria, não a adesão autorreferida do modelo.

O subconjunto experimental de consulta-primeiro também é significativo. O LongTextJS canônico é exposto como um mundo de consulta finito somente leitura. Consultas digitadas e valores de tabela de reconhecimento de evidências são reduzidos para a execução CircuitJS ordinária. As verificações do compilador suportam relações, expressões, anti-joins, ordenar e pressionar políticas; contratos de consulta de registros de publicação e mapas de origem; os benchmarks comparam a avaliação de referência direta com a execução reduzida. O atual lançamento editorial da produção permanece deliberadamente mais simples, enquanto agregados, padrões ordenados, índices nativos e raciocínio avançado são tratados através de operadores diretos ou gráficos. Esta restrição é preferível a reivindicar uma linguagem de consulta universal madura antes que a semântica e a otimização sejam estáveis.

O modelo de publicação do repositório fortalece a reprodutibilidade. Os agentes nomeados possuem material de autoridade separado, esquemas, perfis de extração, circuitos, conjuntos de referência, candidatos, lançamentos, corridas, problemas e feedback. Um agente de codificação pode propor candidatos, mas não pode torná-los ativos. A produção lê um lançamento imutável. Esta estrutura torna possível reproduzir qual teoria, fundação, perfil do produtor, resumo de extensão e verificador gerou um resultado. Ele também permite que vários agentes de codificação ou famílias modelo constroem teorias concorrentes sem compartilhar silenciosamente uma ontologia ou liberação ativa.

A reivindicação implementada mais forte é, portanto, arquitetônica e não semântica: o nllAgent tem um caminho disciplinado de origem imutável para observações versionadas, da demanda de observação à compatibilidade, de circuitos restritos à publicação controlada por verificador e de um produto canônico a um relatório legível por humanos. Tem estados de parada explícitos. Não precisa fingir que cada documento é ontologicamente compatível. Trata os resultados do modelo como propostas limitadas. Estas são conquistas substanciais em

comparação com os pipelines convencionais de agentes LLM, onde o prompt, a interpretação, o raciocínio e a explicação geralmente existem apenas em um contexto transitório.

A arquitetura não estabelece, no entanto, que a linguagem natural arbitrária tenha sido fundamentada corretamente. Os períodos estáveis provam que existem evidências em um local; eles não provam que um ator, evento, exceção ou modalidade foi interpretado corretamente. Uma observação digitada ainda pode referir-se à entidade errada. Uma corrida completa pode ser concluída apenas para o contrato de observação limitado, não para todos os significados relevantes que um ser humano poderia inferir. A documentação própria do projeto e o registro de problemas são devidamente cautelosos sobre este ponto. O valor da implementação é que essas limitações têm lugares a serem representados, em vez de serem escondidas dentro de uma resposta final.

7.2 Avaliação Independente: Pontos Fortes, Problemas Graves Atuais e Evolução Necessária

Minha avaliação geral é positiva sobre os limites de confiança e cautelosa sobre as alegações semânticas. nllAgent é mais do que um invólucro rápido, mais do que um pipeline de vetores e mais do que uma coleção de esquemas JSON. Seus artefatos do lado da fonte e do lado da teoria, publicação imutável, limite de autoria restrito, verificação de repetição, verificações de cobertura e produtos canônicos constituem um substrato de pesquisa credível. A arquitetura está especialmente bem alinhada com o padrão federativo identificado em revisões neuro-simbólicas recentes: o componente neural propõe ou realiza a linguagem, enquanto o componente simbólico mantém sua própria identidade, semântica e critérios de aceitação [Chatzikyriakidis and Lappin 2026].

A evidência atual do projeto, no entanto, suporta a viabilidade limitada, não a linguagem natural executável geral. A fundação padrão reconhece um pequeno fragmento controlado. A demonstração editorial usa operadores estreitos e verificadores. A compilação do Consulty-first é experimental. A realização controlada é limitada. Nenhuma avaliação independente estabeleceu a fidelidade de formalização, resolução de identidade, transferência entre domínios, calibração de erros ou superioridade em relação a linhas de base mais simples. A estratégia de publicação correta é, portanto, indicar exatamente quais propriedades são implementadas e anexar todas as reivindicações mais fortes a um envelope de referência e garantia.

O atual registro de questões sérias identifica seis limitações de engenharia e semânticas. O primeiro é o agendador limitado de incrementalidade. O tempo de execução tem mecanismos de dependência e cache úteis, mas ainda não reivindica um mecanismo de ponto de referência incremental totalmente geral para circuitos arbitrários, relações recursivas, mundos alternativos e atualizações não monotone. Isso é importante para documentos longos, porque uma pequena edição deve invalidar apenas os artefatos semânticos e estados de circuito que dependem dele. Um replay completo ingênuo está correto, mas caro; um agendador incremental insalubre é rápido, mas perigoso. A solução requer contratos explícitos de monotonidade e efeito, proveniência de dependência, separação de época para retrações e comparações de referência entre execução incremental e completa.

A segunda questão é que o suporte de interpretação ainda não flui uniformemente através de cada caminho de execução. Um documento pode conter leituras, mundos, contextos ou condições de presença concorrentes, mas um operador, chave de cache, agregação ou verificador pode acidentalmente colapsá-los. Este é o obstáculo central para o tratamento do LongTextJS como um estado semântico abstrato. Cada valor que depende de uma interpretação deve levar seu contexto. As junções devem combinar contextos compatíveis. A agregação deve dizer se ela varia dentro de um mundo ou através de alternativas. A identidade do cache deve incluir a fórmula do contexto. Os operadores que não podem preservar alternativas devem ser declarados incompatíveis em vez de escolher silenciosamente um.

Esta questão não é uma pequena lacuna de recursos. A promessa científica central da linguagem natural executável é muitas vezes que o sistema pode dizer o que é verdadeiro sob todas as leituras admissíveis, o que é possível sob pelo menos um, e que resulta de uma interpretação contestada. Se as alternativas forem perdidas em uma rota de execução interna, uma interface polida pode / deveria seria enganosa. O projeto deve, portanto, fazer da

preservação da interpretação uma invariante em tempo integral e testá-la através de mundos finitos gerados, propriedades metamórficas e replay diferencial.

A terceira questão é a compatibilidade conservadora para o comportamento processual opaco. Extensões e operadores registrados são necessários porque alguns algoritmos úteis são difíceis de expressar de forma declarativa. Mas um compilador não pode inferir demandas sintéticas precisas, efeitos, monotonicidade, uso de recursos ou obrigações de testemunhas de JavaScript opaco. A rejeição conservadora é mais segura do que a aceitação otimista, mas a opacidade excessiva reduz a composibilidade e torna a materialização menos eficaz. Cada procedimento primitivo precisa de um contrato de propriedade do registro declarando tipos de entrada e saída, status epistêmico aceito, requisitos de cobertura, possíveis efeitos, determinismo, monotonicidade, comportamento de cache, classe de custo e testemunha legível por verificador. O contrato deve ser mais autoritário do que os comentários escritos pelo autor da extensão.

A quarta edição é o planejamento mínimo de consultas e indexação temporal. Um mundo de consulta finito pode ser executado corretamente sem otimização sofisticada, mas o uso em escala de documentos exigirá planejamento de associação, estimativas de seletividade, índices de extensão de origem e entidade, índices de intervalo temporal e manutenção incremental. Essas otimizações não são apenas detalhes de desempenho. Um planejador que reescreve as consultas deve preservar a multiplicidade de evidências, fórmulas de contexto, ordenação, semântica anti-join e cobertura. Os índices nativos devem permanecer aceleradores descartáveis em vez de fontes de verdade. Os testes diferenciais devem comparar a avaliação otimizada e de referência sobre os mundos gerados, especialmente para consultas de ausência, sobreposição temporal, interpretações alternativas e evidência duplicada.

A quinta questão é a geração controlada deliberadamente estreita. A realização exata ou verificada de descobertas e planos limitados é valiosa, mas ainda não equivale a uma prosa verificada de forma longa. Geração de forma longa introduz agregação, pronomes, elipse, ênfase retórica, relações de discurso e implicações criadas pela ordem da sentença. Uma realização pode conter apenas declarações atômicas licenciadas e ainda implicar uma narrativa causal ou normativa não suportada. O próximo passo deve ser a realização da prova em vez de um escritor mais restrito. Um resultado canônico deve fornecer identificadores de reivindicação, entalhes de entidade e quantidade, modalidade, incerteza e relações de discurso permitidas. O texto gerado deve ser novamente comparado e comparado com essas reivindicações. As incompatibilidades críticas devem rejeitar o texto; material estilístico fora do fragmento verificado deve ser rotulado em um nível de garantia inferior.

A sexta questão é o suporte de mutação específico do cenário. A precisão comum de referência pode ocultar regras que são inertes, redundantes ou incorretamente de escopo. Um circuito pode passar todos os exemplos selecionados, mesmo que uma exceção seja ignorada porque nenhum caso o ativa. Uma avaliação séria deve gerar mutantes semânticos tipados que alteram os limiares, comparadores, quantificadores, negação, modalidade, links de identidade, ordem temporal, estado da evidência, predicados de cobertura, escopo de exceção, prioridade de regra e rotas de verificadores. Um conjunto de testes “mata” um mutante quando o resultado canônico esperado muda da maneira necessária. Os mutantes sobreviventes revelam testes fracos ou cláusulas semanticamente redundantes. A mutação deve cobrir programas de origem, teorias de circuito, rebaixamentos do compilador e restrições de realização, não apenas strings em exemplos.

Essas seis edições são mais informativas do que um rótulo genérico de “protótipo” porque identificam onde a arquitetura atual pode perder escala ou significado. Eles também sugerem um caminho de evolução coerente. O primeiro marco deve ser uma semântica formal limitada para o mundo da loja semântica e consulta. Não é necessário formalizar o inglês irrestrito. Deve definir mundos de origem imutáveis, identidade de observação, estado epistêmico, cobertura, alternativas, transações, resultados de consultas, publicação e retração. Os principais invariantes podem ser verificados por meio de instâncias finitas e testados em termos de propriedades na implementação.

O segundo marco deve ser a execução consciente da interpretação. Condições de presença ou contextos fatorados devem acompanhar observações de dependentes de alternativas. O agendador, o mecanismo de consulta, o registro primitivo, o cache, o verificador e os produtos canônicos devem preservar esses contextos ou declarar incompatibilidade. Pode, não deve, e desconhecido deve ser computado a partir de conjuntos explícitos ou abstrações de mundos admissíveis, não da confiança do modelo. Isso criaria a ponte concreta da arquitetura existente para a interpretação abstrata.

O terceiro marco deve ser um registro de formalismo, em vez de um CircuitJS em expansão indefinidamente. CircuitJS é melhor usado como uma orquestração, contrato de evidência e linguagem de publicação. Os motores especializados devem possuir semântica especializada: avaliação relacional e Datalog para fechamento e junções; tabelas de decisão para precedência da política; SAT, SMT, CSP ou MaxSAT para restrições; autômatos para padrões ordenados; solucionadores temporais e verificadores de modelos para rastreamentos; execução simbólica para testemunhas de caminho; estruturas de fluxo de dados para propagação ampliada; declarações de depoimentos de tesedores ou des. Cada adaptador deve declarar um fragmento de entrada digitado, suposições de tradução, certificado de saída, verificador de repetição e modos de falha.

O quarto marco deve ser o refinamento semântico guiado por contra-exemplos. Uma análise abstrata grosseira pode identificar uma possível violação. Um solucionador ou executor simbólico então procura uma testemunha coerente. Se a testemunha estiver satisfatória e rejogável, o sistema o relata com o vão de origem. Se for espúrio, um núcleo ou conflito insatisfatório deve identificar qual distinção de identidade, temporal, modal, quantidade, exceção ou cobertura foi perdida. O pedido de refinamento deve visar apenas a fonte e o produtor relevantes. O artefato semântico reparado torna-se uma nova versão e o contra-exemplo torna-se um caso de regressão. Este loop seria uma síntese genuinamente nova da proposta semântica baseada em LLM, interpretação abstrata e execução simbólica.

O quinto marco deve ser uma camada de fundamentação mais forte. A segurança e as âncoras de origem são necessárias, mas insuficientes. O tempo de execução precisa de conjuntos de identidade de candidatos, restrições explícitas, validade temporal, verificações de unidade e jurisdição, autoridade de origem e políticas de conflito. Os links top-one gerados por modelo não devem mesclar destrutivamente objetos. As entidades críticas devem utilizar identificadores autorizados ou confirmação humana. O conhecimento externo deve entrar através de pacotes somente de leitura, com recursos, intervalos efetivos, jurisdição, expiração, resolução de conflitos e limites máximos de garantia. A recuperação da Web pode propor atualizações de pacotes, mas não deve se tornar uma verdade implícita atemporal.

O sexto marco deve ser uma avaliação independente. Os dispositivos de determinação estabelecem reprodutibilidade do software, mas não a qualidade da tradução semântica ao

vivo. Um perfil de avaliação opt-in deve fixar modelo, provedor, prompts, versões de ferramentas, temperatura e política de captura; repetir casos para medir a variância; registrar custo e latência; e proibir o fallback silencioso. Um avaliador de referência implementado separadamente deve comparar os resultados de consultas e circuitos sempre que possível. A anotação humana deve se concentrar na fidelidade de fonte para semântica e nas ambiguidades relevantes para os resultados. As linhas de base devem incluir julgamento direto LLM, LLM mais recuperação, LLM-para-um-formalismo, compilação de linguagem controlada e sistemas de padrões determinísticos. O projeto deve demonstrar quando sua arquitetura adicional produz valor mensurável.

Uma avaliação útil pode ser resumida sem inflar as reivindicações. Preservação da fonte, lançamentos imutáveis, persistência de traços, separação de observações documentais da teoria reutilizável, cobertura explícita, autoria restrita de circuitos, publicação controlada por verificador e aprendizado governado são decisões de design fortes. A compreensão semântica geral, a execução completa, o planejamento de consultas em escala de produção, a geração de forma longa, a mutação abrangente e a validação empírica independente permanecem em aberto. A credibilidade da arquitetura vem em parte da admissão desse limite.

A principal oportunidade científica não é adicionar mais sintaxe DSL. É demonstrar solidez semântica condicional para um fragmento delimitado, mas não trivial. Um resultado publicável poderia especificar uma linguagem documental contendo entidades, eventos, relações temporais, quantidades, modalidade, exceções explícitas e referências alternativas; definir o conjunto de interpretação admitido; provar ou testar mecanicamente que todo resultado publicado pode/deve é correto em relação a esse conjunto; mostrar que o refinamento simbólico elimina uma classe mensurável de advertências espúrias; e verificar se as explicações geradas preservam o resultado canônico. Tal resultado seria mais estreito do que o “inglês executável”, mas substancialmente mais importante.

A principal oportunidade prática é uma nova classe de sistemas de documentos e agentes que estão dispostos a parar. Um assistente LLM normal é otimizado para continuar produzindo uma resposta. Um tempo de execução simbolicamente fundamentado pode retornar ontologia incompatível, identidade não resolvida, cobertura insuficiente, formalismo sem suporte, exaustão orçamentária, contraexemplo espúrio ou realização não licenciada. Esses resultados podem parecer menos impressionantes em uma demonstração, mas são pré-requisitos para uso confiável em ciência, regulamentação, engenharia e tomada de decisões institucionais.

Meu julgamento independente é, portanto, que o nllAgent escolheu muitos dos limites arquitetônicos certos e deve resistir à pressão para ampliar suas reivindicações mais rapidamente do que amplia suas evidências semânticas. Já é um protótipo credível de análise de linguagem governada, fundamentada em termos de origem, separada por teoria. Ainda não é evidência de que a linguagem natural irrestrita pode ser executada com garantias formais. Seu melhor caminho a seguir é se tornar uma plataforma experimental para a reivindicação mais estreita e mais defensável desenvolvida ao longo deste livro: LLMs descobre candidatos formais, um kernel simbólico controla fundamentação e consequência, e a saída de linguagem natural permanece subordinada a um resultado canônico reproduzível.

8. Construindo e Validando um Runtime Neuro-Simbólico

8.1 Estratégia de Implementação: Um Sistema de Linguagem Executável Mínimo, mas Cientificamente Honesto

A arquitetura proposta neste livro é ambiciosa apenas se for descrita vagamente. Uma vez que suas obrigações são separadas, o primeiro sistema implementável é comparativamente modesto. Não precisa entender a linguagem arbitrária, construir uma ontologia universal ou provar cada conclusão. Ele precisa aceitar uma classe de documentos limitada, preservar evidências exatas, materializar uma pequena família de observações digitadas, compilar uma família restrita de regras em um ou dois motores formais, retornar testemunhas ou desconhecidos explícitos e impedir que a camada final de linguagem natural altere o resultado simbólico. O valor científico vem de tornar cada limitação visível e mensurável.

A primeira decisão de design é a reivindicação de alvo. Um protótipo de pesquisa não deve começar com “o sistema entende e executa a linguagem natural”. Deve começar com uma reivindicação como: documentos dados de um gênero declarado, uma ontologia de versão, uma família finita de produtores de observação e um conjunto de regras publicadas, o tempo de execução determina se cada regra é satisfeita, violada, potencialmente violada, não apoiada ou fora da cobertura semântica do lançamento. O resultado é rejogável e ligado a extensões de origem. Essa afirmação é estreita o suficiente para testar e ampla o suficiente para suportar a revisão de conformidade, análise de requisitos, lignação de documentos científicos, inspeção de protocolo e planejamento controlado.

A camada de origem deve ser deliberadamente chata. Cada documento é armazenado como uma revisão imutável com um resumo estável. Unidades estruturais — cabeçalhos, parágrafos, frases, itens de lista, células de tabela, legendas, notas de rodapé e objetos incorporados — recebem identificadores persistentes. A normalização é registrada em vez de silenciosamente aplicada. Os deslocamentos de caracteres devem se referir a uma representação de origem canônica, enquanto os mapeamentos preservam as posições no arquivo original. Uma declaração em uma tabela e uma frase visualmente idêntica em um parágrafo pode ter papéis estruturais diferentes; essa distinção deve sobreviver à extração. Quando OCR é inevitável, a saída OCR e a imagem da página são objetos de evidência separados, e a incerteza na transcrição não é recolhida em texto comum.

Esta disciplina de origem não é uma sobrecarga administrativa. É o que torna as conclusões simbólicas contestáveis. Uma descoberta que cita apenas um resumo gerado não pode ser verificada de forma independente. Uma descoberta vinculada a texto exato, células de tabela e registros de transformação pode ser revisada mesmo que todos os modelos de idiomas envolvidos tenham mudado. O pacote de origem deve, por conseguinte, ser considerado parte do registro de auditoria fideiocomissário responsável, embora os extratores que o criam permanecem falíveis e exijam testes de formato específicos.

A segunda camada é um gráfico de evidências em vez de um único parse. O grafo contém menções, entidades, eventos, estados, quantidades, expressões temporais, modalidade, relações e links de origem. Crucialmente, também representa alternativas e status. Um link de entidade proposto por modelo não é armazenado como uma igualdade incondicional. É uma

relação de identidade candidata com um produtor, pontuação, vãos de apoio, evidências conflitantes e um status epistêmico, como proposto, corroborado, aceito, rejeitado ou não resolvido. Uma relação temporal inferida de “logo depois” não deve ser normalizada a uma duração exata arbitrária; deve reter a relação qualitativa e qualquer interpretação de domínio declarada.

Esta camada deve permanecer finita e limitada à tarefa. O sistema não está construindo um modelo de mundo completo. Está materializando as distinções exigidas pelos circuitos publicados e pela consulta atual. A materialização de consulta, já explorada no nllAgent, é o padrão correto. Um circuito declara que ele precisa de identidade de atores, relações documentais, modalidade deontica e datas efetivas. O compilador deriva essas demandas antes de uma interpretação cara. Os produtores então geram apenas as observações necessárias para decidir a compatibilidade e executar a regra. Isso reduz o custo e, mais importante, limita a superfície semântica sobre a qual os erros podem ocorrer.

A terceira camada é um núcleo de fundamentação simbólico. “Fundamentação” é frequentemente usada vagamente para significar que uma resposta cita uma passagem. Aqui tem um significado mais rigoroso: todo símbolo admitido à execução formal tem um referente datilografado, proveniência, escopo e status suficientes para o formalismo alvo. Um predicado como aprovado (x, y, t) não pode ser introduzido simplesmente porque um LLM produziu esses tokens. O kernel verifica se x e y se referem a tipos de entidade admissível, que t é um valor de tempo ou intervalo suportado, que o produtor de observação está autorizado para este predicado, que o intervalo de origem existe na revisão declarada e que o status atende ao requisito de garantia da regra.

Os contratos de fundamentação devem distinguir evidências da ontologia. Evidências dizem que um período suporta uma observação. A ontologia diz que tipo de coisa a observação denota e quais relações são admissíveis. Os axiomas de domínio dizem o que se segue a essas relações. O conhecimento mundial fornece reivindicações de origem externa. Essas camadas não devem ser mescladas. Um registo da empresa pode estabelecer que duas denominam uma entidade jurídica durante um período determinado. Uma ontologia de domínio pode declarar que as entidades legais podem assinar acordos. Uma teoria política pode declarar que uma assinatura de um representante autorizado conta como aprovação. Um modelo pode sugerir todas as três conexões, mas cada conexão pertence a uma autoridade diferente e deve ser aceita ou rejeitada separadamente.

A identidade merece seu próprio serviço porque é a dependência oculta mais comum em raciocínio de documentos longos. O serviço deve gerenciar clusters de menção, aliases explícitos, identificadores, funções organizacionais, validade temporal e mundos selecionados. Deve permitir vários candidatos não resolvidos em vez de forçar a unificação precoce. Em uma primeira versão, a identidade pode ser intencionalmente conservadora: identificadores explícitos e aliases declarados exatos são aceitos; links baseados em funções e contextuais permanecem propostos; identidade de documentos cruzados sem suporte impede regras que o exigem. Essa política conservadora reduzirá a cobertura aparente, mas melhorará muito o significado dos resultados aceitos.

O tempo precisa de um domínio igualmente explícito. Uma representação inicial prática pode combinar instantes exatos, intervalos fechados ou abertos, ordem qualitativa e limites desconhecidos. O tempo de execução deve distinguir a ordem do documento da ordem do evento e do tempo de publicação do tempo efetivo. Frases temporais como “antes da

implantação”, “dentro de trinta dias”, “enquanto a autorização permanece válida” e “depois de qualquer mudança material” induzem estruturas diferentes. O compilador deve rejeitar uma regra temporal se o gráfico de evidência tiver apenas observações de ordem de documentos quando a semântica do tempo de evento for necessária. Substituição silenciosa entre domínios temporais é uma fonte clássica de conclusões plausíveis, mas inválidas.

A moderação e a força normativa requerem uma pequena rede em vez de um booleano. No mínimo, o sistema deve distinguir afirmação, relatório, crença, possibilidade, capacidade, permissão, obrigação, proibição, recomendação, intenção, condição e contrafactual. Deverá igualmente registrar o titular e beneficiário de uma norma, se for caso disso. O provedor pode divulgar dados e “o provedor deve divulgar dados” compartilhar vocabulário, mas ter consequências opostas à conformidade. “O documento afirma que o provedor cumpriu” é evidência de um relatório, não necessariamente evidência do evento subjacente. Um tempo de execução formal torna-se útil precisamente ao recusar esses colapsos.

Uma vez que existem observações fundamentadas, a quarta camada calcula um estado semântico abstrato. Um domínio mínimo pode representar, para cada proposição relevante para a consulta, seja ela verdadeira em todas as interpretações admissíveis, verdadeiras em algumas, falsas ao todo, contraditórias ou não resolvidas por causa da falta de cobertura. A implementação não precisa começar com uma biblioteca de treliza sofisticada. Pode começar com alternativas finitas explícitas e um resumo de pode/deve-se de origem mecânica. Conforme a escala cresce, os domínios abstratos podem resumir candidatos a identidade, intervalos numéricos, intervalos temporais, status modais e conjuntos de proveniência sem enumerar cada mundo.

O estado abstrato realiza dois trabalhos. Primeiro, fornece uma propagação conservadora em todo o documento. Se toda interpretação aceita vincular uma definição a um termo, essa relação é um fato obrigatório. Se apenas uma das três atribuições de identidade plausíveis criar um conflito de interesses, o conflito é uma descoberta pode ser uma violação verificada. Segundo, decide onde a execução mais cara é justificada. Um resultado seguro pode terminar cedo. Um resultado de pode-violação desencadeia uma busca de testemunhas. Um desconhecido causado pela cobertura do produtor ausente desencadeia um problema de compatibilidade semântica, em vez de uma chamada de solucionador.

A quinta camada é um plano de formalismo. O plano não é uma prosa de forma livre escrita por um agente. É uma decisão de compilação tipada que afirma qual pergunta será feita sobre qual mecanismo, quais observações são entradas, o que garante o retorno do motor e como seu resultado mapeia para o vocabulário de resultados canônicos. A escolha deve ser impulsionada por forma semântica em vez de moda.

A avaliação relacional é o primeiro mecanismo a ser implementado porque muitas verificações úteis de documentos reduzem para junções, comparações de pedidos, agrupamentos e definem diferenças. Regras como “todo termo técnico definido é usado de forma consistente”, “toda reivindicação aceita tem pelo menos uma fonte”, “nenhum identificador de requisito é duplicado” ou “todos os riscos listados têm um proprietário de mitigação” não exigem um provador de teoremas. Um núcleo relacional é transparente, eficiente e fácil de testar. O Datalog estende esse núcleo quando o fechamento transitivo ou dependências recursivas são necessárias. Exemplos incluem herança, propagação parcial, cadeias de dependência e acessibilidade por meio de gráficos de citação ou exigência.

As tabelas de decisão são o segundo formalismo útil para políticas limitadas. Uma tabela de decisão torna as combinações e resultados de condições explícitos, expõe linhas sobrepostas ou ausentes e suporta a análise de cobertura. É especialmente apropriado para elegibilidade, encaminhamento, classificação e regras políticas cuja complexidade vem de combinações em vez de recursão. Um circuito pode compilar uma política de linguagem natural em uma tabela de candidatos, mas a publicação deve exigir exemplos de nível de linha, detecção de sobreposição, verificações de integridade e aprovação humana da partição semântica.

Os motores SAT, SMT, CSP e MaxSAT tornam-se úteis quando a tarefa contém restrições de interação, aritmética, horários, atribuições ou otimização. O compilador deve escolher teorias suportadas e evitar esconder semânticas não suportadas em strings não interpretadas. Um resultado SMT pode provar inviabilidade sob restrições codificadas, produzir um modelo satisfatório ou expor um núcleo insatisfável. O núcleo deve ser mapeado de volta às obrigações e suposições de origem. Esse mapeamento é mais informativo do que pedir a um LLM para explicar um erro de resolução genérico; ele identifica os menores compromissos formais que não podem ser mantidos em conjunto.

A lógica temporal e a verificação de modelos são apropriadas para procedimentos, fluxos de trabalho, protocolos e regras de ciclo de vida. O gráfico de evidências fornece eventos e proposições estaduais; o compilador constrói um sistema de transição e propriedade temporal; o verificador retorna satisfação, um traço de contra-exemplo ou um resultado inconclusivo sob os limites. A parte difícil não é o verificador temporal. É decidir se a fonte define um sistema de transição completo, se as transições omitidas são impossíveis ou meramente sem documentos, e se as suposições justas ou ambientais são justificadas. O resultado canônico deve, portanto, expor o limite do modelo.

A execução simbólica é apropriada quando uma regra se refere a caminhos viáveis através de código executável, um procedimento stateful ou um artefato semântico semelhante a um programa. Sua saída deve ser uma testemunha concreta: atribuições, condições de ramificação, transições de estado e links de origem. Em configurações somente de documentos, a execução simbólica pode explorar encadernações alternativas ou ramificações processuais, mas não deve ser invocada apenas porque o sistema contém símbolos. O termo deve manter seu significado sensível ao caminho. O SymGPT é instrutivo porque a regra da linguagem natural é compilada em uma condição de violação e depois testada contra caminhos contratuais reais, em vez de contra uma prosa mais gerada [Xia et al. 2026].

Lógica natural, autômatos finitos, análise de gráficos, computação de semi-anel e provadores de teoremas especializados pertencem ao portfólio quando seus pressupostos estruturais se encaixam. A lógica natural pode lidar eficientemente com a monotonicidade e certas implicações lexicais perto da linguagem superficial. Os autômatos são excelentes para padrões locais ordenados. Métodos de gráfico e semiquartamento podem manter várias análises computação compartilhada. Os assistentes de prova são apropriados quando um pequeno número de reivindicações de alto valor justificam a formalização manual e as provas verificadas pelo kernel. A arquitetura deve fazer da adição de um motor uma extensão tipada, não um convite para incorporar código executável arbitrário em circuitos.

A sexta camada é refinamento semântico guiado por contra-exemplos. Uma descoberta pode ser compilada em um problema de viabilidade. Se o solucionador retornar uma testemunha, o tempo de execução a repete contra o gráfico de evidências e relata as escolhas de interpretação exatas necessárias. Se o caminho for inviável, o motor retorna um núcleo

insatisfatório, pré-condição com falha ou conjunto de conflitos. O sistema traça esse objeto de volta através da compilação para as observações de origem e mapeamentos de candidatos que o produziram. Só então um LLM é solicitado a ajudar.

O prompt de reparo deve ser local e digitado. Pode perguntar se duas menções foram incorretamente unificadas, se uma condição foi pretendida como necessária ou suficiente, se uma exceção se aplica, ou se uma relação temporal tem o ponto final errado. Deve apresentar a fonte relevante, contratos de ontologia, alternativas atuais e o conflito simbólico. O LLM propõe um ou mais artefatos semânticos revisados. Essas propostas passam pelos mesmos controles de base e compilação que os originais. Um ser humano pode ser inserido quando a ambiguidade é material normativa ou cientificamente.

Este processo é fundamentalmente diferente da auto-reflexão genérica. O modelo não é dito que sua resposta estava errada e convidado a improvisar outra cadeia de pensamento. É dado um conflito mecanicamente identificado em um artefato semântico persistente. O reparo pode ser testado. Se o reparo mudar uma ontologia ou produtor, a publicação requer novos casos de referência e um diff semântico. O contra-exemplo torna-se uma parte permanente do corpus de avaliação.

A sétima camada é o resultado canônico. Deve ser um produto estável e independente de resolução que separa o veredicto, evidência, suposições, interpretações, execução e limitações. Para uma regra de auditoria, ele registra a versão da regra, escopo aplicável, estado de compatibilidade, cobertura, observações aceitas, observações rejeitadas, resultado de execução, referência de testemunha ou prova, resultado do verificador e problemas não resolvidos. Para um plano, ele registra metas, restrições, etapas, dependências, recursos, alternativas e as verificações exatas que o plano satisfaz. O produto canônico é a autoridade para qualquer resposta em linguagem natural subsequente.

O caminho de retorno à linguagem deve ser tratado como compilação, não geração gratuita. Um renderizador recebe um conjunto finito de reivindicações licenciadas. Cada reivindicação tem modalidade, polaridade, quantidade, referências de entidade, incerteza e proveniência. O renderizador pode escolher a ordem do discurso e o fraseado explicativo, mas não pode introduzir novas entidades, fortalecer pode-se, converter evidências ausentes em ausência, suprimir uma suposição material ou inventar uma explicação causal. Os identificadores de reclamação em nível de condenação devem permanecer disponíveis para auditoria. Um segundo verificador deve comparar o texto renderizado com o resultado canônico, usando testes determinísticos para nomes, quantidades, datas, negação e modalidade e, quando necessário, um modelo separado limitado a implicação textual em vez de avaliação aberta.

A percepção comprovada é especialmente importante porque os usuários julgarão o sistema por sua prosa. Um traço simbólico perfeito pode ser minado por um resumo elegante, mas impreciso. A arquitetura deve, portanto, tornar a verbosidade opcional, mas obrigatória a fidelidade. No modo de alta garantia, o sistema pode gerar linguagem simples e repetitiva porque cada frase segue um modelo licenciado pelo tipo de resultado. No modo explicativo, um LLM pode melhorar a legibilidade, mas as sentenças não suportadas são rejeitadas e regeneradas. O artefato canônico, não a prosa, continua sendo o registro da decisão.

A segurança e a governança devem ser integradas desde o início. Os documentos em linguagem natural são entradas não confiáveis e podem conter injeção rápida, exemplos adversários, formatação enganosa ou fragmentos de código. Os produtores semânticos devem receber dados de origem através de interfaces digitadas, em vez de um prompt de sistema

privilegiado que pode ser substituído pelo conteúdo do documento. Artefatos formais gerados devem ser analisados por gramáticas restritivas. Os solutores devem ser executados abaixo dos limites de recursos. As extensões de tempo de execução devem ser revisadas, versionadas e bloqueadas por digestão. Os espaços de trabalho de aprendizagem não devem publicar diretamente na produção. Esses limites já estão parcialmente presentes no nllAgent e devem ser fortalecidos em vez de ignorados por conveniência [AssistOS-AI 2026].

Um programa de execução realista pode prosseguir em cinco lançamentos. A primeira versão deve suportar a ingestão imutável de Markdown ou DOCX, extensões de origem, observações estruturais, um mecanismo de consulta relacional, observações controladas restritas em inglês, estados de cobertura, circuitos reutilizáveis, verificação de repetição e achados canônicos. Deve resolver tarefas limitadas, como completude de proveniência, consistência de definição, cobertura de requisitos, regras de seção ordenada e distinções modais simples. Nenhuma identidade geral, inferência temporal ou recuperação do mundo atual devem ser reivindicadas.

A segunda versão deve adicionar a identidade digitada e o tempo. A identidade começa com identificadores explícitos, aliases e funções declaradas por documentos; a semântica temporal começa com datas exatas, intervalos, ordem de documentos e um pequeno conjunto de padrões de ordem de eventos. Os benchmarks devem conter aliases ambíguos, mudar de papéis, intervalos de sobreposição e faltar deliberadamente elos que faltam. O sistema deve parar em vez de adivinhar quando um circuito requer uma identidade mais forte do que a liberação fornece.

A terceira versão deve adicionar tabelas de decisão e execução SMT/CSP, com refinamento semântico guiado por núcleo insatisfatório e refinamento semântico guiado por contador. Esta versão pode abordar políticas de elegibilidade, cronogramas, restrições de atribuição, consistência quantitativa e viabilidade de procedimento limitado. Cada resultado do solver deve ser reproduzido através de um verificador independente ou um verificador menor, sempre que possível. Os testes de mutação devem alterar restrições e observações para demonstrar que os conjuntos de referência detectam regressões semânticas.

A quarta versão deve adicionar um adaptador de verificação de modelos e um adaptador de execução simbólica sensível ao caminho para um domínio concreto, como especificações de fluxo de trabalho ou regras de contrato inteligente. O objetivo não é uma cobertura ampla, mas a prova de que o registro do formalismo e a interface de certificação canônica suportam semântica genuinamente diferente. Os testes de consistência do formador cruzado devem verificar se os fatos em terra compartilhados mantêm identidade, tempo, modalidade e proveniência em rebaixamentos.

A quinta versão deve adicionar planejamento e realização controlada. Uma ideia é compilada em um plano canônico, verificada em relação a restrições formais, realizadas em prosa e auditadas pelo mesmo sistema de regras. A avaliação deve distinguir a validade do plano, a fidelidade da realização, a qualidade linguística e a eficiência do reparo. O plano não se torna confiável apenas porque a prosa era gerada a partir de um objeto estruturado; torna-se mais auditável porque o objeto e suas verificações são persistentes.

Artifact stack for a replayable executable-language runtime

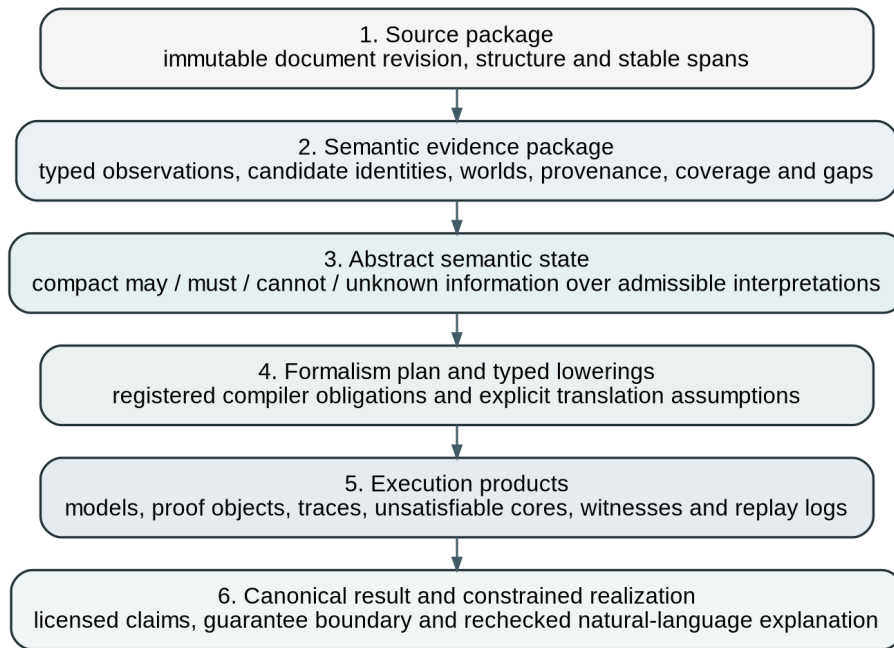


Figura 4. Pilha de artefatos para um tempo de execução executável-natural-idioma rejogável. Cada camada é versionada, mapeada em origem e inspecionável de forma independente.

Essa implementação encenada incorpora um importante princípio metodológico: a ambição semântica só deve crescer quando o sistema pode detectar que sua representação anterior é inadequada. O tempo de execução não deve ocultar uma linguagem sem suporte por trás da confiança do modelo. Ele deve expor as lacunas de compatibilidade como resultados de primeira classe e transformar lacunas recorrentes em novos produtores, ontologia, o motor e o trabalho de referência. A linguagem natural executável avança através da expansão governada de um envelope de solidez, não através de um salto para a compreensão universal.

8.2 Questões de Pesquisa, Benchmarks e um Programa de Publicação

A arquitetura só se torna uma contribuição científica quando suas reivindicações são refutáveis. Uma demonstração em que um LLM gera um artefato formal plausível e um solver retorna uma resposta é insuficiente. O mesmo modelo pode gerar o benchmark, a tradução e a explicação, criando um ciclo fechado de erro correlacionado. Uma avaliação séria deve decompor o gasoduto e testar cada obrigação de forma independente. Deve também incluir condições de insucesso em que a abordagem proposta deve ser rejeitada ou estreitada.

A primeira questão de pesquisa diz respeito à representação: a manutenção de várias interpretações semânticas admissíveis melhora a confiabilidade na seleção de uma tradução? A hipótese correspondente é que a semântica candidata-conjunto, combinada com a análise de maio/deve, reduz a falsa garantia em insumos ambíguos e subespecificados. Pode diminuir a taxa de resposta aparente porque mais casos se tornam condicionais ou desconhecidos. Isso não é necessariamente um defeito. As medidas de comparação relevantes se as conclusões aceitas devem ser mais frequentemente corretas e se os resultados podem capturar alternativas materialmente plausíveis sem revisores esmagadores.

Uma referência para esta questão deve conter textos com conjuntos de interpretação anotados de especialistas, em vez de uma forma lógica de ouro. Os pares mínimos devem variar o escopo do quantificador, negação, anexo de exceção, coordenação, co-referência, limites temporais, força modal e jurisdição. Alguns itens devem ter uma leitura claramente pretendida, algumas leituras legítimas e algumas evidências insuficientes para qualquer resposta determinada. Os sistemas são avaliados na cobertura de leituras admissíveis, exclusão de leituras inadmissíveis, calibração do status de maio/deve e veredictos downstream. A correspondência exata nos gráficos é muito frágil; a equivalência semântica deve ser testada através de suítes de consultas ou implicação bidirecional sob a ontologia de referência.

A segunda questão de pesquisa diz respeito ao refinamento: os conflitos derivados do solver são mais úteis para reparar a formalização do que a autocrítica do modelo genérico? A hipótese é que os contraexemplos, os núcleos insatisfeitos e as obrigações de prova falhadas produzem reparos mais localizados, estáveis e com eficiência de amostra, pois identificam a dependência responsável pelo resultado. Um experimento controlado deve comparar quatro condições: repetição direta, auto-reflexão verbal, feedback de erro de execução e refinamento semântico guiado por contraexemplos com mapeamento de origem. O mesmo modelo, temperatura e orçamento devem ser usados sempre que possível.

A qualidade do reparo deve ser medida não apenas se o exemplo de alvo se tornar correto. O reparo deve preservar casos não relacionados, reduzir o diff semântico e evitar a introdução de suposições sem suporte. As suítes de regressão baseadas em mutações são essenciais. Um reparo que resolva um exemplo enfraquecendo a regra globalmente não é um sucesso. O experimento deve registrar o número de iterações, fichas, intervenções humanas, mutantes sobreviventes, observações alteradas e se o artefato final era reproduzível de forma independente.

A terceira questão de pesquisa diz respeito ao contrato de prova. Uma separação de fontes tipificada de vãos de origem, observações, asserções de ontologia, axiomas de domínio e premissas de conhecimento mundial melhora a transferência e a auditabilidade em comparação com um gráfico de conhecimento plano ou cadeia de pensamento de forma livre? A hipótese é que os contratos tipados reduzem a introdução da premissa silenciosa e tornam a

reutilização de vários domínios mais previsível. O custo provável é o de autoria e mais paradas correm.

O benchmark deve incluir a mesma regra aplicada em documentos de vários domínios, com variação controlada no vocabulário e na ontologia. Por exemplo, um requisito geral de que todas as reivindicações aceitas tenham evidências rastreáveis podem ser testados em documentos de pesquisa, documentos de políticas, especificações técnicas e planos. Observações estruturais podem transferir diretamente; entidades específicas do domínio e produtores de relações não podem. A avaliação registra quais verificações de compatibilidade param, se o motivo de parada está correto e quanto material semântico novo é necessário para retomar a execução. A transferência bem-sucedida não é apenas obter uma resposta; é reconhecer com precisão o que pode e o que não pode ser reutilizado.

A quarta questão de pesquisa diz respeito à escolha do formalismo. O roteamento para um portfólio de motores especializados supera uma linguagem intermediária universal sob supervisão semântica igual? Beiser e Penz mostram que a escolha de linguagem intermediária pode alterar substancialmente a precisão geral e a precisão da execução em tarefas de raciocínio neuro-simbólico [Beiser and Penz 2025]. A hipótese aqui é mais específica: um roteador digitado que seleciona o formalismo adequado mais simples melhorará o sucesso da compilação, a qualidade da explicação e a eficiência sem sacrificar a consistência entre tarefas.

Uma comparação significativa não deve permitir ao sistema de portfólio mais informações ocultas. Cada tarefa recebe a mesma evidência e ontologia. A linha de base monoformalismo pode usar lógica de primeira ordem, ASP, SMT, ou uma linguagem geral de consulta gráfica. O sistema de portfólio pode escolher consultas relacionais, Datalog, tabelas de decisão, SMT, lógica temporal ou execução simbólica. As métricas incluem validade de formalização, equivalência semântica, sucesso do solver, tempo de execução, qualidade de testemunhas e custo de manutenção. Os casos de formacionalismo cruzado devem exigir dois motores e testar se os símbolos compartilhados permanecem consistentes.

O experimento também deve buscar contra-evidência. Uma representação universal pode superar o portfólio em domínios estreitos porque o roteamento introduz erros e vários compiladores aumentam a base confiável. A conclusão esperada não é que os portfólios sejam sempre superiores. É que a pluralidade formalidade se justifica apenas quando a distribuição de tarefas contém genuinamente diferentes estruturas computacionais e quando as interfaces preservam a semântica.

A quinta questão de pesquisa diz respeito à camada de linguagem final. A realização com restrições de prova reduz as reivindicações sem suporte ou semanticamente fortalecidas, preservando a legibilidade aceitável? A hipótese é que um resultado canônico mais geração licenciada por reivindicações reduz substancialmente a alucinação e a prodízia da modalidade em relação à geração de respostas diretas ou à geração de respostas a partir de um traçado de solucionador bruto. O custo pode ser menos natural e mais regeneração.

O benchmark deve incluir resultados canônicos com incerteza sutil, evidências conflitantes, condições aninhadas, quantidades e datas. Os avaliadores humanos julgam a fidelidade factual, a completude das limitações materiais, a legibilidade e a utilidade. Os verificadores determinísticos medem a entidade, número, data, polaridade e preservação da modalidade. Um conjunto adversário deve tentar o renderizador a produzir uma conclusão mais forte e simples do que o resultado suporta. A métrica central não é a preferência estilística, mas a taxa na qual a prosa excede a autoridade do produto simbólico.

A sexta questão de pesquisa diz respeito à própria arquitetura federativa. A revisão sistemática de 2026 do NLP neuro-simbólico relata ganhos promissores para os sistemas federativos, mas adverte que a evidência é confundida pelas diferenças de tarefa [Chatzikyriakidis and Lappin 2026]. Um experimento direto deve comparar um pipeline federativo com uma alternativa injetiva ou de ponta a ponta nas mesmas tarefas, dados e escala de modelo. Os benefícios relevantes podem aparecer menos na precisão bruta do que na auditabilidade, abstenção, localização de erros e reparo.

O estudo deve, portanto, relatar várias dimensões. A precisão da tarefa crua continua a ser necessária. Deve ser acompanhada por precisão semântica-forma, precisão climatizada do solver, calibração desconhecida, reprodutibilidade entre as versões do modelo, fidelidade de explicação, tempo de revisão humana e a fração de erros atribuíveis à extração, fundamentação, compilação, execução ou realização. Um sistema modular que corresponda em vez de bater a precisão de ponta a ponta ainda pode ser preferível para um trabalho de alta garantia se transformar erros silenciosos em artefatos diagnosticáveis. Por outro lado, a modularidade não deve ser defendida como uma virtude intrínseca se apenas adicionar burocracia sem melhorar a evidência.

Uma sétima questão de pesquisa diz respeito à interpretação abstrata sobre alternativas semânticas. Um domínio abstrato útil pode ser definido que é conservador em relação a um conjunto de interpretação de referência e mais escalável do que a enumeração explícita? Os primeiros experimentos podem usar conjuntos finitos como referência concreta. Domínios abstratos para identidade, intervalos temporais, modalidade, quantidades e cobertura são implementados separadamente e depois combinados. A sonoridade é verificada mecanicamente contra a enumeração em casos pequenos. A precisão é medida pela taxa de achados que podem ser eliminados por busca ou refinamento de testemunhas.

Este programa deve ter cuidado com a palavra solidez. Se a anotação especializada define um conjunto de interpretação finita, o intérprete abstrato pode ser sólido em relação a esse conjunto. Não pode, assim, provar que o conjunto esgota o significado humano. A contribuição é uma semântica condicional executável e evidências sobre sua utilidade. Um artigo teórico posterior pode definir a relação de admissibilidade, funções de abstração e concretização, condições de transferência e operadores de refinamento. O sistema empírico não deve esperar por uma semântica completa da linguagem, mas deve explicitar a natureza condicional de suas reivindicações.

Uma oitava questão de pesquisa diz respeito à execução simbólica sobre programas semânticos. A construção de testemunhas sensíveis ao caminho melhora a compreensão do revisor e a descoberta de defeitos em relação aos resultados das regras estáticas? Em um fluxo de trabalho ou domínio de política, o tempo de execução pode comparar um avaliador de regras simples com a exploração do caminho simbólico. O avaliador diz que uma violação é possível; o executor retorna uma sequência concreta de eventos e atribuições. Os revisores dizem se a testemunha é inteligível e útil. Defeitos da verdade no solo e mutantes semeados determinam se a testemunha corresponde a um caso real.

A explosão do caminho deve fazer parte da avaliação, em vez de ser tratada como um inconveniente de implementação. O estudo deve variar ramificação, limites de loop, alternativas de identidade e escolhas temporais. A orientação LLM pode priorizar caminhos, como sugere o recente trabalho de execução simbólica guiado pelo LLM, mas o mecanismo deve distinguir a cobertura heurística da completude formal. As métricas de cobertura devem relatar estados

explorados, estados poados, tempo limite de resolução e comportamento externo não suportado. Uma testemunha é uma forte evidência para a existência; a falta de encontrar não é evidência de ausência, a menos que a exploração ligada e o modelo justifiquem essa conclusão.

A arquitetura de referência deve ser em camadas. A camada de origem testa a ingestão e a ancoragem. A camada de observação testa extração e status. A camada de fundamentação testa identidade, tempo, modalidade, quantidades e autoridade. A camada de formalização testa equivalência semântica e validade de idioma alvo. A camada de execução testa saídas e certificados do solver. A camada de resultado testa a composição e a incerteza do veredicto. A camada de realização testa a fidelidade de respostas em linguagem natural. Cada exemplo deve identificar a camada mais antiga em que uma falha injetada deve ser detectada.

Os casos de benchmark devem ser pequenos o suficiente para validar manualmente e composável em documentos mais longos. Pequenos exemplos fornecem uma verdade de base confiável para a lógica sutil. Os compostos longos testam a recuperação, a indexação, a continuidade da identidade e a escalabilidade. A mesma regra subjacente pode aparecer em várias formas linguísticas e estruturas de documentos. Parafrasear famílias testam invariância; os conjuntos de contraste mudam uma característica semântica e a sensibilidade ao teste. O benchmark deve evitar a falha comum em que um sistema funciona bem memorizando padrões de superfície, ignorando a distinção pretendida.

O teste de mutação deve ser central. As mutações podem alterar um quantificador, negar uma condição, mover uma exceção, alterar um limite de data, mesclar identidades, excluir evidências, alterar a força modal, trocar de agente e objeto, duplicar um requisito ou modificar um limite de circuito. Um conjunto robusto deve detectar a mudança esperada e deixar os resultados não relacionados estáveis. Pontuação de mutação não é uma medida de qualidade completa, mas revela testes que apenas confirmam um caminho feliz. Para nllAgent, as mutações executáveis abordariam diretamente uma das lacunas reconhecidas do repositório [AssistOS-AI 2026].

A avaliação deve incluir injeção rápida e adversários semânticos. Os documentos podem conter instruções direcionadas ao modelo, títulos enganosos, texto de política citado, Unicode adversário, conteúdo oculto ou declarações que se assemelham a declarações de ontologia. O analisador de origem deve preservá-los como dados. Os produtores semânticos devem funcionar sob a autoridade do sistema fixo. O benchmark deve testar se o texto do documento pode alterar o circuito, suprimir descobertas ou causar chamadas de ferramentas privilegiadas. Os resultados de segurança devem ser registrados separadamente da precisão semântica.

A avaliação do modelo ao vivo deve ser reproduzível o suficiente para apoiar alegações científicas. Cada corrida deve fixar o provedor, o identificador do modelo, solicitar esquema, temperatura, semente quando suportado, construção de contexto e política de repetição. Entradas e saídas devem ser capturadas sujeitas a restrições de privacidade. Os ensaios repetidos estimam a variância. Os duplos determinísticos permanecem úteis para testes de software, mas não podem estabelecer qualidade de tradução. Pelo menos duas famílias modelo devem ser testadas quando possível para distinguir os efeitos arquitetônicos do comportamento de um provedor.

A avaliação humana é necessária, mas deve ser projetada de forma restrita. Especialistas não devem ser questionados se uma resposta “parece bem”. Eles devem julgar obrigações específicas: se a formalização preserva a fonte, se as identidades são justificadas, se uma

testemunha é viável, se a explicação afirma todas as suposições materiais e se uma desconhecida é adequadamente devolvida. O desacordo interavali é em si uma evidência de ambiguidade semântica e não deve ser forçado a um único rótulo de ouro sem análise.

As métricas quantitativas centrais devem incluir acurácia fonte-âncora; precisão de observação e recordação por status; ontologia e validade do tipo; precisão, recordação e calibração de identidade-link; precisão da relação temporal e modal; cobertura do candidato; pode/deve solidez em relação às interpretações anotadas; sintaxe de formalização e equivalência semântica; sucesso do solver; validade da testemunha; sucesso da prova-verificação; taxa de garantia falsa; taxa de garantia apropriada; abstenção adequada; cobertura; conversão de custos; conversão de custos. Uma única pontuação agregada ocultaria o mecanismo e deveria ser secundária.

A falsa garantia merece uma ênfase especial. Ele mede os casos em que o sistema apresenta uma conclusão determinada verificada, embora a fonte, a fundamentação ou o conjunto de interpretação não o justifique. Em aplicações de alta segurança, a redução da falsa garantia pode importar mais do que maximizar as perguntas respondidas. O sistema deve receber crédito por um estado de parada preciso quando um LLM direto inventa com confiança uma ponte. Desconhecido apropriado é um resultado positivo.

As linhas de base devem incluir atendimento direto LLM, prompting de raciocínio de cadeia ou equivalente, execução LLM com reforço de ferramentas sem artefatos formais persistentes, tradução de monoformalismo, tradução de candidato-conjunto sem interpretação abstrata, execução simbólica sem refinamento semântico e arquitetura completa. Para o estudo de caso nllAgent, a atual versão da fundação limitada pode servir como uma linha de base implementada, enquanto as extensões de identidade, abstrato-semântico e multiformalismo são avaliadas de forma incremental. Isso evita comparar uma visão de pesquisa completa apenas contra sistemas externos mais fracos.

O programa de publicação pode ser organizado em torno de cinco contribuições, em vez de uma reivindicação de grandes dimensões. O primeiro artigo deve ser um papel de posição e arquitetura que defina a linguagem natural executável, o limite da confiança, a lacuna de formalização, o envelope da solidez e o design do multiformalismo. Sua contribuição é clareza conceitual e uma arquitetura de referência, apoiada por um levantamento sistemático e um pequeno demonstrador de trabalho.

O segundo artigo deve formalizar o pivô semântico e a solidez condicional. Ele define objetos de origem, observações digitadas, interpretações de candidatos, semântica de maio/deve e compatibilidade. Isso prova a solidez da computação abstrata em relação a um conjunto de interpretação finito admissível e relata experimentos sobre identidade, tempo, modalidade e cobertura. Este é o núcleo de linguagem de programação e de semântica.

O terceiro artigo deve introduzir o refinamento semântico guiado por contra-exemplos. Ele define mapeamentos de conflitos de solver para compromissos semânticos ligados a origem e compara o refinamento com a auto-reflexão e o feedback de execução genérico. O resultado mais forte seria menor falsa garantia, reparos menores, melhor preservação de regressão e esforço humano reduzido. Mesmo um resultado negativo seria valioso se ele identifica qual feedback do solucionador é muito baixo para orientar o reparo semântico.

O quarto trabalho deve apresentar o portfólio de formalismo e os contratos de motores tipados. Ele avalia adaptadores relacionais, Datalog, mesa de decisão, SMT, temporal e de

execução simbólica em um modelo de artefato comum. O artigo deve enfatizar a preservação semântica e os certificados, em vez do número de integrações. Um pequeno conjunto de motores genuinamente diferentes é mais persuasivo do que muitos invólucros rasos.

O quinto artigo deve ser o sistema nllAgent e o papel de avaliação. Deve distinguir claramente a liberação implementada do tempo de execução mais amplo. Ele pode documentar revisões de fontes imutáveis, LongTextJS, CircuitJS, observações orientadas para a demanda, compatibilidade, cobertura, execução restrita, aprendizado e governança de publicação, verificação de repetição, auditoria canônica e produtos de plano e o processo de questões sérias. A avaliação deve incluir mutação executável, registros de modelo ao vivo, casos de identidade, governança de pacotes do mundo atual, benchmarks de planejamento e um verificador independente ou implementado separadamente, quando possível.

Um sexto artigo posterior poderia estudar a prova de realização restrita. Ele compararia geração direta, renderização de modelos, renderização LLM restrita e verificação post-hoc. A contribuição central seria uma representação de licenciamento de reivindicações e evidência de que a linguagem legível pode ser gerada sem alterar a modalidade, a incerteza ou o conteúdo factual. Este trabalho conecta a execução formal à necessidade prática de comunicação humana acessível.

O projeto deve definir critérios explícitos de falha. A abordagem de conjunto de candidatos deve ser reconsiderada se produzir tantas alternativas que podem / devem os resultados não são informativos e o refinamento não pode recuperar a precisão. O portfólio de formalismo deve ser reduzido se o roteamento e o bugs de formacionalismo excederem os benefícios da especialização. A formalização assistida por LLM deve ser restrita ao apoio de autoria se a equivalência semântica permanecer pobre sob documentos realistas. A realização restrita à prova deve cair de volta para modelos se a prosa gerada por modelo exceder repetidamente sua licença. O raciocínio atual deve permanecer fora do núcleo confiável se os controles de origem, hora, jurisdição e conflito não puderem ser aplicados.

A honestidade científica também requer comparação com alternativas mais simples. Muitas verificações de documentos podem ser resolvidas por analisadores determinísticos, expressões regulares, validação de esquema, consultas de banco de dados ou regras de autoria humana. Um LLM é justificado quando a variação de linguagem, o mapeamento de ontologias ou a proposta semântica reduzem o custo de autoria ou aumentam a cobertura. Um núcleo simbólico é justificado quando exato estado, repetição, testemunhas ou verificação independente. Um trabalho de pesquisa deve relatar quando um sistema mais simples funciona igualmente bem. Linguagem natural executável não é uma razão para formalizar cada preferência editorial ou inserir um solucionador em tarefas que precisam apenas de um analisador confiável.

O resultado prático provável não é uma máquina que executa toda a linguagem natural. É uma família de compiladores semânticos governados para domínios de documentos e agentes, compartilhando fonte, fundamentação, execução e infraestrutura de certificado. Cada compilador tem um envelope semântico declarado. Os LLMs torna esses compiladores mais fáceis de criar, adaptar e explicar. Os métodos formais tornam seus resultados aceitos reproduzíveis e contestáveis. A combinação é útil precisamente porque nenhum componente é solicitado a fornecer o que não pode fornecer de forma confiável sozinho.

O programa de pesquisa é, portanto, mais modesto e mais consequente do que o slogan “linguagem natural como código”. A linguagem natural permanece um meio aberto, social,

dependente do contexto. O tempo de execução cria projeções executáveis para perguntas declaradas. Essas projeções podem ser rigorosas o suficiente para apoiar auditorias reais, planos, protocolos e verificações científicas, desde que o sistema registre a distância entre a fonte e o objeto formal, em vez de fingir que a distância desapareceu.

Conclusão: Da Forma Probabilística à Consequência Simbólica

O recente renascimento do AI neuro-simbólico produziu um padrão arquitetônico reconhecível. Grandes modelos de linguagem são usados onde o espaço de pesquisa é linguístico, heterogêneo ou difícil de enumerar. Sistemas formais são usados onde o estado exato, a consistência, a busca limitada exaustiva, as testemunhas ou as provas são importantes. A questão interessante não é mais se os métodos neurais e simbólicos podem ser combinados. Eles claramente podem. A questão importante é onde a autoridade está e o que é verificado de forma independente.

O inquérito mostra que os sistemas mais credíveis não concedem ao modelo de linguagem a autoridade final sobre o resultado. Os idiomas controlados traduzem uma superfície restrita em semântica formal. Os analisadores semânticos geram consultas executáveis ou programas. PAL delega computação para Python. SatLM delega restrições a um solver. A dedução de desalgate da lógica-LM e do LINC aos motores lógicos. NL2TL e nl2spec traduzem requisitos para especificações temporais. Os sistemas de planejamento usam modelos formais e ferramentas de verificação para manter a consistência do horizonte longo. O SymGPT compila regras ERC em linguagem natural em restrições e usa a execução simbólica para construir caminhos de contrato concretos. A SAIL usa LLMs para procurar transformadores abstratos, mas admite apenas candidatos que satisfaçam as obrigações mecânicas. Esses sistemas diferem muito, mas convergem no princípio de que a geração propõe e um mecanismo independente decide.

Esta convergência não deve ser exagerada. O uso do Solver não verifica a compreensão. A lacuna de formalização continua a ser o problema central. Um provador de teorema pode derivar uma conclusão válida de predicados que deturpam o texto. Um programa pode retornar a resposta esperada através de um caminho espúrio. Um modelo SMT pode satisfazer restrições que omitem o requisito decisivo do usuário. Um executor simbólico pode expor um caminho de software real para uma regra mal traduzida. Os sistemas de corrente mais fortes reduzem o erro dedutivo e computacional ao realocar o risco para análise, fundamentação, fundamentação e escopo semânticos.

A resposta apropriada é não abandonar as ferramentas formais porque a tradução é imperfeita. É para fazer a tradução um artefato persistente, digitado e analisável. A fonte deve permanecer disponível. Toda observação semântica deve identificar seu produtor, evidência, status e ontologia. Identidade, tempo, modalidade, quantidades e autoridade devem ser representados em vez de inferidos novamente durante cada resposta. As interpretações alternativas devem permanecer alternativas quando a distinção afeta o resultado. Um motor simbólico deve executar apenas objetos aterrados que atendam aos contratos explícitos.

A interpretação abstrata fornece uma teoria útil para o lado global desta arquitetura. Ele pergunta o que pode ser concluído conservadoramente a partir de um grande conjunto de estados possíveis. Aplicados à linguagem natural, os estados concretos não são “significados” indefinidos, mas estruturas semânticas limitadas admissíveis sob uma política declarada de gramática, ontologia, a revisão de fontes e a base. Um estado abstrato pode então representar o que deve ser mantido, pode conter, não pode conter, conflitos ou permanecer desconhecido.

A garantia é condicionada ao conjunto de interpretação, mas a solidez condicional ainda é significativa quando as condições são explícitas e testadas.

A execução simbólica fornece o complemento local. Quando uma análise abstrata relata uma possível violação, a execução simbólica ou outro mecanismo de restrição pode procurar uma interpretação e um caminho concretos. Uma pesquisa bem-sucedida retorna uma testemunha que um revisor pode inspecionar. Uma busca fracassada expõe uma abstração excessivamente grosseira ou uma escolha semântica incompatível. Mapear essa falha de volta aos vãos de origem cria um refinamento semântico guiado por contra-exemplos: o modelo ou o humano revisa o menor compromisso formal relevante e a análise é repetida. Este é um uso mais disciplinado do feedback LLM do que a autocritica genérica, porque o sinal de erro está vinculado a uma dependência formal.

Nenhum idioma de destino único é susceptível de suportar todas as formas úteis de prosa executável. A literatura já demonstra a dependência da escolha de linguagem intermediária, e os sistemas contemporâneos conseguem usando Python, SQL, lógica de primeira ordem, lógica temporal, PDDL, linguagens de restrição ou gramáticas específicas de domínio de acordo com a tarefa. Um tempo de execução prático deve, por conseguinte, utilizar um pivô semântico com proveniência e uma carteira de motores formais. Consultas e fechamento de documentos de avaliação relacional e Datalog lidam com consultas e fechamento; tabelas de decisão lidam com políticas finitas; SAT / SMT / CSP lidando com restrições de interação; a verificação de modelo lida com comportamentos temporais; execução simbólica lida com testemunhas de caminho; automatas manipulam padrões ordenados; alças de lógica natural implicam que os assistentes de prova lidam com reivindicações formais de alto valor.

A pluralidade introduz seus próprios riscos de engenharia. Cada compilador pode distorcer a semântica, cada mecanismo expande a base confiável e o roteamento pode falhar. O portfólio é justificado apenas se as interfaces são digitadas, os símbolos compartilhados mantêm identidade e proveniência, as garantias do motor são explícitas e os resultados são normalizados em um produto canônico de certificado. Uma lógica universal pode permanecer preferível em um domínio restrito. A alegação arquitetônica não é que a pluralidade é sempre melhor; é que a ampla linguagem executável requer diferentes estruturas computacionais e não deve disfarçá-las atrás de um vago DSL.

O retorno dos símbolos para a linguagem faz parte do limite da confiança. Um LLM final não deve ser permitido transformar um resultado condicional em uma recomendação incondicional, uma possível violação em uma violação confirmada ou evidência ausente em evidência de ausência. O resultado canônico deve definir um conjunto finito de reivindicações licenciadas com modalidade, quantidades, entidades, pressupostos e proveniência. O renderizador pode melhorar a legibilidade, mas deve permanecer subordinado a essa autoridade. A realização com restrição de provas é a conclusão natural da arquitetura.

A implementação pública do nllAgent é significativa porque já incorpora várias dessas disciplinas. O LongTextJS preserva um programa de documentos fundamentado de origem; CircuitJS representa teorias reutilizáveis; a demanda de observação é derivada de circuitos; compatibilidade e cobertura são explícitas; módulos restritos e extensões revisadas limitam a execução oculta; o aprendizado ocorre em encenação descartável; a publicação é manual e imutável; a verificação de repetição de replay controla as descobertas canônicas; e o conhecimento ausente não é tratado como conformidade. Essas decisões tornam o projeto

uma plataforma credível para a pesquisa, mesmo que não estabeleçam a compreensão semântica geral.

Os mesmos materiais públicos identificam o importante trabalho inacabado: materialização geral da identidade, contratos primitivos tipados completos, mutação semântica executável, avaliação independente de sombra, benchmarking de modelo ao vivo, benchmarks de planejamento e realização e pacotes de conhecimento do mundo atual. Nenhuma avaliação independente peer-reviewed do nllAgent foi localizada para esta edição. O julgamento oferecido aqui é, portanto, arquitetônico: a implementação escolheu limites excepcionalmente sólidos, mas suas reivindicações mais fortes defensáveis permanecem limitadas pela análise estrutural e controlada. Seu valor aumentará se a amplitude semântica for expandida apenas por meio de produtores explícitos, contratos, benchmarks e evidências.

O próximo passo proposto não é substituir LongTextJS ou CircuitJS por uma linguagem universal maior. LongTextJS deve amadurecer como um mundo semântico, queificável e composto por proveniência. CircuitJS deve continuar a ser uma orquestração restrita e camada de contrato de evidência. Os mecanismos registrados especializados devem executar uma execução relacional, constrangimento, temporal, fluxo de dados, autômato ou execução sensível ao caminho. Maio/deve a semântica deve tornar-se operacional. Os contraexemplos devem conduzir o refinamento semântico focado. Os produtos canônicos CNL devem controlar a saída final da linguagem.

Um programa de pesquisa rigoroso pode testar cada parte. A semântica do site pode ser comparada com sistemas de tradução única. O refinamento derivado do Solver pode ser comparado com a auto-reflexão. Os contratos de evidência tipados podem ser testados quanto à transferência e garantia falsa. Um portfólio de formalismo pode ser comparado com uma linguagem intermediária universal. Domínios abstratos podem ser verificados em relação a conjuntos de interpretação explícita. Testemunhas simbólicas podem ser avaliadas quanto à descoberta de defeitos e utilidade do reviso. A realização com restrição de provas pode ser testada para a fidelidade. Arquiteturas modulares e de ponta a ponta podem ser comparadas nas mesmas tarefas e não em diferentes famílias de referência.

A métrica decisiva deve ser falsa garantia, não apenas taxa de resposta. Um sistema que retorna “desconhecido porque a identidade não é resolvida” pode ser mais valioso do que aquele que produz um veredicto confiante ao fundir silenciosamente duas pessoas. Um sistema que pára porque a evidência do mundo atual é obsoleta é mais confiável do que uma que recupera um fato sem suporte. Um sistema que expõe três interpretações é melhor do que aquele que escolhe o mais fluente. Absorção adequada é parte da semântica de execução.

A linguagem natural executável deve, portanto, ser entendida como uma relação de compilação governada, não como uma propriedade intrínseca da prosa. O texto de origem permanece mais rico do que qualquer projeção executável. O sistema seleciona uma tarefa, materializa a semântica relevante, preserva a incerteza, fundamenta os símbolos, compila para um formalismo apropriado, executa sob limites explícitos, verifica o resultado e retorna uma explicação restrita. O que se torna executável não é a linguagem em sua totalidade, mas uma projeção semântica declarada cujas suposições e limites permanecem visíveis.

Esta concepção mais estreita é suficiente para um impacto prático substancial. As reivindicações científicas podem ser verificadas quanto à proveniência e consistência. Os requisitos podem ser vinculados a testes e estados de ciclo de vida. Políticas podem ser compiladas em estruturas de decisão. Os planos podem ser verificados antes da realização. As

regras de protocolo podem ser mapeadas para análises de estado e caminho. Documentos longos podem ser marcados por teorias reutilizáveis sem pedir a um modelo que se lembre de cada detalhe. Os agentes podem usar a linguagem natural como uma camada de criação e coordenação, ao mesmo tempo em que delegam consequências exatas em tempos simbólicos.

O princípio central pode ser declarado sem simbolismo formal: deixe o LLM descobrir a forma possível; deixe o kernel de fundamentação decidir o que os símbolos podem denotar; deixe os motores formais determinarem o que se segue; e deixe o LLM final explicar apenas o que o registro simbólico licencia. Este princípio não resolve todos os problemas da linguagem. Ele cria uma fronteira defensável entre interpretação e consequência, que é a base necessária para que a linguagem natural participe com segurança em sistemas executáveis.

Referências

- Abzianidze, Lasha, Rik van Noord, Hessel Haagsma, and Johan Bos. 2019. “The First Shared Task on Discourse Representation Structure Parsing and Evaluation.” Proceedings of the IWCS Shared Task on Semantic Parsing. Association for Computational Linguistics. <https://aclanthology.org/W19-1201/>
- Abzianidze, Lasha. 2017. “LangPro: Natural Language Theorem Prover.” Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing: System Demonstrations. Association for Computational Linguistics. <https://aclanthology.org/D17-2022/>
- Angeli, Gabor, and Christopher D. Manning. 2014. “NaturalLI: Natural Logic Inference for Common Sense Reasoning.” Proceedings of EMNLP 2014. Association for Computational Linguistics. <https://aclanthology.org/D14-1209/>
- AssistOS-AI. 2026. “NaturalLanguageLinterAgent (nllAgent): LongTextJS–CircuitJS Architecture, Documentation, and Serious Issues.” Public repository and generated documentation, inspected 2 August 2026. <https://github.com/assistos-ai/nllAgent>
- Badreddine, Samy, Artur d’Avila Garcez, Luciano Serafini, and Michael Spranger. 2022. “Logic Tensor Networks.” Artificial Intelligence 303: 103649. <https://doi.org/10.1016/j.artint.2021.103649>
- Baldoni, Roberto, Emilio Coppa, Daniele Cono D’Elia, Camil Demetrescu, and Irene Finocchi. 2018. “A Survey of Symbolic Execution Techniques.” ACM Computing Surveys 51(3): 1–39. <https://doi.org/10.1145/3182657>
- Beg, Arshad, Diarmuid P. O’Donoghue, and Rosemary Monahan. 2025. “A Short Survey on Formalising Software Requirements Using Large Language Models.” arXiv:2506.11874. <https://doi.org/10.48550/arXiv.2506.11874>
- Beiser, Alexander, and David Penz. 2025. “Making LLMs Reason? The Intermediate Language Problem in Neurosymbolic Approaches.” arXiv:2502.17216. <https://arxiv.org/abs/2502.17216>
- Bos, Johan. 2008. “Wide-Coverage Semantic Analysis with Boxer.” Proceedings of the 2008 Conference on Semantics in Text Processing. <https://aclanthology.org/W08-2222/>
- Chatzikyriakidis, Stergios, and Shalom Lappin. 2026. “Neuro-Symbolic NLP: Taxonomy, Assessment, and Directions.” Frontiers in Artificial Intelligence 9. <https://doi.org/10.3389/frai.2026.1797587>
- Chen, Lingfeng, Tao Xiao, Masanari Kondo, and Yasutaka Kamei. 2026. “Directed Symbolic Execution for Vulnerability Discovery: An LLM-Guided Approach in KLEE.” arXiv:2607.21676. <https://arxiv.org/abs/2607.21676>
- Chen, Yongchao, Rujul Gandhi, Yang Zhang, and Chuchu Fan. 2023. “NL2TL: Transforming Natural Languages to Temporal Logics Using Large Language Models.” Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics. <https://aclanthology.org/2023.emnlp-main.985/>
- Clarke, Edmund, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. 2000. “Counterexample-Guided Abstraction Refinement.” Computer Aided Verification, LNCS 1855, 154–169. https://doi.org/10.1007/10722167_15
- Cosler, Matthias, Christopher Hahn, Daniel Mendoza, Frederik Schmitt, and Caroline Trippel. 2023. “nl2spec: Interactively Translating Unstructured Natural Language to Temporal Logics with Large Language Models.” Computer Aided Verification, LNCS 13964, 383–396. https://doi.org/10.1007/978-3-031-37703-7_18
- Cousot, Patrick, and Radhia Cousot. 1977. “Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints.” Proceedings of POPL 1977, 238–252. <https://doi.org/10.1145/512950.512973>
- Delvecchio, Giovanni Pio, Lorenzo Molfetta, and Gianluca Moro. 2025. “Neuro-Symbolic Artificial Intelligence: A Task-Directed Survey in the Black-Box Models Era.” Proceedings of IJCAI 2025, Survey Track, 10418–10426. <https://doi.org/10.24963/ijcai.2025/1157>
- Dong, Wenkai, and Yifan Wang. 2026. “From Interpretation to Compilation: Compilation-Based Execution of Semantic Operators.” arXiv:2607.13407. <https://doi.org/10.48550/arXiv.2607.13407>
- Ferrari, Alessio, and Paola Spoletini. 2025. “Formal Requirements Engineering and Large Language Models: A Two-Way Roadmap.” Information and Software Technology 181: 107697. <https://doi.org/10.1016/j.infsof.2025.107697>

- Fuchs, Norbert E., and Rolf Schwitter. 1996. "Attempto Controlled English (ACE)." Proceedings of the First International Workshop on Controlled Language Applications. <https://attempto.ifi.uzh.ch/site/pubs/papers/ace96.pdf>
- Fuchs, Norbert E., Kaarel Kaljurand, and Tobias Kuhn. 2008. "Attempto Controlled English for Knowledge Representation." Reasoning Web, LNCS 5224, 104–124. https://doi.org/10.1007/978-3-540-85658-0_3
- Gao, Luyu, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2022. "PAL: Program-Aided Language Models." arXiv:2211.10435. <https://arxiv.org/abs/2211.10435>
- Goldman, Omer, Veronica Latcinnik, Udi Naveh, Amir Globerson, and Jonathan Berant. 2018. "Weakly Supervised Semantic Parsing with Abstract Examples." Proceedings of ACL 2018. Association for Computational Linguistics. <https://aclanthology.org/P18-1168/>
- Grimaldi, Pasquale. 2026. "pamaldi at SemEval-2026 Task 11: Neuro-Symbolic Syllogistic Reasoning via LLM-Guided Structure Extraction and Deterministic Validation." Proceedings of the 20th International Workshop on Semantic Evaluation. Association for Computational Linguistics. <https://doi.org/10.18653/v1/2026.semeval-1.186>
- Gu, Qiuhan, Avaljot Singh, and Gagandeep Singh. 2026. "SAIL: Sound Abstract Interpreters with LLMs." Proceedings of the ACM on Programming Languages 10 (PLDI): 1534–1559. <https://doi.org/10.1145/3808308>
- Guu, Kelvin, Panupong Pasupat, Evan Liu, and Percy Liang. 2017. "From Language to Programs: Bridging Reinforcement Learning and Maximum Marginal Likelihood." Proceedings of ACL 2017. Association for Computational Linguistics. <https://aclanthology.org/P17-1016/>
- Hamilton, Kyle, Aparna Nayak, Bojan Božić, and Luca Longo. 2024. "Is Neuro-Symbolic AI Meeting Its Promises in Natural Language Processing? A Structured Review." Semantic Web 15(4): 1265–1306. <https://doi.org/10.3233/SW-223228>
- Han, Simeng, Hailey Schoelkopf, Yilun Zhao, Zhenting Qi, Martin Riddell, Wenfei Zhou, James Coady, David Peng, Yujie Qiao, Luke Benson, Lucy Sun, Alex Wardle-Solano, Hannah Szabó, Ekaterina Zubova, Matthew Burtell, Jonathan Fan, Yixin Liu, Brian Wong, Malcolm Sailor, Ansong Ni, Linyong Nan, Jungo Kasai, Tao Yu, Rui Zhang, Alexander Fabbri, Wojciech Kryściński, Semih Yavuz, Ye Liu, Xi Victoria Lin, Shafiq Joty, Yingbo Zhou, Caiming Xiong, Rex Ying, and Dragomir Radev. 2024. "FOLIO: Natural Language Reasoning with First-Order Logic." Proceedings of EMNLP 2024. Association for Computational Linguistics. <https://aclanthology.org/2024.emnlp-main.1229/>
- Hao, Yilun, Yongchao Chen, Yang Zhang, and Chuchu Fan. 2025. "Large Language Models Can Solve Real-World Planning Rigorously with Formal Verification Tools." Proceedings of NAACL 2025, 3434–3483. <https://aclanthology.org/2025.naacl-long.176/>
- Hu, Ruikang, Shaoyu Lin, Yeliang Xiu, and Yongmei Liu. 2025. "LTRAG: Enhancing Autoformalization and Self-refinement for Logical Reasoning with Thought-Guided RAG." Findings of ACL 2025, 2483–2493. <https://aclanthology.org/2025.findings-acl.126/>
- Huang, Pei, Jiayu Li, Yuhao Zhang, Changliu Liu, and Shih-hsi Alex Liu. 2026. "Parameterized Abstract Interpretation for Transformer Verification." Proceedings of AAAI 2026. <https://doi.org/10.1609/aaai.v40i42.40860>
- Jia, Robin, Aditi Raghunathan, Kerem Göksel, and Percy Liang. 2019. "Certified Robustness to Adversarial Word Substitutions." Proceedings of EMNLP-IJCNLP 2019. Association for Computational Linguistics. <https://aclanthology.org/D19-1423/>
- Karthikeyan, T, Om Dehlan, Mausam, and Manish Gupta. 2025. "LRPLAN: A Multi-Agent Collaboration of Large Language and Reasoning Models for Planning with Implicit and Explicit Constraints." Findings of EMNLP 2025, 8280–8310. <https://doi.org/10.18653/v1/2025.findings-emnlp.440>
- Kartáč, Ivan, Kristýna Onderková, Jan Bronec, Zdeněk Kasner, Mateusz Lango, and Ondřej Dušek. 2026. "UFAL-CUNI at SemEval-2026 Task 11: An Efficient Modular Neuro-Symbolic Method for Syllogistic Reasoning." Proceedings of the 20th International Workshop on Semantic Evaluation. Association for Computational Linguistics. <https://aclanthology.org/2026.semeval-1.418/>
- King, James C. 1976. "Symbolic Execution and Program Testing." Communications of the ACM 19(7): 385–394. <https://doi.org/10.1145/360248.360252>
- Kirtania, Shashank, Priyanshu Gupta, and Arjun Radhakrishna. 2024. "LOGIC-LM++: Multi-Step Refinement for Symbolic Formulations." Proceedings of the 2nd Workshop on Natural Language Reasoning and Structured Explanations, 56–63. <https://aclanthology.org/2024.nlrse-1.6/>

- Kuhn, Tobias. 2014. "A Survey and Classification of Controlled Natural Languages." *Computational Linguistics* 40(1): 121–170. https://doi.org/10.1162/COLI_a_00168
- Lalwani, Vivek, et al. 2025. "Autoformalizing Natural Language to First-Order Logic: A Case Study in Logical Fallacy Detection." *Findings of IJCNLP-AACL 2025*, 132–147. <https://aclanthology.org/2025.findings-ijcnlp.8/>
- Lee, Kang-il, Segwang Kim, and Kyomin Jung. 2023. "Weakly Supervised Semantic Parsing with Execution-based Spurious Program Filtering." *Proceedings of EMNLP 2023*, 6884–6894. <https://aclanthology.org/2023.emnlp-main.425/>
- Li, Yihe, Ruijie Meng, and Gregory J. Duck. 2025. "Large Language Model Powered Symbolic Execution." *Proceedings of the ACM on Programming Languages* 9 (OOPSLA2), Article 385, 3148–3176. <https://doi.org/10.1145/3763163>
- Liang, Chen, Jonathan Berant, Quoc Le, Kenneth D. Forbus, and Ni Lao. 2017. "Neural Symbolic Machines: Learning Semantic Parsers on Freebase with Weak Supervision." *Proceedings of ACL 2017*. Association for Computational Linguistics. <https://aclanthology.org/P17-1003/>
- Liu, Jiangming, Shay B. Cohen, and Mirella Lapata. 2018. "Discourse Representation Structure Parsing." *Proceedings of ACL 2018*. Association for Computational Linguistics. <https://aclanthology.org/P18-1040/>
- Locascio, Nicholas, Karthik Narasimhan, Eduardo DeLeon, Nate Kushman, and Regina Barzilay. 2018. "Neural Generation of Regular Expressions from Natural Language with Minimal Domain Knowledge." *Proceedings of EMNLP 2018*. Association for Computational Linguistics. <https://aclanthology.org/D18-1436/>
- MacCartney, Bill, and Christopher D. Manning. 2007. "Natural Logic for Textual Inference." *Proceedings of the ACL-PASCAL Workshop on Textual Entailment and Paraphrasing*. <https://aclanthology.org/W07-1401/>
- MacCartney, Bill, and Christopher D. Manning. 2009. "An Extended Model of Natural Logic." *Proceedings of IWCS 2009*. <https://aclanthology.org/W09-3714/>
- Manhaeve, Robin, Sebastijan Dumančić, Angelika Kimmig, Thomas Demeester, and Luc De Raedt. 2018. "DeepProbLog: Neural Probabilistic Logic Programming." *Advances in Neural Information Processing Systems* 31. <https://proceedings.neurips.cc/paper/2018/hash/dc5d637ed5e62c36ecb73b654b05ba2a-Abstract.html>
- Mao, Ziyu, Jingyi Wang, Jun Sun, Shengchao Qin, and Jiawen Xiong. 2025. "LLM-Aided Automatic Modeling for Security Protocol Verification." *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering*, 2025. https://ink.library.smu.edu.sg/sis_research/10297/
- Mitchell, Jacqueline L., Brian Hyeongseok Kim, Chenyu Zhou, and Chao Wang. 2025. "Can LLMs Formally Reason as Abstract Interpreters for Program Analysis?" *arXiv:2503.12686*. <https://arxiv.org/abs/2503.12686>
- Mou, Lili, Zhengdong Lu, Hang Li, and Zhi Jin. 2017. "Coupling Distributed and Symbolic Execution for Natural Language Queries." *Proceedings of ICML 2017*. <https://proceedings.mlr.press/v70/mou17a.html>
- Olausson, Theo X., Alex Gu, Benjamin Lipkin, Cedegao E. Zhang, Armando Solar-Lezama, Joshua B. Tenenbaum, and Roger Levy. 2023. "LINC: A Neurosymbolic Approach for Logical Reasoning by Combining Language Models with First-Order Logic Provers." *Proceedings of EMNLP 2023*. Association for Computational Linguistics. <https://aclanthology.org/2023.emnlp-main.313/>
- Pan, Liangming, Alon Albalak, Xinyi Wang, and William Yang Wang. 2023. "Logic-LM: Empowering Large Language Models with Symbolic Solvers for Faithful Logical Reasoning." *Findings of EMNLP 2023*. Association for Computational Linguistics. <https://aclanthology.org/2023.findings-emnlp.248/>
- Pasupat, Panupong, and Percy Liang. 2016. "Inferring Logical Forms from Denotations." *Proceedings of ACL 2016*. Association for Computational Linguistics. <https://aclanthology.org/P16-1004/>
- Pei, Yu, Yongping Du, and Xingnan Jin. 2025. "FoVer: First-Order Logic Verification for Natural Language Reasoning." *Transactions of the Association for Computational Linguistics* 13: 1340–1359. <https://aclanthology.org/2025.tacl-1.61/>
- Tafjord, Oyvind, Bhavana Dalvi Mishra, and Peter Clark. 2021. "ProofWriter: Generating Implications, Proofs, and Abductive Statements over Natural Language." *Findings of ACL-IJCNLP 2021*. Association for Computational Linguistics. <https://aclanthology.org/2021.findings-acl.317/>
- Tantakoun, Nat, Christian Muise, and Yilun Zhu. 2025. "Large Language Models as Planning Formalizers: A Survey." *Findings of ACL 2025*. <https://aclanthology.org/2025.findings-acl.1291/>
- Tziafas, Georgios, and Hamidreza Kasaei. 2026. "Neuro-Symbolic Task Planning and Object Manipulation with Large Language Models." *Autonomous Robots* 50. <https://doi.org/10.1007/s10514-026-10230-6>

- Wang, Bailin, Richard Shin, Xiaodong Liu, Oleksandr Polozov, and Matthew Richardson. 2018. “Execution-Guided Neural Program Decoding.” *Proceedings of the First Workshop on Learned Program Synthesis*. <https://arxiv.org/abs/1807.03100>
- Wang, Boxin, Chejian Xu, Xiangyu Liu, Yu Cheng, and Bo Li. 2023. “Robustness-Aware Word Embedding Improves Certified Robustness to Adversarial Word Substitutions.” *Findings of ACL 2023*. Association for Computational Linguistics. <https://aclanthology.org/2023.findings-acl.620/>
- Wang, Jiayimei, Tao Ni, Wei-Bin Lee, and Qingchuan Zhao. 2025. “A Contemporary Survey of Large Language Model Assisted Program Analysis.” *Trends in Artificial Intelligence*. <https://arxiv.org/abs/2502.18474>
- Wang, Wenguan, Yi Yang, and Fei Wu. 2025. “Towards Data- and Knowledge-Driven AI: A Survey on Neuro-Symbolic Computing.” *IEEE Transactions on Pattern Analysis and Machine Intelligence* 47(2): 878–899. <https://doi.org/10.1109/TPAMI.2024.3483273>
- Wang, Yu, You Zhang, Hao Zhang, and Dan Xu. 2026. “YNU-NLP at SemEval-2026 Task 11: A Neuro-Symbolic Approach with Reflexion Mechanism Disentangling Content and Formal Reasoning in Language Models.” *Proceedings of the 20th International Workshop on Semantic Evaluation*. Association for Computational Linguistics. <https://doi.org/10.18653/v1/2026.semeval-1.126>
- Weng, Ke, Lun Du, Sirui Li, Wangyue Lu, Haozhe Sun, Hengyu Liu, and Tiancheng Zhang. 2025. “Autoformalization in the Era of Large Language Models: A Survey.” *arXiv:2505.23486*. <https://arxiv.org/abs/2505.23486>
- Xia, Shihao, Mengting He, Shuai Shao, Tingting Yu, Yiyi Zhang, Nobuko Yoshida, and Linhai Song. 2026. “SymGPT: Auditing Smart Contracts via Combining Symbolic Execution with Large Language Models.” *Proceedings of the ACM on Programming Languages* 10 (OOPSLA1), Article 109. <https://doi.org/10.1145/3798217>
- Yang, Zonglin, et al. 2024. “Autoformalization of Natural Language Reasoning Problems to First-Order Logic.” *Proceedings of EMNLP 2024*. Association for Computational Linguistics. <https://aclanthology.org/2024.emnlp-main.1132/>
- Ye, Mao, Chengyue Gong, and Qiang Liu. 2020. “SAFER: A Structure-Free Approach for Certified Robustness to Adversarial Word Substitutions.” *Proceedings of ACL 2020*. Association for Computational Linguistics. <https://aclanthology.org/2020.acl-main.317/>
- Ye, Xi, Qiaochu Chen, Isil Dillig, and Greg Durrett. 2023. “SatLM: Satisfiability-Aided Language Models Using Declarative Prompting.” *Advances in Neural Information Processing Systems* 36. <https://arxiv.org/abs/2305.09656>
- Zheng, Zihan, Tianle Cui, Chuwen Xie, Jiahui Pan, Qianglong Chen, and Lewei He. 2025. “PlanningArena: A Modular Benchmark for Multidimensional Evaluation of Planning and Tool Learning.” *Proceedings of ACL 2025*, 31047–31086. <https://doi.org/10.18653/v1/2025.acl-long.1499>